

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

Khoa Mạng máy tính và Truyền thông



UIT

TRƯỜNG ĐẠI HỌC
CÔNG NGHỆ THÔNG TIN

BÁO CÁO THỰC HÀNH

Môn học: Mật mã học

BÁO CÁO CTF

Giảng viên hướng dẫn	NGUYỄN NGỌC TỰ
Sinh viên thực hiện	NGUYỄN DUY HÙNG

I. Mod 26 (picoCTF)

- **Challenge:** Challenge “Mod 26” thuộc dạng crypto cơ bản trong picoCTF, gợi ý trực tiếp đến việc sử dụng ROT13. Đề bài cung cấp một file chứa chuỗi ký tự đã bị mã hóa và yêu cầu người chơi giải mã để tìm flag. Vì ROT13 là một dạng Caesar Cipher đặc biệt với độ dịch 13, nên việc mã hóa và giải mã là giống nhau áp dụng lại ROT13 sẽ trả về plaintext ban đầu. Do đó, đây là bài nhằm kiểm tra kiến thức nền tảng về các kỹ thuật mã hóa cổ điển và khả năng nhận diện dạng mã hóa.

Mod 26 

Easy

Cryptography

picoCTF 2021

AUTHOR: PANDU

Hints 

Description

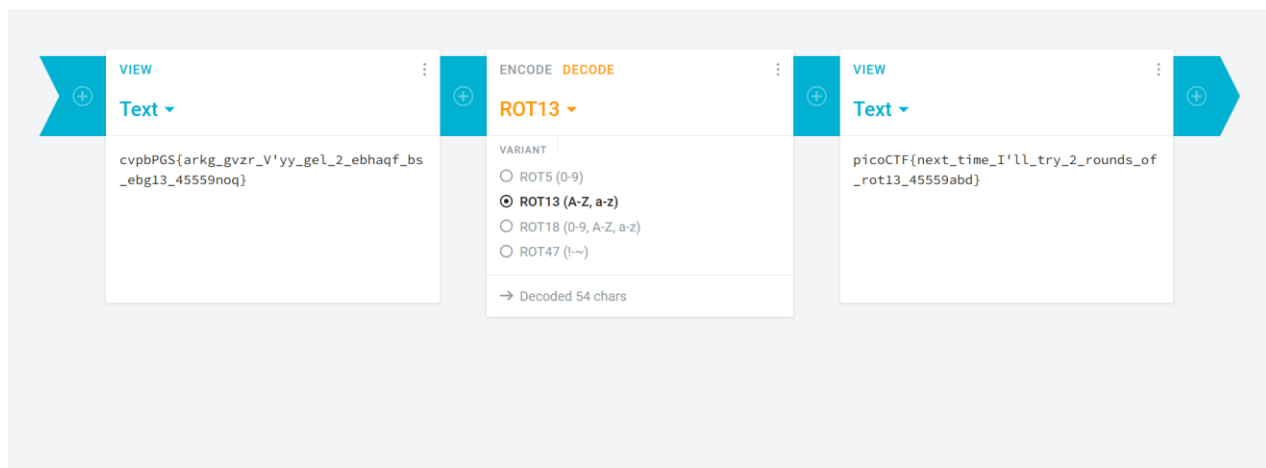
1

Cryptography can be easy, do you know what ROT13 is?

[values.txt](#)<https://play.picoctf.org/practice/challenge/144?category=2&page=1>`cvpbPGS{arkg_gvzr_v'yy_gel_2_ebhaqf_bs_ebg13_45559noq}`

Cipher text

- **Solution:** Dựa vào gợi ý của đề bài, ta xác định chuỗi đã cho được mã hóa bằng ROT13. Sử dụng công cụ Cryptii, ta tiến hành giải mã bằng cách áp dụng phép dịch 13 ký tự cho từng chữ cái trong chuỗi. Sau khi decode, ta thu được plaintext có định dạng rõ ràng của flag picoCTF.

<https://cryptii.com/>

II. Vigenere (picoCTF)

- **Challenge:** Bài này là một challenge Vigenere cipher mức Medium trong picoCTF. Đề bài cho sẵn một đoạn ciphertext và yêu cầu mình giải mã nó bằng khóa "CYLAB"

Vigenere



Medium

Cryptography

picoCTF 2022

AUTHOR: MUBARAK MIKAIL

Hints ?

Description

1

Can you decrypt this message?

Decrypt this [message](#) using this key "CYLAB".
<https://play.picoctf.org/practice/challenge/316?category=2&page=2>

```
rgnoDVD{00NU_WQ3_G1G303T3_A1AH3S_cc82272b}
```

Ciphertext

- **Solution:** Do ta sử dụng công cụ giải mã Vigenere trên trang web dCode để giải mã vì ta đã có KEY rồi nên dễ dàng có thể giải mã được.

VIGENERE CIPHER
Cryptography > Poly-Alphabetic Cipher > Vigenere Cipher

VIGENERE DECODER

★ VIGENERE CIPHERTEXT
rgnoDVD{00NU_WQ3_G1G303T3_A1AH3S_cc82272b}

★ PLAINTEXT LANGUAGE English

★ ALPHABET ABCDEFGHIJKLMNOPQRSTUVWXYZ

▶ AUTOMATIC DECRYPTION

DECRYPTION METHOD

● KNOWING THE KEY/PASSWORD: CYLAB

<https://www.dcode.fr/vigenere-cipher>

III. Substitution0 (picoCTF)

- **Challenge:** Bài substitution0 (Medium – picoCTF) là một challenge về monoalphabetic substitution cipher. Đề bài cho một đoạn thông điệp bị xáo trộn và gợi ý rằng khóa (key) nằm ở phần đầu của file.

substitution0



Medium

Cryptography

picoCTF 2022

Substitution

AUTHOR: WILL HONG

Hints ?

Description

1

A message has come in but it seems to be all scrambled.

Luckily it seems to have the key at the beginning. Can you crack this substitution cipher?

Download the message [here](#).

<https://play.picoctf.org/practice/challenge/307?category=2&page=2>

- **Solution:** Bài này khai thác điểm yếu cố hữu của **monoalphabetic substitution cipher**, cụ thể là việc mỗi chữ cái chỉ được thay thế bằng một chữ cái khác theo một **fixed substitution mapping**. Vì phép thay thế là tĩnh và không thay đổi theo vị trí, hệ mã này rất dễ bị tấn công bằng **frequency analysis attack**. Thay vì dùng key ở đầu file, ta có thể dựa vào **letter frequency distribution** của tiếng Anh, kết hợp với việc nhận diện các **common patterns**, hoặc cấu trúc của flag. Bằng cách so khớp tần suất và thử xây dựng dần **mapping hypothesis**, rồi kiểm tra tính hợp lý của plaintext thu được, ta từng bước hoàn thiện bảng ánh xạ. Do cipher này thiếu tính **diffusion** và không có cơ chế thay đổi khóa theo vị trí ký tự, nên chỉ cần đủ dữ liệu ciphertext là có thể suy ra toàn bộ mapping và giải mã hoàn chỉnh.

```
PS G:\My Drive\NT219\Demo> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe "g:/My Drive/NT219/Demo/Classic
al-Ciphers/Substitution-Techniques/Monoalphabetic-Ciphers-1-1/Simple-Substitution-Cipher/HackSimpleSubstitutionCipher.py"
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
-----
R Q O T P D N B V S J V F C A P Y W H Z E I G L U X
```

Decrypted Text:

ABCDEFGHIXVLPNOPZRSTUVWJYQ

Hereupon Legrand arose, with a grave and stately air, and brought pe the beetle
frop a glass case in which it was enclosed. It was a beautiful scarabaeus, and, at
that tife, unvknown to naturalists--of course a great pride in a scientific point
of view. There were two round blacv spots near one ejtrepity of the bacv, and a
long one near the other. The scales were ejceedingly hard and glossy, with all the
appearance of burnished gold. The weight of the insect was very reparvable, and,
taving all things into consideration, I could hardly blape Xupiter for his opinion
respecting it.

The flag is: picoCTF{5UB5717U710N_3V0LU710N_03055505}

Challenge's flag

```

1 from collections import Counter
2 import string
3
4 # Known English Letter frequencies (from most frequent to Least frequent)
5 english_frequencies = 'ETAOINSHRDLCLUMWFGYPBVKJXQZ'
6
7 cipher_text = ("
8 OHNFUMWSVZLXEGCPTAJDYIRKQB
9
10 Suauypcg Xuwaogf oacju, rvds o waoiu ogf jdoduxq ova, ogf hacywsd eu dsu huudxu
11 mace o wxojj noju vg rsvns vd roj ugnxcjuf. Vd roj o huoydmyx jnoahouy, ogf, od
12 dsod dveu, yglgcrq dc godyaoxvdj-cm ncyaju o wauod pavbu vg o jnvugdvmvn pcvgd
13 cm ivur. Dsuau ruau drc acygf hxonl jpcdj guoa cgu ukdauevdq cm dsu honl, ogf o
14 xcgw cgu guoa dsu cdsua. Dsu jnoxuj ruau uknuufvgwxq soaf ogf wxcjja, rvds oxx dsu
15 oppuaognu cm hyagvjsuf wcx. Dsu ruvwsd cm dsu vgjund roj iuaq aueoalohxu, ogf,
16 dolvgw oxx dsvgwj vgdc ncgjvfuaodvcg, V ncyxf soafxq hxoeu Zypvdua mca svj cpvgvcg
17 aujpundvgw vd.
18
19 Dsu mxow vj: pvnCNM{5YH5717Y710G_3I0XY710G_03055505}
20 """)
21
22 # Count Letter frequencies in the ciphertext (ignoring non-alphabet characters)
23 cipher_counts = Counter(''.join(filter(str.isalpha, cipher_text.upper())))
24
25 # Sort the ciphertext Letters by frequency (most frequent first)
26 sorted_cipher = ''.join([item[0] for item in cipher_counts.most_common()])
27
28 # Create an initial mapping from ciphertext Letters to the English frequency order
29 mapping = {}
30 for i, letter in enumerate(sorted_cipher):
31     mapping[letter] = english_frequencies[i]
32
33 # For any Letters not present in the ciphertext, add an identity mapping
34 for letter in string.ascii_uppercase:
35     if letter not in mapping:
36         mapping[letter] = letter
37
38 # Optional manual adjustments to improve decryption quality
39 # Sửa mapping ở đây
40 mapping["A"] = "R"
41 mapping["D"] = "T"
42 mapping["S"] = "H"
43 mapping["O"] = "A"
44 mapping["N"] = "C"
45 mapping["C"] = "O"
46 mapping["J"] = "S"
47 mapping["H"] = "B"
48 mapping["X"] = "L"
49 mapping["F"] = "D"
50 mapping["G"] = "N"
51 mapping["Q"] = "Y"
52 mapping["P"] = "P"
53 mapping["W"] = "G"
54 mapping["I"] = "V"
55
56 def print_key_mapping_table(mapping):
57     """
58     Displays the key mapping in the requested table format:
59
60     A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
61     +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
62     R A T X U K E Y H O I D V F G M P L Z W S Q J C B N
63     """
64     plain_letters = list(string.ascii_uppercase)
65     # Build the first row: plain Letters with a fixed width (2 characters per Letter)
66     row1 = " ".join(f"{letter:2}" for letter in plain_letters)
67     # Build the border row: "-+-+...+-"
68     row2 = " " + "-" + "+" + "-" + "+" + "-" * (len(plain_letters)-1) + " "
69     # Build the third row: corresponding cipher Letters from the mapping
70     row3 = " ".join(f"{mapping[letter]:2}" for letter in plain_letters)
71
72     print(row1)
73     print(row2)
74     print(row3)
75
76 # Display the final mapping using the desired table format
77 print_key_mapping_table(mapping)
78
79 # Decrypt the ciphertext using the mapping (preserving letter case)
80 decrypted_text = []
81 for char in cipher_text:
82     if char.isalpha():
83         if char.isupper():
84             decrypted_text.append(mapping.get(char, char))
85         else:
86             decrypted_text.append(mapping.get(char.upper(), char.upper()).lower())
87     else:
88         decrypted_text.append(char)
89 decrypted_text = ''.join(decrypted_text)
90
91 print("\nDecrypted Text:")
92 print(decrypted_text)
93

```

Dictionary-based substitution mapping

IV. Substitution1 (picoCTF)

- **Challenge:** Bài substitution1 (Medium – picoCTF) là một challenge về monoalphabetic substitution cipher. Đề bài cho một đoạn thông điệp bị xáo trộn nhưng khác với substitution0 là substitution1 không có gợi ý (key).

substitution1 

Medium

Cryptography

picoCTF 2022

Substitution_cipher

AUTHOR: WILL HONG

Hints ?

Description

1 2

A second message has come in the mail, and it seems almost identical to the first one. Maybe the same thing will work again.

Download the message [here](#).

<https://play.picoctf.org/practice/challenge/308?category=2&page=2>

- **Solution:** Bài này khai thác điểm yếu cố hữu của **monoalphabetic substitution cipher**, cụ thể là việc mỗi chữ cái chỉ được thay thế bằng một chữ cái khác theo một **fixed substitution mapping**. Vì phép thay thế là tĩnh và không thay đổi theo vị trí, hệ mã này rất dễ bị tấn công bằng **frequency analysis attack**. Thay vì dùng key ở đầu file, ta có thể dựa vào **letter frequency distribution** của tiếng Anh, kết hợp với việc nhận diện các **common patterns**, hoặc cấu trúc của flag. Bằng cách so khớp tần suất và thử xây dựng dần **mapping hypothesis**, rồi kiểm tra tính hợp lý của plaintext thu được, ta từng bước hoàn thiện bảng ánh xạ. Do cipher này thiếu tính **diffusion** và không có cơ chế thay đổi khóa theo vị trí ký tự, nên chỉ cần đủ dữ liệu ciphertext là có thể suy ra toàn bộ mapping và giải mã hoàn chỉnh.

```
PS G:\My Drive\NT219\Demo> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe "g:/My Drive/NT219/Demo/Classic
al-Ciphers/Substitution-Techniques/Monoalphabetic-Ciphers-1-1/Simple-Substitution-Cipher/HackSimpleSubstitutionCipher.py"
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
-----
A I P F H R S P Y O K Q U K O U L G Y B U N T E V C

Decrypted Text:

CTFs (short for capture the flag) are a type of computer security competition.
Contestants are presented with a set of challenges which test their creativity,
technical (and googling) skills, and problem-solving ability.
Challenges usually cover a number of categories, and when solved,
each yields a string (called a flag) which is submitted to an online scoring service.
CTFs are a great way to learn a wide array of computer security skills in a safe,
legal environment, and are hosted and played by many security groups around the world for fun and practice.
For this problem, the flag is: picoCTF{FR3QU3NCY_4774CK5_4R3_C001_4871E6FB}
```

Challenge's flag

```

1 from collections import Counter
2 import string
3
4 # Known English Letter frequencies (from most frequent to least frequent)
5 english_frequencies = 'ETAOINSHRDLCLUMWFGYPBVKJXQZ'
6
7 cipher_text = ("""
8 ZMDg (gejfw djf zacwpxf wex dqar) afx a wscx jd zjicpwxf gxzpfbws zjicxwbwbjv.
9 Zjvwgxwawvg afx cfxxgvwxm hbwe a gxw jd zeaqxvrxg hebze wxgw wexbf zfxawbybws,
10 wxzevbzqa (avm rjjrqbvr) gnbqag, avm cfjtxqi-gjqybvr atqbws.
11 Zeaqxvrxg pgaqqs zjyxf a vpitxf jd zawxrjfbxg, avm hexv gjqyxm,
12 xaze sbxqmg a gwfivr (zaqxmx a dqar) hebze bg gptibwxm wj av jvqbxv gzjfbvr gxfybxz.
13 ZMDg afx a rfxaw has wj qxafv a hbmz affas jd zjicpwxf gxzpfbws gnbqag bv a gadx,
14 qxraq xvybfjvixvw, avm afx ejgxwm avm cqasxm ts iavs gxzpfbws rfjpcg afjpvw wex hjfqm djf dpv avm cfazwbzx.
15 Djf webg cfjtxqi, wex dqar bg: cbzjZMD(DF3LP3VZS_4774ZN5_4F3_Z001_4871X6DT)
16 """)
17
18 # Count letter frequencies in the ciphertext (ignoring non-alphabet characters)
19 cipher_counts = Counter(''.join(filter(str.isalpha, cipher_text.upper())))
20
21 # Sort the ciphertext letters by frequency (most frequent first)
22 sorted_cipher = ''.join([item[0] for item in cipher_counts.most_common()])
23
24 # Create an initial mapping from ciphertext letters to the English frequency order
25 mapping = {}
26 for i, letter in enumerate(sorted_cipher):
27     mapping[letter] = english_frequencies[i]
28
29 # For any letters not present in the ciphertext, add an identity mapping
30 for letter in string.ascii_uppercase:
31     if letter not in mapping:
32         mapping[letter] = letter
33
34 # Optional manual adjustments to improve decryption quality
35 # Súa mapping ó dáy
36 mapping["A"] = "A"
37 mapping["F"] = "R"
38 mapping["C"] = "P"
39 mapping["Z"] = "C"
40 mapping["J"] = "O"
41 mapping["W"] = "T"
42 mapping["D"] = "F"
43 mapping["E"] = "H"
44 mapping["Q"] = "L"
45 mapping["R"] = "G"
46 mapping["G"] = "S"
47 mapping["P"] = "U"
48 mapping["S"] = "Y"
49 mapping["L"] = "Q"
50 mapping["V"] = "N"
51
52 def print_key_mapping_table(mapping):
53     """
54     Displays the key mapping in the requested table format:
55
56     A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
57     -+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+--+
58     R A T X U K E Y H O I D V F G M P L Z W S Q J C B N
59     """
60     plain_letters = list(string.ascii_uppercase)
61     # Build the first row: plain letters with a fixed width (2 characters per letter)
62     row1 = " ".join(f"{letter:2}" for letter in plain_letters)
63     # Build the border row: "-+--+...+--"
64     row2 = " " + "--" + "+--"*(len(plain_letters)-1) + " "
65     # Build the third row: corresponding cipher letters from the mapping
66     row3 = " ".join(f"{mapping[letter]:2}" for letter in plain_letters)
67
68     print(row1)
69     print(row2)
70     print(row3)
71
72 # Display the final mapping using the desired table format
73 print_key_mapping_table(mapping)
74
75 # Decrypt the ciphertext using the mapping (preserving letter case)
76 decrypted_text = []
77 for char in cipher_text:
78     if char.isalpha():
79         if char.isupper():
80             decrypted_text.append(mapping.get(char, char))
81         else:
82             decrypted_text.append(mapping.get(char.upper(), char.upper()).lower())
83     else:
84         decrypted_text.append(char)
85 decrypted_text = ''.join(decrypted_text)
86
87 print("\nDecrypted Text:")
88 print(decrypted_text)
89

```

- substitution2   

Medium Cryptography picoCTF 2022 Substitution

Hints ?

1

Download the message [here](#).

- **Solution:** Bài này khai thác điểm yếu cố hữu của **monoalphabetic substitution cipher**, cụ thể là việc mỗi chữ cái chỉ được thay thế bằng một chữ cái khác theo một **fixed substitution mapping**. Vì phép thay thế là tĩnh và không thay đổi theo vị trí, hệ mã này rất dễ bị tấn công bằng **frequency analysis attack**. Thay vì dùng key ở đầu file, ta có thể dựa vào **letter frequency distribution** của tiếng Anh, kết hợp với việc nhận diện các **common patterns**, hoặc cấu trúc của flag. Bằng cách so khớp tần suất và thử xây dựng dần **mapping hypothesis**, rồi kiểm tra tính hợp lý của plaintext thu được, ta từng bước hoàn thiện bảng ánh xạ. Do cipher này thiếu tính **diffusion** và không có cơ chế thay đổi khóa theo vị trí ký tự, nên chỉ cần đủ dữ liệu ciphertext là có thể suy ra toàn bộ mapping và giải mã hoàn chỉnh.

Challenge's flag

```

1 from collections import Counter
2 import string
3
4 # Known English letter frequencies (from most frequent to least frequent)
5 english_frequencies = "ETAOINSHRDLUMFGPYBVCXZQJ"
6
7 cipher_text = ("***fnjdjjzsqsfjppdxwmfndjcjwjsxfxhwsnjyngensknmmkwmuavafjd
8 sjkadqftkmuvjfjqfqmgsdqkwaygektthjdvxfdmqfyaskthjdknwxwjg
9 ejfnjskmuvjffqfqmgslmkasvdquxdqmgtmgstfsjusxyugqsdfxqmgla
10 gyxujefxwschnkdxdjpjdtasjlawgxuydofjfxhjsoagwnsmcnjpjdcjhj
11 wqpjpfnnjdvmvjdvadvmsjmnlxnqnsknmmkuavafjdsjkadqftkmuvjffq
12 qmgsgfmfwgmfmfjxknpxwahwjsoqswshafxwsmfmejfsfayjgfsqgfjd
13 jsfjyagxyejzkqfjyxhaufkmuvafjdsqkjgkyjlljgsqpkmuvfjqfqmgs
14 xdmnfjgwghmdmasxlxxqdsxygyumjcmcfmadaggageknjkwagsfsxygj
15 zjkafagekgmlgeskdqvfsmlljgsmjgnfmjnfnjdnxgyqsnjxpqtwlmkasjy
16 mgjzvmdmxfgmqgyquvdmpqsgxfmgxymlfjgnxsjwjujgfsmlkmuvafjds
17 wqpjpkmuvfjqfqmgfmaekngemgnfmlljgsqpkmuvfjqfqmgslnmkasjy
18 jkadqftksnfndjldmjkhfffjdpjpnqkwjldmfjknjpxgejwqsufmsfayjg
19 sggusqdqxsgnqnsknmmwsladfndcjhljwqpjfnxfgagygjdsfxgyqgem
20 lmljgsqpfjfkngqiajsqssjsjgfcxwldmdumagfagegexljjkfapjyjlg
21 sjxyzfnxfnnfjnmqwsxyknqlqeadxfgmglmkasjgknagfjidjyqajyljgsq
22 pjkmuvjffqfqmgsysgmjfyxysfayjgfsfmoegcfnjdqfejutuxsjlljkfa
23 pjwtuxsfxjknqgefjnufnxkfqpjwtnfqagwoqjgzpxffkojdvqkmklqszx
24 mlljgsqpfwdmdqgefjynqnsknmmkwmuavafjdsjkadqftkmuvjffqfqmgn
25 xfsjjosfmejgdjzfagfjdjsfagkmuvafjdsqdkjgkxungengensknmmw
26 dsfjknqngefjnjgmaenxhaufkmuvafjdsjkadqftmvqlajfjnjdkaadm
27 sqfttmfagnpqeegnufnuzvmdjgnfndgcexgyjzxhwagefnjufmhffj
28 jdyjljgyfnjqduxknagjsfnjlwxeqsvqmkfl(6604U_4G41T515_15_73Y10A5_423K1770)")
29
30 # Count Letter frequencies in the ciphertext (ignoring non-alphabet characters)
31 cipher_counts = Counter(''.join(filter(str.isalpha, cipher_text.upper())))
32
33 # Sort the ciphertext letters by frequency (most frequent first)
34 sorted_cipher = ''.join([letter[i] for item in cipher_counts.most_common()])
35
36 # Create an initial mapping from ciphertext letters to the English frequency order
37 mapping = {}
38 for i, letter in enumerate(sorted_cipher):
39     mapping[letter] = english_frequencies[i]
40
41 # For any Letters not present in the ciphertext, add an identity mapping
42 for letter in string.ascii_uppercase:
43     if letter not in mapping:
44         mapping[letter] = letter
45
46 # Optional manual adjustments to improve decryption quality
47 # Súa mapping á dáy
48 mapping["A"] = "U"
49 mapping["C"] = "W"
50 mapping["D"] = "R"
51 mapping["E"] = "G"
52 mapping["F"] = "T"
53 mapping["G"] = "N"
54 mapping["H"] = "B"
55 mapping["I"] = "O"
56 mapping["J"] = "E"
57 mapping["K"] = "C"
58 mapping["L"] = "F"
59 mapping["M"] = "O"
60 mapping["N"] = "H"
61 mapping["O"] = "K"
62 mapping["P"] = "V"
63 mapping["Q"] = "I"
64 mapping["S"] = "S"
65 mapping["T"] = "V"
66 mapping["U"] = "M"
67 mapping["V"] = "P"
68 mapping["W"] = "L"
69 mapping["X"] = "A"
70 mapping["Y"] = "D"
71 mapping["Z"] = "X"
72
73 def print_key_mapping_table(mapping):
74     """
75     Displays the key mapping in the requested table format:
76
77     A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
78     +-+--+---+-----+-----+-----+-----+-----+----+
79     R A T X U K E Y H O I D V F G M P L Z W S Q J C B N
80     """
81     plain_letters = list(string.ascii_uppercase)
82     # Build the first row: plain letters with a fixed width (2 characters per letter)
83     row1 = "".join(f"{letter:2}" for letter in plain_letters)
84     # Build the border row: "-+-+-.+...-+"
85     row2 = "-" + "+" + "-" + "+" + "-" * ((len(plain_letters)-1) + 1) + "-"
86     # Build the third row: corresponding cipher letters from the mapping
87     row3 = "".join(f"{mapping[letter]:2}" for letter in plain_letters)
88
89     print(row1)
90     print(row2)
91     print(row3)
92
93 # Display the final mapping using the desired table format
94 print_key_mapping_table(mapping)
95
96 # Decrypt the ciphertext using the mapping (preserving Letter case)
97 decrypted_text = []
98 for char in cipher_text:
99     if char.isalpha():
100         if char.isupper():
101             decrypted_text.append(mapping.get(char, char))
102         else:
103             decrypted_text.append(mapping.get(char.upper(), char.upper()).lower())
104     else:
105         decrypted_text.append(char)
106 decrypted_text = ''.join(decrypted_text)
107
108 print("\nDecrypted Text:")
109 print(decrypted_text)

```

Dictionary-based substitution mapping

VI. Rail-fence (picoCTF)

- **Challenge:** Bài rail-fence trong picoCTF 2022 thuộc dạng transposition cipher, cụ thể là Rail Fence Cipher. Đề bài cung cấp một thông điệp đã bị mã hóa và cho biết rằng nó được mã hóa bằng phương pháp rail fence với 4 rails. Trong loại mật mã này, plaintext được viết theo dạng zigzag pattern qua nhiều hàng (rails), sau đó ciphertext được tạo ra bằng cách đọc lần lượt các ký tự theo từng hàng từ trên xuống dưới. Nhiệm vụ của người giải là khôi phục lại cách sắp xếp zigzag ban đầu để tìm lại thông điệp gốc, sau đó đặt kết quả vào định dạng flag của picoCTF.

rail-fence

Medium Cryptography picoCTF 2022

AUTHOR: WILL HONG

Hints ?

1

Description

A type of transposition cipher is the rail fence cipher, which is described [here](#). Here is one such cipher encrypted using the rail fence with 4 rails. Can you decrypt it?

Download the message [here](#).

Put the decoded message in the picoCTF flag format, `picoCTF{decoded_message}`.

<https://play.picoctf.org/practice/challenge/289?category=2&page=2>

- **Solution:** Để giải bài này, ta sử dụng công cụ Cryptii, một công cụ trực tuyến hỗ trợ nhiều loại mã hóa cổ điển. Ciphertext được đưa vào module Rail Fence Cipher và cấu hình số rails = 4 theo đúng thông tin đề bài cung cấp. Sau khi chọn chế độ decrypt, công cụ sẽ tự động tái tạo lại zigzag pattern và sắp xếp lại các ký tự theo đúng thứ tự ban đầu, từ đó thu được plaintext. Kết quả giải mã là một thông điệp có nghĩa, trong đó chứa nội dung cần đặt vào định dạng picoCTF {...} để hoàn thành challenge.

cryptii

VIEW Text

ENCODE DECODE

Rail fence cipher

KEY 4

OFFSET 0

Decoded 56 chars

VIEW Text

The flag is: WH3R3_D035_7H3_F3NC3_8361N_4ND_3ND_83F6D8D7

<https://cryptii.com/>

b
 c
 g
 n

- transposition-trial

Medium

Cryptography

picoCTF 2022

cryptography

AUTHOR: WILL HONG

Hints

Description

1

Our data got corrupted on the way here. Luckily, nothing got replaced, but every block of 3 got scrambled around! The first word seems to be three letters long, maybe you can use that to recover the rest of the message.

Download the corrupted message [here](#).

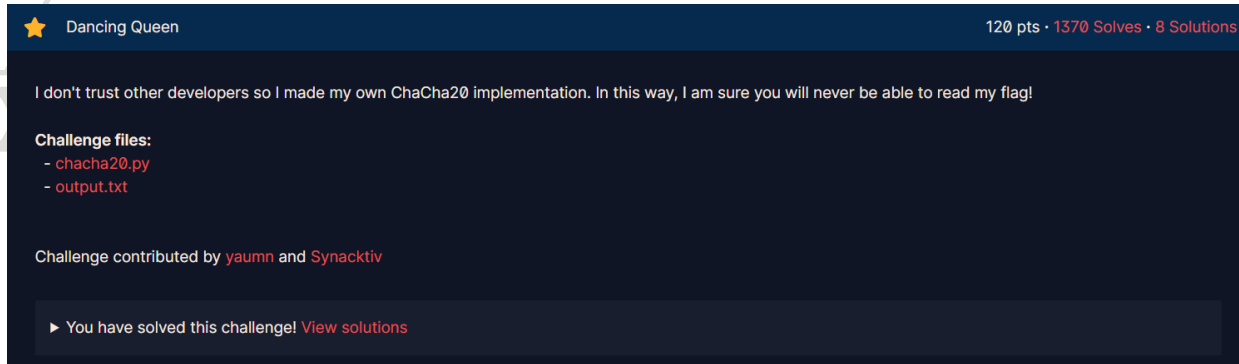
Table 1

- ```
1 cipher = "heTfl g as iicpCTo{7F4NRP051N5_16_35P3X51N3_V9AAB1F8}7"
2
3 result = ""
4 for i in range(0, len(cipher), 3):
5 block = cipher[i:i+3]
6 if len(block) == 3:
7 result += block[2] + block[0] + block[1]
8
9 print(result)
```

## Script Python

## VIII. Dancing Queen (CRYPTOHACK)

- **Challenge:** Bài Dancing Queen trên CryptoHack cung cấp một bản thực thi tùy chỉnh của thuật toán mã hóa luồng ChaCha20. Thử thách cho sẵn một cặp bản rõ msg và bản mã tương ứng msg\_enc, cùng với bản mã của flag flag\_enc được mã hóa bằng một khóa bí mật khác nhưng dùng chung cấu trúc. Điểm yếu cốt lõi nằm ở việc người tạo ra thử thách đã "tự chế" lại ChaCha20 và vô tình bỏ qua bước cộng dồn trạng thái ban đầu vào trạng thái sau khi biến đổi trước khi XOR với bản rõ. Điều này khiến cho các bước biến đổi trong Quarter Round của ChaCha20 trở thành một phép hoán vị có tính đảo ngược hoàn toàn.



<https://cryptohack.org/challenges/aes/>

- **Solution:** Vì thuật toán thiếu bước cộng dồn, khối keystream thu được chính là trạng thái sau 10 vòng lặp inner\_block. Ta thực hiện tấn công Known Plaintext Attack bằng cách XOR bản rõ đã biết msg với bản mã msg\_enc để thu được toàn bộ khối keystream đầu tiên. Sau đó, ta thực hiện đảo ngược 10 vòng biến đổi bằng hàm inv\_inner\_block để quay ngược từ keystream về lại trạng thái khởi tạo. Từ trạng thái khởi tạo này, ta có thể dễ dàng trích xuất được khóa bí mật nằm ở các words từ vị trí số 4 đến 11. Cuối cùng, dùng khóa đã khôi phục được kết hợp với iv2 để giải mã flag\_enc và thu được flag hoàn chỉnh.

```

1 #!/usr/bin/env python3
2
3 from os import urandom
4
5
6 FLAG = b'crypto{????????????????????????????}'
7
8
9 def bytes_to_words(b):
10 return [int.from_bytes(b[i:i+4], 'little') for i in range(0, len(b), 4)]
11
12 def rotate(x, n):
13 return ((x << n) & 0xffffffff) | ((x >> (32 - n)) & 0xffffffff)
14
15 def word(x):
16 return x % (2 ** 32)
17
18 def words_to_bytes(w):
19 return b''.join([i.to_bytes(4, 'little') for i in w])
20
21 def xor(a, b):
22 return b''.join([bytes([x ^ y]) for x, y in zip(a, b)])
23
24
25 class ChaCha20:
26 def __init__(self):
27 self.state = []
28
29 def _inner_block(self, state):
30 self._quarter_round(state, 0, 4, 8, 12)
31 self._quarter_round(state, 1, 5, 9, 13)
32 self._quarter_round(state, 2, 6, 10, 14)
33 self._quarter_round(state, 3, 7, 11, 15)
34 self._quarter_round(state, 0, 5, 10, 15)
35 self._quarter_round(state, 1, 6, 11, 12)
36 self._quarter_round(state, 2, 7, 8, 13)
37 self._quarter_round(state, 3, 4, 9, 14)
38
39 def _quarter_round(self, x, a, b, c, d):
40 x[a] = word(x[a] + x[b]); x[d] ^= x[a]; x[d] = rotate(x[d], 16)
41 x[c] = word(x[c] + x[d]); x[b] ^= x[c]; x[b] = rotate(x[b], 12)
42 x[a] = word(x[a] + x[b]); x[d] ^= x[a]; x[d] = rotate(x[d], 8)
43 x[c] = word(x[c] + x[d]); x[b] ^= x[c]; x[b] = rotate(x[b], 7)
44
45 def _setup_state(self, key, iv):
46 self.state = [0x61707865, 0x3320646e, 0x79622d32, 0x6b206574]
47 self.state.extend(bytes_to_words(key))
48 self.state.append(self._counter)
49 self.state.extend(bytes_to_words(iv))
50
51 def decrypt(self, c, key, iv):
52 return self.encrypt(c, key, iv)
53
54 def encrypt(self, m, key, iv):
55 c = b''
56 self._counter = 1
57
58 for i in range(0, len(m), 64):
59 self._setup_state(key, iv)
60 for j in range(10):
61 self._inner_block(self.state)
62 c += xor(m[i:i+64], words_to_bytes(self.state))
63
64 self._counter += 1
65
66 return c
67
68
69 if __name__ == '__main__':
70 msg = b'Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula.'
71 key = urandom(32)
72 iv1 = urandom(12)
73 iv2 = urandom(12)
74
75 c = ChaCha20()
76 msg_enc = c.encrypt(msg, key, iv1)
77 flag_enc = c.encrypt(FLAG, key, iv2)
78
79 print(f"iv1 = '{iv1.hex()}'")
80 print(f"iv2 = '{iv2.hex()}'")
81 print(f"msg_enc = '{msg_enc.hex()}'")
82 print(f"flag_enc = '{flag_enc.hex()}'")

```

## Script Challenge

```

1 #!/usr/bin/env python3
2
3 iv1_hex = 'e42758d6d218013ea63e3c49'
4 iv2_hex = 'a99f9a7d097daabd2aa2a235'
5 msg_enc_hex = 'f3afhada8237af6e94c7d2065ee0e221a1748b8c7b11105a8cc8a1c74253611c94fe70a6fa8' \
6 'a9133505772ef619f04b05d2e2b0732cc483df72cceb09a92c211ef5a52628094f09a30fc6' \
7 '92cb25647f'
8 flag_enc_hex = 'b6327e9a2253034096344ad5694a2040b114753e24ea9c1af17c10263281fb0fe622b32732'
9 msg = b'Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula.'
10
11 def bytes_to_words(b):
12 return [int.from_bytes(b[i:i+4], 'little') for i in range(0, len(b), 4)]
13
14 def words_to_bytes(w):
15 return b''.join([i.to_bytes(4, 'little') for i in w])
16
17 def xor(a, b):
18 return b''.join([bytes([x ^ y]) for x, y in zip(a, b)])
19
20 def rotate(x, n):
21 return ((x << n) & 0xffffffff) | ((x >> (32 - n)) & 0xffffffff)
22
23 def word(x):
24 return x % (2 ** 32)
25
26 class ChaCha20:
27 def __init__(self):
28 self._state = []
29
30 def _quarter_round(self, x, a, b, c, d):
31 x[a] = word(x[a] + x[b]); x[d] ^= x[a]; x[d] = rotate(x[d], 16)
32 x[c] = word(x[c] + x[d]); x[b] ^= x[c]; x[b] = rotate(x[b], 12)
33 x[a] = word(x[a] + x[b]); x[d] ^= x[a]; x[d] = rotate(x[d], 8)
34 x[c] = word(x[c] + x[d]); x[b] ^= x[c]; x[b] = rotate(x[b], 7)
35
36 def _inner_block(self, state):
37 self._quarter_round(state, 0, 4, 8, 12); self._quarter_round(state, 1, 5, 9, 13)
38 self._quarter_round(state, 2, 6, 10, 14); self._quarter_round(state, 3, 7, 11, 15)
39 self._quarter_round(state, 0, 5, 10, 15); self._quarter_round(state, 1, 6, 11, 12)
40 self._quarter_round(state, 2, 7, 8, 13); self._quarter_round(state, 3, 4, 9, 14)
41
42 def _setup_state(self, key, iv):
43 self._state = [0x61707865, 0x3320646e, 0x79622d32, 0x6b206574]
44 self._state.extend(bytes_to_words(key))
45 self._state.append(self._counter)
46 self._state.extend(bytes_to_words(iv))
47
48 def encrypt(self, m, key, iv):
49 c = b''
50 self._counter = 1
51 for i in range(0, len(m), 64):
52 self._setup_state(key, iv)
53 for j in range(10): self._inner_block(self._state)
54 c += xor(m[i:i+64], words_to_bytes(self._state))
55 self._counter += 1
56 return c
57
58 def rotate_right(x, n):
59 return ((x >> n) & 0xffffffff) | ((x << (32 - n)) & 0xffffffff)
60
61 def word_sub(a, b):
62 return (a - b) % (2**32)
63
64 def inv_quarter_round(x, a, b, c, d):
65 x[b] = rotate_right(x[b], 7); x[b] ^= x[c]; x[c] = word_sub(x[c], x[d])
66 x[d] = rotate_right(x[d], 8); x[d] ^= x[a]; x[a] = word_sub(x[a], x[b])
67 x[b] = rotate_right(x[b], 12); x[b] ^= x[c]; x[c] = word_sub(x[c], x[d])
68 x[d] = rotate_right(x[d], 16); x[d] ^= x[a]; x[a] = word_sub(x[a], x[b])
69
70 def inv_inner_block(state):
71 inv_quarter_round(state, 3, 4, 9, 14)
72 inv_quarter_round(state, 2, 7, 8, 13)
73 inv_quarter_round(state, 1, 6, 11, 12)
74 inv_quarter_round(state, 0, 5, 10, 15)
75 inv_quarter_round(state, 3, 7, 11, 15)
76 inv_quarter_round(state, 2, 6, 10, 14)
77 inv_quarter_round(state, 1, 5, 9, 13)
78 inv_quarter_round(state, 0, 4, 8, 12)
79
80 if __name__ == '__main__':
81 iv1 = bytes.fromhex(iv1_hex)
82 iv2 = bytes.fromhex(iv2_hex)
83 msg_enc = bytes.fromhex(msg_enc_hex)
84 flag_enc = bytes.fromhex(flag_enc_hex)
85
86 keystream_block1 = xor(msg[:64], msg_enc[:64])
87
88 state = bytes_to_words(keystream_block1)
89
90 for _ in range(10):
91 inv_inner_block(state)
92
93 recovered_key_words = state[4:12]
94 recovered_key = words_to_bytes(recovered_key_words)
95
96 print("[*] Recovered Key: {recovered_key.hex()}")
97
98 c = ChaCha20()
99 flag = c.encrypt(flag_enc, recovered_key, iv2)
100 print("[+] Flag: {flag.decode()}")

```

## Script Python

```

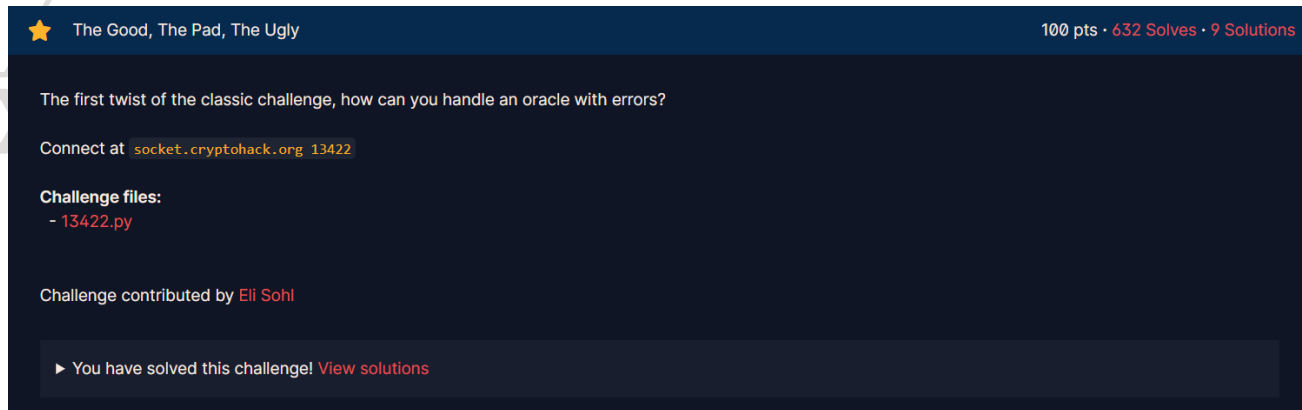
PS C:\Users\ASUS> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe c:/Users/ASUS/Downloads/solve.py
[*] Recovered Key: 39fd1410fef6485bf3068ea0fb3a8ff6385b4483bc1f321cea4f15cc1c43496c
[+] Flag: crypto{M1x1n6_r0und5_4r3_1nv3r71bl3!}

```

## Challenge's flag

IX. The Good, The Pad, The Ugly (**CRYPTOHACK**)

- **Challenge:** Bài The Good, The Pad, The Ugly trên CryptoHack là một thử thách về lỗi thực thi Padding Oracle Attack trong chế độ mã hóa AES-CBC, nhưng có một nút thắt quan trọng là Noisy Oracle. Thay vì trả về kết quả chính xác hoàn toàn, Oracle kiểm tra padding của hệ thống này sẽ "nói dối": nếu padding đúng, nó luôn trả về True; nhưng nếu padding sai, nó vẫn có xác suất 60% trả về True và chỉ 40% trả về False. Người chơi được cấp tối đa 12,000 lần truy vấn để giải mã một thông điệp dài 32 ký tự hex được server tạo ngẫu nhiên.



<https://cryptohack.org/challenges/aes/>

- **Solution:** Để vượt qua yếu tố nhiễu, ta áp dụng kỹ thuật Statistical Filtering dựa trên đặc điểm: kết quả False chỉ xuất hiện khi padding sai. Ta thực hiện gửi lặp lại nhiều lần khoảng 20 lần cho mỗi byte dự đoán; nếu có bất kỳ lần nào Oracle trả về False, ta có thể khẳng định chắc chắn dự đoán đó là sai và loại bỏ ngay lập tức. Ngoài ra, vì thông điệp gốc là chuỗi hex, ta thu hẹp không gian tìm kiếm từ 256 giá trị xuống còn 16 ký tự (0-9, a-f), giúp tối ưu số lượng truy vấn trong giới hạn cho phép. Bằng cách kết hợp thuật toán Padding Oracle truyền thống với việc lấy mẫu lặp lại, ta từng bước khôi phục lại bản rõ của từng block, từ đó lấy được thông điệp hoàn chỉnh để đổi lấy flag.

```

1 #!/usr/bin/env python3
2
3 from Crypto.Util.Padding import unpad
4 from Crypto.Cipher import AES
5 from os import urandom
6 from random import SystemRandom
7
8 from utils import listener
9
10 FLAG = 'crypto{????????????????????????????????????}'
11 rng = SystemRandom()
12
13
14 class Challenge:
15 def __init__(self):
16 self.before_input = "That last challenge was pretty easy, but I'm positive that this one will be harder!\n"
17 self.message = urandom(16).hex()
18 self.key = urandom(16)
19 self.query_count = 0
20 self.max_queries = 12_000
21
22 def update_query_count(self):
23 self.query_count += 1
24 if self.query_count >= self.max_queries:
25 self.exit = True
26
27 def get_ct(self):
28 iv = urandom(16)
29 cipher = AES.new(self.key, AES.MODE_CBC, iv=iv)
30 ct = cipher.encrypt(self.message.encode("ascii"))
31 return {"ct": (iv+ct).hex()}
32
33 def check_padding(self, ct):
34 ct = bytes.fromhex(ct)
35 iv, ct = ct[:16], ct[16:]
36 cipher = AES.new(self.key, AES.MODE_CBC, iv=iv)
37 pt = cipher.decrypt(ct) # does not remove padding
38 try:
39 unpad(pt, 16)
40 except ValueError:
41 good = False
42 else:
43 good = True
44 self.update_query_count()
45 return {"result": good | (rng.random() > 0.4)}
46
47 def check_message(self, message):
48 if message != self.message:
49 self.exit = True
50 return {"error": "incorrect message"}
51 return {"flag": FLAG}
52
53 #
54 # This challenge function is called on your input, which must be JSON
55 # encoded
56 #
57 def challenge(self, msg):
58 if "option" not in msg or msg["option"] not in ("encrypt", "unpad", "check"):
59 return {"error": "Option must be one of: encrypt, unpad, check"}
60
61 if msg["option"] == "encrypt": return self.get_ct()
62 elif msg["option"] == "unpad": return self.check_padding(msg["ct"])
63 elif msg["option"] == "check": return self.check_message(msg["message"])
64
65
66 import builtins; builtins.Challenge = Challenge # hack to enable challenge to be run locally, see https://cryptohack.org/faq/#Listener
67 listener.start_server(port=13422)

```

## Script Challenge

```

1 import socket
2 import json
3
4 HOST = "socket.cryptohack.org"
5 PORT = 13422
6 HEX_CHARS = "0123456789abcdef"
7 SAMPLES = 20
8
9 def solve():
10 s = socket.create_connection((HOST, PORT))
11
12 def json_rcv():
13 data = b""
14 while not data.endswith(b"\n"):
15 chunk = s.recv(1)
16 if not chunk: break
17 data += chunk
18 return json.loads(data.decode())
19
20 def json_send(hsh):
21 s.sendall(json.dumps(hsh).encode() + b"\n")
22
23 initial_msg = s.recv(1024).decode()
24 print(f"[+] Server: {initial_msg.strip()}")
25
26 json_send({"option": "encrypt"})
27 data = json_rcv()
28 ct_hex = data['ct']
29 ct_bytes = bytes.fromhex(ct_hex)
30
31 iv = ct_bytes[:16]
32 block1 = ct_bytes[16:32]
33 block2 = ct_bytes[32:48]
34
35 def is_padding_valid(ct_to_test):
36 for _ in range(SAMPLES):
37 json_send({"option": "unpad", "ct": ct_to_test.hex()})
38 if not json_rcv()['result']:
39 return False
40 return True
41
42 def crack_block(target_block, control_block):
43 decrypted_chars = [""] * 16
44 for i in range(15, -1, -1):
45 pad_val = 16 - i
46 print(f"[*] Decoding byte {i}...", end="\r")
47 for char in HEX_CHARS:
48 test_control = list(control_block)
49 for j in range(i + 1, 16):
50 test_control[j] = control_block[j] ^ ord(decrypted_chars[j]) ^ pad_val
51 test_control[i] = control_block[i] ^ ord(char) ^ pad_val
52
53 if is_padding_valid(bytes(test_control) + target_block):
54 decrypted_chars[i] = char
55 break
56 print(f"\n[+] Block Decoded: {''.join(decrypted_chars)}")
57 return ''.join(decrypted_chars)
58
59 print("--- Block 2 ---")
60 m2 = crack_block(block2, block1)
61 print("--- Block 1 ---")
62 m1 = crack_block(block1, iv)
63
64 full_message = m1 + m2
65 print(f"\n[!] Message: {full_message}")
66
67 json_send({"option": "check", "message": full_message})
68 print(f"\n[FINISH] {json_rcv()}")
69
70 if __name__ == "__main__":
71 try:
72 solve()
73 except Exception as e:
74 print(f"\n[ERROR] {e}")

```

## Script Python

```

PS C:\Users\ASUS> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/ASUS/Downloads/import_socket.py"
[+] Server: That last challenge was pretty easy, but I'm positive that this one will be harder!
--- Block 2 ---
[*] Decoding byte 0....
[+] Block Decoded: 07d4880f581e5778
--- Block 1 ---
[*] Decoding byte 0....
[+] Block Decoded: 1288b779899a68c7

[!] Message: 1288b779899a68c707d4880f581e5778

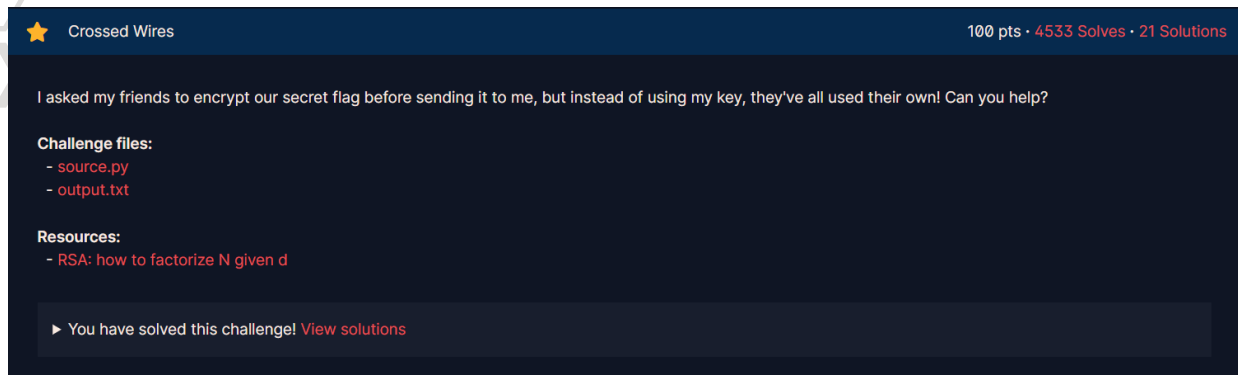
[FINISH] {'flag': 'crypto{even_a_faulty_oracle_leaks_all_information}'}
PS C:\Users\ASUS>

```

## Challenge's flag

## X. Crossed Wires (CRYPTOHACK)

- **Challenge:** Bài toán này xây dựng dựa trên hệ mật mã RSA, trong đó một modulus  $N = p \cdot q$  được tạo từ hai số nguyên tố lớn và sử dụng public exponent  $e = 65537$ . Tuy nhiên, điểm bất thường là đề bài lại tiết lộ trực tiếp private key  $d$  cùng với  $N$ , điều này phá vỡ hoàn toàn tính bảo mật của RSA. Ngoài ra, flag không được mã hóa một lần như thông thường mà được mã hóa lặp lại nhiều lần thông qua các “friend keys”, tức là các public exponent nhỏ khác nhau nhưng đều dùng chung modulus  $N$ . Do đó, ciphertext cuối cùng có dạng  $c = m^{e_1 \cdot e_2 \cdot \dots \cdot e_k} \bmod N$ , biến bài toán thành một trường hợp của repeated RSA encryption with shared modulus, từ đó gợi ý khả năng khai thác dựa trên việc lộ  $d$ .



<https://cryptohack.org/challenges/rsa/>

- **Solution:** Để giải bài này, ta tận dụng việc đề bài cung cấp private key  $d$ . Trong RSA luôn tồn tại quan hệ  $e \cdot d \equiv 1 \bmod \phi(N)$ , từ đó suy ra  $e \cdot d - 1$  là một bội số của  $\phi(N)$ . Dựa vào điều này, ta áp dụng một thuật toán factorization để phân tích  $N$  thành hai thừa số nguyên tố  $p, q$ , từ đó tính lại  $\phi(N) = (p - 1)(q - 1)$ . Tiếp theo, do ciphertext được tạo bằng cách mã hóa nhiều lần với các exponent khác nhau, ta gộp tất cả lại thành một exponent duy nhất  $E = e_1 \cdot e_2 \cdot \dots \cdot e_k$ , khi đó bài toán trở về dạng RSA chuẩn  $c = m^E \bmod N$ . Để giải mã, ta tính nghịch đảo modular  $d' = E^{-1} \bmod \phi(N)$ , rồi tính  $m = c^{d'} \bmod N$ . Cuối cùng, chuyển kết quả từ số nguyên về dạng bytes bằng `long_to_bytes`, ta thu được plaintext ban đầu chính là flag theo định dạng `crypto{...}`.

```

1 from Crypto.Util.number import getPrime, long_to_bytes, bytes_to_long, inverse
2 import math
3 from gmpy2 import next_prime
4
5 FLAG = b"crypto{??}"
6
7 p = getPrime(1024)
8 q = getPrime(1024)
9 N = p*q
10 phi = (p-1)*(q-1)
11 e = 0x10001
12 d = inverse(e, phi)
13
14 my_key = (N, d)
15
16 friends = 5
17 friend_keys = [(N, getPrime(17)) for _ in range(friends)]
18
19 cipher = bytes_to_long(FLAG)
20
21 for key in friend_keys:
22 cipher = pow(cipher, key[1], key[0])
23
24 print(f"My private key: {my_key}")
25 print(f"My Friend's public keys: {friend_keys}")
26 print(f"Encrypted flag: {cipher}")

```

## Script Challenge

```

1 from Crypto.Util.number import GCD, inverse, long_to_bytes
2 import random
3
4 def factorize(N, e, d):
5 k = e * d - 1
6 if k % 2 == 1:
7 return None
8
9 t = k
10 r = 0
11 while t % 2 == 0:
12 t //= 2
13 r += 1
14
15 while True:
16 a = random.randrange(2, N - 1)
17 x = pow(a, t, N)
18
19 if x == 1 or x == N - 1:
20 continue
21
22 for _ in range(r):
23 y = pow(x, 2, N)
24
25 if y == 1:
26 p = GCD(x - 1, N)
27 q = N // p
28 return p, q
29
30 x = y
31 if x == N - 1:
32 break
33
34 N = int("""
35 2171130822534631554270684461844156574104649827771697994347836059805314497137
36 9956916575370343448988601905854572029635846626259487297950305231661109855854
37 9474942091352055892586435179615215949243684986720642932082308024410773901936
38 829580951119220826778131758047756288843772437764742838584183127705927417298
39 228054523776555996922870506857561215268491024097063920337721783673060530181
40 6371615774015891265585561825468967833073705172750465227040473857861114894470
41 6479421001080276170861590724552349258589628637499608808931782616279827852829
42 6206977900274431829829206103227171839270887476436899494428371323874689055690
43 729986771
44 """, "\n", ""))
45
46 e = 65537
47
48 d = int("""
49 2734411677251148030723138005716109733838866545375527602018255159319631026653
50 1907836704931079364016039814291718805043605604947710172464687029026473709542
51 2031245254134285874759057627377510787045085353371711668432697626300643573338
52 2045807971890762018747729574021057430331778033982359184838159747331236538501
53 8499653292647749276075704103470194184074519378756843734549823069231784031612
54 1681723789096265121471883195421520063765110390720934790085782472265321717954
55 814814568718137722054486452180823012273096745298143535334932104265488075777
56 6386080413252567762752000675415330225279647434785549487925780577085223508121
57 54888097
58 """, "\n", ""))
59
60 friend_keys = [
61 (N, 106979),
62 (N, 108533),
63 (N, 69557),
64 (N, 97117),
65 (N, 103231),
66]
67
68 cipher = int("""
69 2030461027957818673817276622422479311988507126246446444886346118409222573605
70 4747976985179673905441502689126216282897704508745403799054734121583968853999
71 7916042816151541007362591314534243853643246302296711853437781728072626407093
72 0183827482460310169248566272622690212110559113743733146320188126424556221401
73 2160875177167442010952439360623396658974413900469093836794752270399520074596
74 3290587258748340821886973775979494057790391391941960653644262132083454614070
75 3077108978752920005710574658449355472279059253047286958131011730034346120775
76 0821737840042745530876391793484035024644475535353227851321505537398888106855
77 012746117
78 """, "\n", ""))
79
80 p, q = factorize(N, e, d)
81 phi = (p - 1) * (q - 1)
82
83 E = 1
84 for _, ei in friend_keys:
85 E *= ei
86
87 d_new = inverse(E, phi)
88 m = pow(cipher, d_new, N)
89
90 print(long_to_bytes(m))

```

## Script Python

```

● PS C:\Users\ASUS> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe c:/Users/ASUS/Downloads/Solution.py
b'crypto{3ncrypt_y0ur_s3cr3t_with_y0ur_fr1end5_public_k3y}'
○ PS C:\Users\ASUS>

```

## Challenge's flag

## XI. Everything is Still Big (CRYPTOHACK)

- **Challenge:** Trong challenge **Everything is Still Big**, mục tiêu là khai thác lỗ hổng trong quá trình sinh khóa của hệ mật mã RSA. Dựa vào file source.py được cung cấp, hệ thống đã thiết lập private key  $d$  quá nhỏ (512 bit) so với module  $N$  (2048 bit). Do vi phạm điều kiện an toàn  $d < N^{0.292}$ , attacker có thể áp dụng phương pháp Coppersmith kết hợp thuật toán thu gọn cơ sở ma trận LLL (Lenstra–Lenstra–Lovász) để khôi phục khóa bí mật  $d$ .



<https://cryptohack.org/challenges/rsa/>

- **Solution:** Lợi dụng lỗ hổng Boneh-Durfee đối với khóa  $d$  nhỏ, ta thực hiện việc xây dựng một ma trận (lattice) thông qua đa thức  $f(x, y) = 1 + x(A + y) \equiv 0 \pmod{e}$ . Do  $d$  và  $N$  trong bài thỏa mãn điều kiện  $d < N^{0.292}$ , ta có thể dùng thuật toán thu gọn cơ sở ma trận LLL (Lenstra–Lenstra–Lovász) để khôi phục khóa bí mật  $d$ . Sau đây là các bước thực hiện giải bài toán trên: Lấy source code mẫu khai thác Boneh-Durfee của mimoo tại kho lưu trữ Github [https://raw.githubusercontent.com/mimoo/RSA-and-LLL-attacks/master/boneh\\_durfee.sage](https://raw.githubusercontent.com/mimoo/RSA-and-LLL-attacks/master/boneh_durfee.sage). Đưa mã nguồn lên môi trường máy chủ tính toán SageMath Cloud. Để tự động hóa việc giải mã ngay khi tìm được khóa  $d$ , tiến hành xóa bỏ hàm example() ở cuối script gốc và thay thế bằng đoạn mã sau:

```

1 N = Điền module
2 e = Điền public key
3 c = Điền cipher text
4
5 delta = 0.26
6 m = 6
7 t = int((1-2*delta) * m)
8
9 X = 2*floor(N^delta)
10 Y = floor(N^(0.5))
11
12 PR.<x,y> = PolynomialRing(ZZ)
13 A = int((N+1)/2)
14 pol = 1 + x * (A + y)
15
16 solx, soly = boneh_durfee(pol, e, m, t, X, Y)
17
18 if solx:
19 d = int(pol(solx, soly) / e)
20
21 m_pt = power_mod(c, d, N)
22 flag_hex = hex(m_pt)[2:]
23
24 if len(flag_hex) % 2 != 0: flag_hex = "0" + flag_hex
25
26 print("\n[+] ===== FLAG =====")
27 print(bytes.fromhex(flag_hex).decode('ascii', errors='ignore'))
28 print("[+] =====")
29 else:
30 print("[-] Error")

```

## Script giải mã flag

About SageMathCell



Type some Sage code below and press Evaluate.

```

359 m_pt = power_mod(c, d, N)
360 flag_hex = hex(m_pt)[2:]
361
362 if len(flag_hex) % 2 != 0: flag_hex = "0" + flag_hex
363
364 print("\n[+] ===== FLAG =====")
365 print(bytes.fromhex(flag_hex).decode('ascii', errors='ignore'))
366 print("[+] =====")
367 else:
368 print("[-] Error")
369

```

Evaluate

Language Sage

```

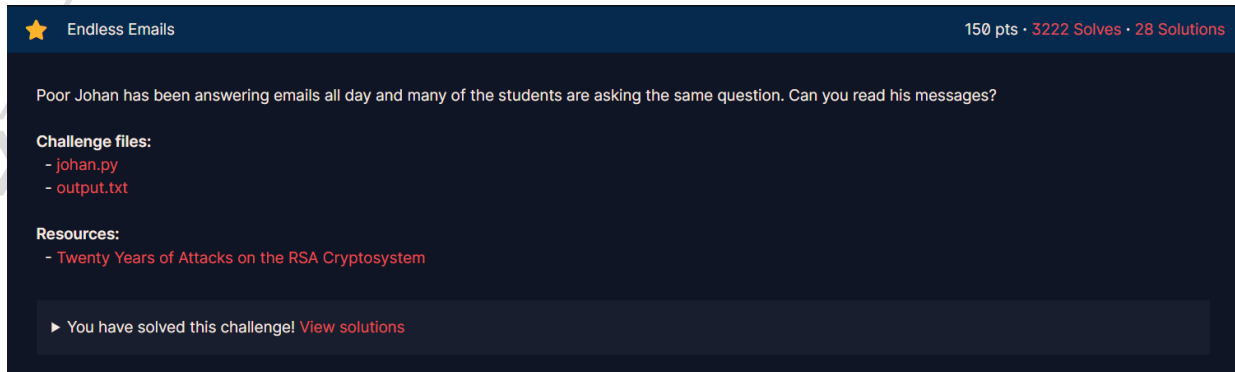
* removing unhelpful vectors 1 and 2
* removing unhelpful vector 0
14 / 30 - vectors are not helpful
det(L) < e^(m*n) (good! If a solution exists < N^delta, it will be found)
00 X 0 ~
01 X X 0 ~
02 X X X 0 ~
03 0 0 0 X 0 ~
04 0 0 0 X X 0 ~
05 0 0 0 X X X 0 ~
06 0 0 0 X X X X 0 ~
07 0 0 0 0 0 0 0 X 0 ~
08 0 0 0 0 0 0 0 0 X 0 ~
09 0 0 0 0 0 0 0 X X 0 ~
10 0 0 0 0 0 0 0 X X X 0 ~
11 0 0 0 0 0 0 0 X X X X 0 ~
12 0 0 0 0 0 0 0 0 X X X X X 0 ~
13 0 0 0 0 0 0 0 0 0 X X 0 ~
14 0 0 0 0 0 0 0 0 0 0 X X 0 ~
15 0 0 0 0 0 0 0 0 0 0 0 X X X 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ~
16 0 0 0 0 0 0 0 0 0 0 0 X X X X 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ~
17 0 0 0 0 0 0 0 0 0 0 0 0 X X X X X 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ~
18 0 0 0 0 0 0 0 0 0 0 0 0 0 0 X 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ~
19 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 X X 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ~
20 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 X X X 0 0 0 0 0 0 0 0 0 0 0 0 ~
21 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 X X X X 0 0 0 0 0 0 0 0 0 ~
22 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 X X X X X 0 0 0 0 0 0 0 ~
23 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 X X X X X 0 0 0 0 0 0 ~
24 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 X X X X X X 0 0 0 0 0 ~
25 X X X X X X 0 ~
26 0 0 0 X X X X X 0 ~
27 0 0 0 0 0 X X X X X X 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 ~
28 0 0 0 0 0 0 0 0 0 0 X X X X X X X 0 0 0 0 0 0 0 0 0 0 0 0 ~
29 0 0 0 0 0 0 X X X X X X X X X 0 0 0 0 0 0 0 0 0 0 0 0 ~
optimizing basis of the lattice via LLL, this can take a long time
LLL is done!
looking for independent vectors in the lattice
found them, using vectors 0 and 1

[+] ===== FLAG =====
crypto(bon3h5_4tt4ck_is_sr0ng3r_th4n_w13n35)
[+] =====

```

XII. Endless Emails (**CRYPTOHACK**)

- **Challenge:** Endless Emails, mục tiêu là khai thác lỗ hổng Håstad's Broadcast Attack trên hệ mật RSA. Dựa vào mã nguồn cung cấp, thông điệp được mã hóa với cùng một số mũ công khai rất nhỏ  $e = 3$  và được gửi đi nhiều lần, mỗi lần sử dụng một module  $N$  sinh ngẫu nhiên khác nhau. Điểm đánh lừa của challenge là không phải tất cả các bản mã đều chứa cùng một thông điệp chỉ có "many", không phải "all", đòi hỏi người giải mã phải lọc đúng các gói tin đồng nhất để tấn công.



<https://cryptohack.org/challenges/rsa/>

- **Solution:** Bài toán áp dụng kỹ thuật **Håstad's Broadcast Attack** do cùng một thông điệp  $m$  được mã hóa với số mũ nhỏ  $e = 3$  trên các module  $N$  khác biệt. Bằng cách chọn 3 bản mã chứa cùng thông điệp, ta lập được hệ phương trình đồng dư  $c_i \equiv m^3 \pmod{N_i}$ . Sử dụng Định lý số dư Trung Hoa (CRT), hệ này được gộp thành phương trình tổng:  $C \equiv m^3 \pmod{N_1 \cdot N_2 \cdot N_3}$ . Do bản chất RSA yêu cầu  $m < N_i$ , ta có  $m^3 < N_1 \cdot N_2 \cdot N_3$ . Khi giá trị  $m^3$  nhỏ hơn không gian module, phép chia dư bị triệt tiêu, trả lại đẳng thức số học đơn thuần  $C = m^3$ . Ta chỉ cần rút căn bậc 3 của  $C$  để khôi phục bản rõ  $m$ . Để lọc các thông điệp nhiều trong 7 gói tin đề cho, ta dùng tổ hợp chập 3 để vét cạn; nếu chọn đúng 3 gói tin đồng nhất,  $C$  sẽ là lập phương hoàn hảo, cho phép rút căn nguyên vẹn ra  $m$  và decode sang ASCII để lấy Flag.

```

1 n1 =
2 c1 =
3
4 n2 =
5 c2 =
6
7 n3 =
8 c3 =
9
10 n4 =
11 c4 =
12
13 n5 =
14 c5 =
15
16 n6 =
17 c6 =
18
19 n7 =
20 c7 =
21
22 from itertools import combinations
23
24 pairs = [
25 (c1, n1), (c2, n2), (c3, n3),
26 (c4, n4), (c5, n5), (c6, n6), (c7, n7)
27]
28
29 for combo in combinations(pairs, 3):
30 c_list = [combo[0][0], combo[1][0], combo[2][0]]
31 n_list = [combo[0][1], combo[1][1], combo[2][1]]
32
33 C = crt(c_list, n_list)
34
35 try:
36 m = Integer(C).nth_root(3)
37
38 flag_hex = hex(m)[2:]
39 if len(flag_hex) % 2 != 0:
40 flag_hex = "0" + flag_hex
41
42 flag_text = bytes.fromhex(flag_hex).decode('ascii', errors='ignore')
43
44 if "crypto{" in flag_text:
45 print("\n[+] ===== FLAG =====")
46 print(flag_text)
47 print("[+] =====")
48 break
49
50 except ValueError:
51 continue

```

## Script giải mã flag

[About SageMathCell](#)



Type some Sage code below and press Evaluate.

```

41
42 flag_text = bytes.fromhex(flag_hex).decode('ascii', errors='ignore')
43
44 if "crypto{" in flag_text:
45 print("\n[+] ===== FLAG =====")
46 print(flag_text)
47 print("[+] =====")
48 break
49
50 except ValueError:
51 continue

```

Evaluate

Language: Sage

```

[+] ===== FLAG =====
yes

Johan Hastad
Professor in Computer Science in the Theoretical Computer Science
Group at the School of Computer Science and Communication at KTH Royal Institute of Technology in Stockholm, Sweden.

crypto{1f_y0u_d0nt_p4d_y0u_4r3_Vuln3r4bl3}
[+] =====

```

[Help](#) | Powered by [SageMath](#)

## XIII. Vote for Pedro (CRYPTOHACK)

- **Challenge:** Vote for Pedro, mục tiêu là khai thác lỗ hổng giả mạo chữ ký số RSA thông qua kỹ thuật Bleichenbacher's  $e = 3$  Signature Forgery. Hệ thống yêu cầu người dùng gửi lên một chữ ký số dưới dạng Hex và tiến hành giải mã chữ ký đó bằng khóa công khai của hệ thống với số mũ  $e = 3$ . Tuy nhiên, quá trình kiểm tra tính toàn vẹn của chuẩn đệm PKCS#1 v1.5 được lập trình thiếu an toàn. Thay vì xác minh toàn bộ cấu trúc byte của khối dữ liệu (bắt đầu bằng 00 01, chuỗi padding FF, và byte ngắt 00), máy chủ sử dụng hàm `.split(b'\x00')[-1]` để cắt chuỗi và chỉ kiểm tra duy nhất phần hậu tố cuối cùng xem có khớp với đoạn văn bản VOTE FOR PEDRO hay không. Lỗ hổng logic này cho phép kẻ tấn công khởi tạo một chữ ký giả mạo chứa dữ liệu rác ở phần đầu mà vẫn lách qua được cơ chế xác thực.



<https://cryptohack.org/challenges/rsa/>

- **Solution:** Quá trình khai thác tập trung vào việc tạo ra một chữ ký  $s$  sao cho khối dữ liệu giải mã  $s^3 \bmod N$  kết thúc bằng đúng chuỗi đích VOTE FOR PEDRO. Do chuỗi có độ dài 15 bytes tương đương  $k = 120$  bit, bài toán giải mã được thu hẹp về việc đồng dư phần đuôi nhị phân:  $s^3 \equiv T \bmod 2^{120}$ . Lợi dụng việc số mũ  $e = 3$  là số lẻ và luôn có nghịch đảo modulo trong không gian lũy thừa của 2, ta áp dụng định lý Euler để tính khóa giải mã cục bộ  $d \equiv 3^{-1} \bmod 2^{119}$ . Chữ ký giả mạo sau đó được đúc ra từ công thức  $s \equiv \text{target}^d \bmod 2^{120}$ . Điểm mấu chốt là do tính toán trong không gian thu nhỏ  $2^{120}$ , giá trị lập phương  $s^3$  sinh ra cực kỳ bé và hoàn toàn nằm lọt thỏm bên trong module  $N$  khổng lồ của máy chủ. Phép mod  $N$  bị vô hiệu hóa, giúp bảo toàn tuyệt đối 120 bits phần đuôi. Cuối cùng, đóng gói chữ ký  $s$  dạng Hex vào payload JSON và đẩy qua kết nối socket, máy chủ sẽ ngắt đúng phần hậu tố hợp lệ và trả về Flag.

```

1 #!/usr/bin/env python3
2
3 from Crypto.Util.number import bytes_to_long, long_to_bytes
4 from utils import listener
5
6 FLAG = "crypto{????????????????}"
7
8
9 class Challenge():
10 def __init__(self):
11 self.before_input = "Place your vote. Pedro offers a reward to anyone who votes for him!\n"
12
13 def challenge(self, your_input):
14 if 'option' not in your_input:
15 return {"error": "You must send an option to this server"}
16
17 elif your_input['option'] == 'vote':
18 vote = int(your_input['vote'], 16)
19 verified_vote = long_to_bytes(pow(vote, ALICE_E, ALICE_N))
20
21 # remove padding
22 vote = verified_vote.split(b'\x00')[-1]
23
24 if vote == b'VOTE FOR PEDRO':
25 return {"flag": FLAG}
26 else:
27 return {"error": "You should have voted for Pedro"}
28
29 else:
30 return {"error": "Invalid option"}
31
32
33 import builtins; builtins.Challenge = Challenge # hack to enable challenge to be run locally, see https://cryptohack.org/faq/#listener
34 listener.start_server(port=13375)

```

## Script Challenge

```
1 import socket
2 import json
3 from Crypto.Util.number import bytes_to_long
4
5 target = b'\x00VOTE FOR PEDRO'
6
7 byte = bytes_to_long(target)
8 k = len(target) * 8
9 phi = 2**k
10 d = pow(3, -1, phi)
11 s = pow(byte, d, phi)
12
13 hex_signature = hex(s)[2:]
14
15 payload = {
16 "option": "vote",
17 "vote": hex_signature
18 }
19
20 HOST = 'socket.cryptohack.org'
21 PORT = 13375
22
23 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as conn:
24 conn.connect((HOST, PORT))
25
26 welcome_msg = conn.recv(1024)
27
28 data_to_send = json.dumps(payload) + "\n"
29 conn.sendall(data_to_send.encode())
30
31 print("\n[+] FLAG:")
32 response = conn.recv(1024)
33 print(response.decode())
```

### Script Python

```
PS C:\Users\ASUS> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe c:/Users/ASUS/Downloads/solve.py
[+] FLAG:
{"flag": "crypto{y0ur_v0t3_is_my_v0t3}"}
```

```
PS C:\Users\ASUS>
```

### Challenge's flag

## XIV. Blinding Light (CRYPTOHACK)

- **Challenge:** Blinding Light, mục tiêu là khai thác tính chất đồng cấu phép nhân của hệ mật RSA nguyên thủy thông qua kỹ thuật RSA Blinding Attack. Hệ thống yêu cầu cung cấp chữ ký số hợp lệ cho thông điệp đặc quyền ADMIN\_TOKEN = b"admin=True" để lấy Flag. Tuy nhiên, hàm sign của máy chủ được trang bị một bộ lọc kiểm duyệt: nếu phát hiện chuỗi byte admin=True nằm trong yêu cầu, máy chủ sẽ từ chối ký. Điểm yếu chết người của kiến trúc này là máy chủ vẫn mù quáng ký lên bất kỳ khối dữ liệu số nguyên nào khác mà không áp dụng các cơ chế đệm an toàn như PSS để phá vỡ cấu trúc đại số, tạo điều kiện cho kẻ tấn công sử dụng dụng toán học để "che mắt" bộ lọc.



<https://cryptohack.org/challenges/rsa/>

- **Solution:** Ý tưởng cốt lõi của bài toán là khai thác lỗ hổng RSA Blinding Attack để lách qua bộ lọc kiểm duyệt từ khóa của máy chủ. Do hệ thống cấm ký trực tiếp thông điệp admin=True, ta lợi dụng tính chất đồng cấu phép nhân của RSA để "ngụy trang" nó. Cụ thể, ta nhân thông điệp gốc với một mật nạ toán học sinh ngẫu nhiên dạng  $r^E \bmod N$  để biến nó thành một khối dữ liệu rác. Khối dữ liệu này dễ dàng qua mặt hàm kiểm duyệt, lừa máy chủ sử dụng khóa bí mật để ký lên nó. Sau khi nhận được chữ ký mù từ máy chủ, ta thực hiện quá trình "giải mù" bằng cách nhân nó với nghịch đảo modulo của mật nạ ban đầu  $r^{-1} \bmod N$ . Phép toán này sẽ tự động triệt tiêu lớp ngụy trang, bóc tách ra chữ ký nguyên bản và hoàn toàn hợp lệ của thông điệp admin=True. Cuối cùng, chỉ việc nộp chữ ký này vào hệ thống xác thực để lấy Flag.

```

1 #!/usr/bin/env python3
2
3 from Crypto.Util.number import bytes_to_long, long_to_bytes
4 from utils import listener
5
6 FLAG = "crypto{????????????????????}"
7 ADMIN_TOKEN = b"admin=True"
8
9
10 class Challenge():
11 def __init__(self):
12 self.before_input = "Watch out for the Blinding Light\n"
13
14 def challenge(self, your_input):
15 if 'option' not in your_input:
16 return {"error": "You must send an option to this server"}
17
18 elif your_input['option'] == 'get_pubkey':
19 return {"N": hex(N), "e": hex(E)}
20
21 elif your_input['option'] == 'sign':
22 msg_b = bytes.fromhex(your_input['msg'])
23 if ADMIN_TOKEN in msg_b:
24 return {"error": "You cannot sign an admin token"}
25
26 msg_i = bytes_to_long(msg_b)
27 return {"msg": your_input['msg'], "signature": hex(pow(msg_i, D, N))}
28
29 elif your_input['option'] == 'verify':
30 msg_b = bytes.fromhex(your_input['msg'])
31 msg_i = bytes_to_long(msg_b)
32 signature = int(your_input['signature'], 16)
33
34 if msg_i < 0 or msg_i > N:
35 # prevent attack where user submits admin token plus or minus N
36 return {"error": "Invalid msg"}
37
38 verified = pow(signature, E, N)
39 if msg_i == verified:
40 if long_to_bytes(msg_i) == ADMIN_TOKEN:
41 return {"response": FLAG}
42 else:
43 return {"response": "Valid signature"}
44 else:
45 return {"response": "Invalid signature"}
46
47 else:
48 return {"error": "Invalid option"}
49
50
51 import builtins; builtins.Challenge = Challenge # hack to enable challenge to be run locally, see https://cryptohack.org/faq/#listener
52 listener.start_server(port=13376)

```

## Script Challenge

```

1 import socket
2 import json
3 from Crypto.Util.number import bytes_to_long, inverse
4
5 HOST = 'socket.cryptohack.org'
6 PORT = 13376
7
8 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
9 s.connect((HOST, PORT))
10 f = s.makefile('rw')
11
12 def send_cmd(cmd_dict):
13 f.write(json.dumps(cmd_dict) + '\n')
14 f.flush()
15 return json.loads(f.readline().strip())
16
17 print("[Server] " + f.readline().strip())
18
19 pubkey = send_cmd({"option": "get_pubkey"})
20 N = int(pubkey["N"], 16)
21 E = int(pubkey["e"], 16)
22
23 target_msg = b"admin=True"
24 m = bytes_to_long(target_msg)
25
26 r = 2
27 blind = (m * pow(r, E, N)) % N
28 blind_hex = hex(blind)[2:]
29
30 sign_resp = send_cmd({"option": "sign", "msg": blind_hex})
31 s_blind = int(sign_resp["signature"], 16)
32
33 r_inv = inverse(r, N)
34 s = (s_blind * r_inv) % N
35 s_hex = hex(s)[2:]
36
37 target_msg_hex = target_msg.hex()
38 verify_resp = send_cmd({"option": "verify", "msg": target_msg_hex, "signature": s_hex})
39
40 print("\n[+] FLAG:")
41 print(verify_resp['response'])

```

## Script Python

```

● PS C:\Users\ASUS> & C:\Users\ASUS\AppData\Local\Programs\Python\Python314\python.exe c:/Users/ASUS/Downloads/solve.py
[Server] Watch out for the Blinding Light

[+] FLAG:
crypto{m1l34b1l17y_c4n_b3_d4n63r0u5}
○ PS C:\Users\ASUS>

```

## Challenge's flag

## XV. Parameter Injection (CRYPTOHACK)

- **Challenge:** Trong challenge **Parameter Injection**, mục tiêu là khai thác lỗ hổng trong quá trình trao đổi khóa Diffie–Hellman giữa Alice và Bob. Do các tham số DH (p, g, A, B) không được xác thực, attacker có thể đóng vai trò Man-in-the-Middle, chặn và chỉnh sửa các giá trị được gửi giữa hai bên. Lỗ hổng chính của hệ thống là thiếu cơ chế xác thực tính toàn vẹn của các tham số trao đổi khóa.

★
Parameter Injection
60 pts · 5287 Solves · 22 Solutions

You're in a position to not only intercept Alice and Bob's DH key exchange, but also rewrite their messages. Think about how you can play with the DH equation that they calculate, and therefore sidestep the need to crack any discrete logarithm problem.

Use the script from "Deriving Symmetric Keys" to decrypt the flag once you've recovered the shared secret.

Connect at `socket.crypthack.org 13371`

▶ You have solved this challenge! [View solutions](#)

<https://cryptohack.org/challenges/diffie-hellman/>

- **Solution:** Lợi dụng công thức tính shared secret  $K = B^a \bmod p$ , ta thực hiện việc đánh cắp gói tin của Bob gửi cho Alice, sau đó gửi lại cho Alice một B giả với giá trị là 1. B nhận giá trị 1 vì khi áp dụng vào công thức tính K ta sẽ được  $1^a \bmod p = 1$ . Sau đó Alice sẽ dùng chính K đó và trả lại cho ta encrypt flag. Vì đã biết sẵn khóa K nên ta chỉ cần sử dụng công cụ có sẵn do CRYPTOHACK cung cấp và giải mã flag.

```

hung@LAPTOP-RI9PA5KK:/mnt/c/Users/ASUS$ nc socket.cryptohack.org 13371
Intercepted from Alice: {"p": "0xffffffffffffffff90fdaa22168c234c4c6628b80dc1cd129024e088a67cc74020bbea63b139b22514a087
98e3404ddef9519b3cd3a431b302b0a6df25f14374fe1356d6d51c245e485b576625e7ec6f44c42e9a637ed6b0bfff5cb6f406b7edee386bfb5a899fa
5ae9f24117c4b1fe649286651ece45b3dc2007cb8a163bf0598da48361c55d39a69163fa8fd24cf5f83655d23dca3ad961c62f356208552bb9ed5290
77096966d670c354e4abc9804f1746c08ca237327ffffffffffffffff", "g": "0x02", "A": "0x804672202c338c51f3404b075c69a099ca0319e
690c4a6047f411c15bfff5b9de561c343211b55f84d2e61c7b67500e28b8d478a7e67ab26127d8b36b6bef3247e082fccc21e131798ee54a26956d7
ca608b0efa6de3393baebb99564e9f96a5d02bab0e8d82e15e8289a79b84aa4e646284e33724ae2341663bea07f7a5723fce057e869ad9a1c384bf39
596d07fdef50dd776271ee9619853d73df55112ae5cee5e25845d360059674ef21470f9cb2e1301d65f2f7468373139ce34292ce0"}
Send to Bob: {"p": "0xffffffffffffffff90fdaa22168c234c4c6628b80dc1cd129024e088a67cc74020bbea63b139b22514a08798e3404ddef
9519b3cd3a431b302b0a6df25f14374fe1356d6d51c245e485b576625e7ec6f44c42e9a637ed6b0bfff5cb6f406b7edee386bfb5a899fa5ae9f24117c
4b1fe649286651ece45b3dc2007cb8a163bf0598da48361c55d39a69163fa8fd24cf5f83655d23dca3ad961c62f356208552bb9ed529077096966d67
0c354e4abc9804f1746c08ca237327ffffffffffffffff", "g": "0x02", "A": "0xc6f0cc38ab58a2afdd0ae2b3f7ec91e10ac48b3e33844975
38b1b93fe01bf20b50f7ffc7b29c8287434dd27f6886ca87891e15a1a7403e9505b2c5be8bdd4ca1622b6f28b45a5767aae656d4539ea75973003873
b064c900a7ef31ffa54b679e727cbfcc1fa6d894ca68f91b615a2cb359406e2a255c028e5cd248c6c791def02ceaaab73c33797c2c17b8be8a7c77c
d5f67d8afe4d093a83db692c0e380383c0d8d38af31d87116496e25fe264b4b6ba71d418abf5af3a1936669c87f578"}
Intercepted from Bob: {"B": "0x21762a89c5cf129430470af815f62c60d5f014a6717e8acb3454f54f93f3bc76feffddabb1a8526b6a94016a3e
958800b926de87009cea45f6276f73555027ad75e202b26461910121fd2d48fb79c7761c78049bc9d2d2a406c002913879712b9579eb519b79a7faf2
78959bbc9ababf6e958d02dfec6ac4f08634465710e194180d9f1016c0016063caee4c5bc2e87997c86da687aa871d5c9c914254341366d616f4b0d
29244bc8eaca0f8999ca5562c9c6aa50c4f70f67b8ff3db914fdf"}
Send to Alice: {"B": "0x1"}
Intercepted from Alice: {"iv": "fbf61ec17bbaa1b1daaa6f7bdc2a9c6b", "encrypted_flag": "6e3bf4ae45bbaad6560b62bce8edf50297
4cfe1ff1bbad9dd93777f9e045ae9e"}

```

Quá trình thực hiện (thực hiện trực tiếp trên terminal)

```

1 from Crypto.Cipher import AES
2 from Crypto.Util.Padding import pad, unpad
3 import hashlib
4
5
6 def is_pkcs7_padded(message):
7 padding = message[-message[-1]:]
8 return all(padding[i] == len(padding) for i in range(0, len(padding)))
9
10
11 def decrypt_flag(shared_secret: int, iv: str, ciphertext: str):
12 # Derive AES key from shared secret
13 sha1 = hashlib.sha1()
14 sha1.update(str(shared_secret).encode('ascii'))
15 key = sha1.digest()[:16]
16 # Decrypt flag
17 ciphertext = bytes.fromhex(ciphertext)
18 iv = bytes.fromhex(iv)
19 cipher = AES.new(key, AES.MODE_CBC, iv)
20 plaintext = cipher.decrypt(ciphertext)
21
22 if is_pkcs7_padded(plaintext):
23 return unpad(plaintext, 16).decode('ascii')
24 else:
25 return plaintext.decode('ascii')
26
27 g = 0x02
28 p = 0xffffffffffffffffc90fdaa22168c234c4c6628b80dc1cd129024e088a67cc74020bbea6
29 A = 0xecb915e9548db0b4b7c451a9e9f1842cfff46e6b3d343eaf96de7cca8fe8f879aa6803b39
30
31 shared_secret = 1
32 iv = "fbf61ec17bbaa1b1daaa6f7bdc2a9c6b"
33 ciphertext = "6e3bfae454bbaad6560b62bce8edf50297c4fe1ff1bbad9dd93777f9e045ae96
34
35 print(decrypt_flag(shared_secret, iv, ciphertext))

```

Công cụ giải mã do CRYPTOHACK cung cấp

```

PS C:\Users\ASUS> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe c:/Users/ASUS/Downloads/decrypt_08c0fede9185868aba4a6ae21aca
crypto{n1c3_0n3_m4ll0ry!!!!!!!}

```

Challenge's flag

## XVI. Export-grade (CRYPTOHACK)

- **Challenge:** Challenge **Export-grade** của CryptoHack mô phỏng một tình huống **man-in-the-middle** trong quá trình thương lượng tham số Diffie–Hellman giữa Alice và Bob. Do cả hai đang dùng hệ thống cũ, họ hỗ trợ nhiều mức bảo mật khác nhau như DH1536, DH1024, DH512... thậm chí DH64. Mục tiêu hacker đứng giữa và có thể can thiệp vào bước “parameter negotiation” là chỉnh sửa danh sách thuật toán được hỗ trợ trước khi hai bên chọn nhóm DH.

Export-grade

100 pts • 4545 Solves • 19 Solutions

Alice and Bob are using legacy codebases and need to negotiate parameters they both support. You've man-in-the-middled this negotiation step, and can passively observe thereafter. How are you going to ruin their day this time?

Connect at [socket.cryptohack.org](https://socket.cryptohack.org) 13379

► You have solved this challenge! [View solutions](#)

<https://cryptohack.org/challenges/diffie-hellman/>

- **Solution:** Ta thực hiện một cuộc tấn công hạ cấp trong bước thương lượng tham số Diffie–Hellman. Ban đầu Alice gửi danh sách các nhóm DH được hỗ trợ, ta can thiệp và chỉnh sửa gói tin này để chỉ còn nhóm yếu DH64 trước khi chuyển tiếp cho Bob. Vì Bob chỉ thấy DH64 nên buộc phải chọn nhóm này và gửi lại tham số công khai tương ứng. Sau đó ta tiếp tục chuyển tiếp đúng thông tin cho Alice để hai bên hoàn tất trao đổi khóa trong nhóm 64-bit. Do kích thước  $p$  chỉ 64-bit, bài toán discrete logarithm trở nên khả thi. Từ giá trị công khai  $A = g^a \bmod p$ , ta giải discrete log để tìm số mũ bí mật  $a$ . Khi đã biết  $a$ , ta tính shared secret  $K = B^a \bmod p$ . Từ khóa bí mật  $K$  này, ta sử dụng công cụ từ bài Parameter Injection thêm vào đó đoạn khai thác discrete log problem, từ đó khôi phục flag.

```

hung@LAPTOP-RI9PA5KK:/mnt/c/Users/ASUS$ nc socket.cryptohack.org 13379
Intercepted from Alice: {"supported": ["DH1536", "DH1024", "DH512", "DH256", "DH128", "DH64"]}
Send to Bob: {"supported": ["DH64", "DH64", "DH64", "DH64", "DH64", "DH64"]}
Intercepted from Bob: {"chosen": "DH64"}
Send to Alice: {"chosen": "DH64"}
Intercepted from Alice: {"p": "0xde26ab651b92a129", "g": "0x2", "A": "0x353d81fea9930428"}
Intercepted from Bob: {"B": "0x9bfe80464f37b7be"}
Intercepted from Alice: {"iv": "622aab75f485cd1b09ef7a595906b753", "encrypted_flag": "df3e8bde0f9a35f9b584d53ddb94c9b048a046c0247dcc160a7d373a875dde53"}

```

**Quá trình thực hiện (thực hiện trực tiếp trên terminal)**

```

1 from Crypto.Cipher import AES
2 from Crypto.Util.Padding import pad, unpad
3 import hashlib
4 from sympy import discrete_log
5
6 def is_pkcs7_padded(message):
7 padding = message[-message[-1]:]
8 return all(padding[i] == len(padding) for i in range(0, len(padding)))
9
10
11 def decrypt_flag(shared_secret: int, iv: str, ciphertext: str):
12 # Derive AES key from shared secret
13 sha1 = hashlib.sha1()
14 sha1.update(str(shared_secret).encode('ascii'))
15 key = sha1.digest()[:16]
16 # Decrypt flag
17 ciphertext = bytes.fromhex(ciphertext)
18 iv = bytes.fromhex(iv)
19 cipher = AES.new(key, AES.MODE_CBC, iv)
20 plaintext = cipher.decrypt(ciphertext)
21
22 if is_pkcs7_padded(plaintext):
23 return unpad(plaintext, 16).decode('ascii')
24 else:
25 return plaintext.decode('ascii')
26
27 g = 0x02
28 p = 0xde26ab651b92a129
29 A = 0x353d81fea9930428
30 B = 0x9bfe80464f37b7be
31 a = discrete_log(p, A, g)
32
33 shared_secret = pow(B, a, p)
34 iv = "622aab75f485cd1b09ef7a595906b753"
35 ciphertext = "df3e8bde0f9a35f9b584d53ddb94c9b048a046c0247dcc160a7d373a875dde53"
36
37 print(decrypt_flag(shared_secret, iv, ciphertext))

```

Công cụ giải mã do CRYPTOHACK cung cấp

```

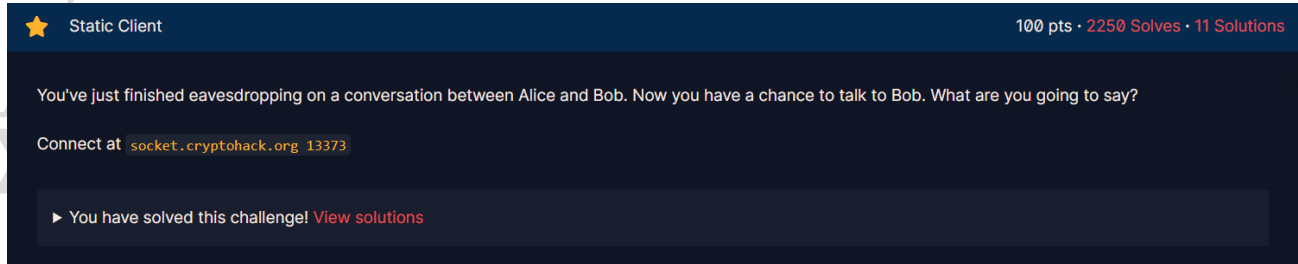
PS C:\Users\ASUS> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe c:/Users/ASUS/Downloads/decrypt_08c0fede9185868aba4a6ae21aca0148.py
crypto{d0wn6r4d35_4r3_d4n63r0u5}

```

Challenge's flag

## XVII. Static Client (CRYPTOHACK)

- **Challenge:** Static Client của CryptoHack mô phỏng tình huống hacker đã nghe lén cuộc trao đổi khóa giữa Alice và Bob, và giờ có cơ hội gửi một thông điệp trực tiếp cho Bob. Điểm mấu chốt là Bob sử dụng khóa hoặc tham số cố định (static), nên nếu phân tích đúng quá trình Diffie–Hellman trước đó, hacker có thể tính được shared secret. Từ đó, hacker tạo một thông điệp hợp lệ (được mã hóa đúng cách) để đánh lừa Bob và lấy flag.



<https://cryptohack.org/challenges/diffie-hellman/>

- **Solution:** Vì trong bài này ta được phép gửi tham số cho Bob có nghĩa là Bob sẽ lấy tham số đó để tính khóa public của mình. Bob sẽ tính  $B = g^b \mod p$ , ta thấy được lỗ hổng ở đây là nếu ta truyền tham số  $g = A$  thì ta sẽ có được chính công thức để tính khóa  $K = A^b \mod p$ . Ta lợi dụng khóa B của Bob để lấy được khóa K. Sau khi lấy được khóa K ta dùng công cụ đã có sẵn của CRYPTOHACK để giải mã nốt phần còn lại.

```

hung@LAPTOP-RI9PA5MK:/mnt/c/Users/ASUS$ nc socket.cryptohack.org 13373
Intercepted from Alice: {"p": "0xffffffffffffffffc90fdaa22168c234c4c6628b80dc1cd129024e88a67cc74020bbea63b139b22514a08798e3404ddef9519b3cd3a431b302b0a6df25
f14374fe1356d6d51c245e485b576625ec6f44c42e9a637ed6b0bfff5cb6f406b7edee386bf5a899fa5ae9f24117c4b1fe649286651ece45b3dc2007cb8a163bf0598da48361c55d39a69163fa
8fd24cf5f83655d23dca3ad961c62f356208552bb9ed529077096966d670c354e4abc9804f1746c08ca237327ffffffffffffffff", "g": "0x02", "A": "0x4984b7b2211ca5efed7e46c30ae
cc03e624e3f5aeb3af9b298d0f34352c270dde427d70686c6c4d94f44f29ad69c226a57ef1e6e98a1a1bfc0572b1c0d9777f4f378982617f5f0f049738f329fef5a62c9d7d8b7bf45d69cda62
623360b2c50a394ee9faa878e8388b528a011657fa8d74a4f5048ed1648c713f281f5d5baf5ed5ed9dd73eda4c81164e6c9ae03b19ec8061e5afab75b45ebfb660d23cdac87b0e6307af6077bd5c
34b5730754cfd5efe09532f04df0c6c337151b50b55e2"}
Intercepted from Bob: {"B": "0x8d79b69390f639501d81bdce911ec9defb0e93d421c02958c8c8dd4e245e61ae861ef9d32aa85dfec628d4046c403199297d6e17f0c9555137b5e8555eb94
1e8dcfd2fe5e68eef666c6b0de91eb8cf2fd0c0f3f47e0c89779276fa7138e138793020c6b8f834be20a1623790ec108f23f872a5f693ca3f93c3fd5a853dfd69518eb4bab9ac2a004d3a11fb
21307149e8f2e1d8e1d7c85d604aa0bee335eade60f191f74ee165cd4baa067b96385aa89cb7722e7426522381fc94ebfa8ef0"}
Intercepted from Alice: {"iv": "d521d0e9a5d2c7b16040fe4bf7a9d862", "encrypted": "3be165ba0c30b2c5975963b1d0f632714491bda70c2c7310b1dcdc5355feace0"}
Bob connects to you, send him some parameters: {"p": "0xffffffffffffffffc90fdaa22168c234c4c6628b80dc1cd129024e88a67cc74020bbea63b139b22514a08798e3404ddef95
19b3cd3a431b302b0a6df25f14374fe1356d6d51c245e485b576625ec6f44c42e9a637ed6b0bfff5cb6f406b7edee386bf5a899fa5ae9f24117c4b1fe649286651ece45b3dc2007cb8a163bf05
98da48361c55d39a69163fa8fd24cf5f83655d23dca3ad961c62f356208552bb9ed529077096966d670c354e4abc9804f1746c08ca237327ffffffffffffffff", "g": "0x4984b7b2211ca5efe
d7e46c30aee03e624e3f5aeb3af9b298d0f34352c270dde427d70686c6c4d94f44f29ad69c226a57ef1e6e98a1a1bfc0572b1c0d9777f4f378982617f5f0f049738f329fef5a62c9d7d8b7bf
45d69cda62623360b2c50a394ee9faa878e8388b528a011657fa8d74a4f5048ed1648c713f281f5d5baf5ed5ed9dd73eda4c81164e6c9ae03b19ec8061e5afab75b45ebfb660d23cdac87b0e6307
af6077bd5c34b5730754cfd5efe09532f04df0c6c337151b50b55e2", "A": "0x0"}
Bob says to you: {"B": "0xd5bf7b5806631481ddfd3580147c0db36751d39b810ce54133151427f14d27a1b2dd20786e6a6626717dc1925cfe43db705185c2d422d556f9afebe6c07c502f6a
26709c75bf78e12161772d2f9403d5da7cd37fd9f1aedd8c05e706c4858826537c9b0c1c0cbdb344fb4160e342c5085e94f98da6a48f19e96d6dc7566153c71f5e6654dd47f0cc2b8dc1f42be7e
81c9973cb7344339bd205e4370f1f37245f1f8841b2e328b9bf384045ad564f34cbd2a8ea5d633b60d8688b2ead0b8c68e"}
Bob says to you: {"iv": "7e2d5912383ccb5fcella6ee333d400", "encrypted": "08d8da4c915238bcb8960a3f736c8bf49a9f7a7384c2f502127a3c4631f57f5cfa6c3cae51946a1a1
efe2b8f778eb60c2c7b52bd733ca2824bc28265fb531a07174dce176aa4bce2f3f8d0cba086d61"}

```

Quá trình thực hiện (thực hiện trực tiếp trên terminal)

```

1 from Crypto.Cipher import AES
2 from Crypto.Util.Padding import pad, unpad
3 import hashlib
4 from sympy import discrete_log
5
6 def is_pkcs7_padded(message):
7 padding = message[-message[-1]:]
8 return all(padding[i] == len(padding) for i in range(0, len(padding)))
9
10
11 def decrypt_flag(shared_secret: int, iv: str, ciphertext: str):
12 # Derive AES key from shared secret
13 sha1 = hashlib.sha1()
14 sha1.update(str(shared_secret).encode('ascii'))
15 key = sha1.digest()[:16]
16 # Decrypt flag
17 ciphertext = bytes.fromhex(ciphertext)
18 iv = bytes.fromhex(iv)
19 cipher = AES.new(key, AES.MODE_CBC, iv)
20 plaintext = cipher.decrypt(ciphertext)
21
22 if is_pkcs7_padded(plaintext):
23 return unpad(plaintext, 16).decode('ascii')
24 else:
25 return plaintext.decode('ascii')
26
27 B = "0xd5bf7b5806631481ddfd3580147c0db36751d39b810ce54133151427f14d27a1b2dd207" \
28 "86e6a6626717dc1925cfe43db705185c2d422d556f9afebe6c07c502f6a26709c75bf78e12161" \
29 "772d2f9403d5da7cd37fd9ef1aedd8c05e706c4858826537c9b0c1c0cbdb344fb4160e342c508" \
30 "5e94f98da6a48f19e96d6dc7566153c71f5e6654dd47f0cc2b8dc1f42be7e81c9973cb7344339" \
31 "bd205e4370f1f37245f1f8841b2e328b9bf384045ad564f34cbd2a8ea5d633b60d8688b2ead0b" \
32 "8c68e"
33
34 shared_secret = int(B, 16)
35 iv = "d521d0e9a5d2c7b16040fe4bf7a9d862"
36 ciphertext = "3be165ba0c30b2c5975963b1d0f632714491bda70c2c7310b1dcdc5355feace0"
37
38 print(decrypt_flag(shared_secret, iv, ciphertext))

```

Công cụ giải mã do CRYPTOHACK cung cấp

```

PS C:\Users\ASUS> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe c:/Users/ASUS/Downloads/decrypt_08c0fede9185868aba4a6ae21aca0148.py
crypto{n07_3ph3m3r41_3n0u6h}

```

Challenge's flag

## XVIII. MD0 (CRYPTOHACK)

- **Challenge:** Bài Length Extension – MD0 trên CryptoHack mô phỏng một cơ chế xác thực thông điệp (MAC – Message Authentication Code). Server cho phép người dùng gửi một message để nhận chữ ký thông qua chức năng sign, sau đó có thể gửi lại message cùng chữ ký để yêu cầu get\_flag. Ý tưởng của hệ thống là chỉ những message có chữ ký hợp lệ mới được chấp nhận.



<https://cryptohack.org/challenges/hashes/>

Tuy nhiên, cơ chế MAC trong bài không sử dụng chuẩn HMAC mà được xây dựng theo dạng chaining tương tự cấu trúc Merkle–Damgård hash construction. Điều này có thể dẫn đến một lỗ hổng mật mã học phổ biến gọi là Length Extension Attack. Khi attacker biết message ban đầu và MAC của nó, họ có thể tạo ra một message dài hơn (bằng cách thêm dữ liệu vào cuối) và tính được MAC hợp lệ mà không cần biết secret key. Mục tiêu của bài là lợi dụng lỗ hổng này để mở rộng message ban đầu bằng cách thêm nội dung như admin=True nhằm đánh lừa server và lấy flag.

```

1 from Crypto.Cipher import AES
2 from Crypto.Util.Padding import pad
3 import os
4 from utils import listener
5
6
7 FLAG = "crypto{????????????????}"
8
9
10 def bxor(a, b):
11 return bytes(x ^ y for x, y in zip(a, b))
12
13
14 def hash(data):
15 data = pad(data, 16)
16 out = b"\x00" * 16
17 for i in range(0, len(data), 16):
18 blk = data[i:i+16]
19 out = bxor(AES.new(blk, AES.MODE_ECB).encrypt(out), out)
20 return out
21
22
23 class Challenge():
24 def __init__(self):
25 self.before_input = "You'll never forge my signatures!\n"
26 self.key = os.urandom(16)
27
28 def challenge(self, msg):
29 if "option" not in msg:
30 return {"error": "You must send an option to this server."}
31
32 elif msg["option"] == "sign":
33 data = bytes.fromhex(msg["message"])
34 if b"admin=True" in data:
35 return {"error": "Unauthorized to sign message"}
36 sig = hash(self.key + data)
37
38 return {"signature": sig.hex()}
39
40 elif msg["option"] == "get_flag":
41 sent_sig = bytes.fromhex(msg["signature"])
42 data = bytes.fromhex(msg["message"])
43 real_sig = hash(self.key + data)
44
45 if real_sig != sent_sig:
46 return {"error": "Invalid signature"}
47
48 if b"admin=True" in data:
49 return {"flag": FLAG}
50 else:
51 return {"error": "Unauthorized to get flag"}
52
53 else:
54 return {"error": "Invalid option"}
55
56
57 import builtins; builtins.Challenge = Challenge # hack to enable challenge to be run locally.
58 """
59 When you connect, the 'challenge' function will be called on your JSON
60 input.
61 """
62 listener.start_server(port=13388)
63

```

—

```
hung@LAPTOP-RI9PA5KK:/mnt/c/Users/ASUS$ nc socket.cryptohack.org 13388
You'll never forge my signatures!
{"option": "sign", "message": "4354460A"}
{"signature": "30d45584b2055056d8be1e27f2e9d527"}
```

```
1 from Crypto.Cipher import AES
2
3 def bxor(a, b):
4 return bytes([x ^ y for x, y in zip(a, b)])
5
6 old_mac = bytes.fromhex("30d45584b2055056d8be1e27f2e9d527")
7 extend_msg = b"admin=True\n" + b"\x05" * 5
8 new_mac = bxor(AES.new(extend_msg, AES.MODE_ECB).encrypt(old_mac), old_mac)
9
10 print(new_mac.hex())
```

```
PS C:\Users\ASUS> & C:/Users/ASUS/AppData/Local/Programs/Python/Python314/python.exe
95deee41ce1c5077a32465fe9faf8de6
```

## Solution files

```

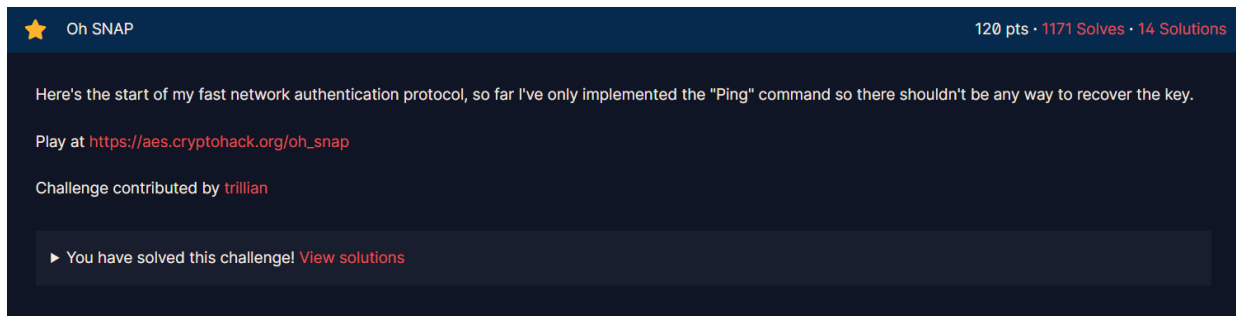
hung@LAPTOP-RI9PA5KK: /mnt/c/Users/ASUS$ nc socket.cryptohack.org 13388
You'll never forge my signatures!
{"option": "sign", "message": "4354460A"}
{"signature": "30d45584b2055056d8be127f2e9d527"}
{"option": "get_flag", "message": "4354460a0c0c0c0c0c0c0c0c1646d696e3d547275650a", "signature": "95deee41ce1c5077a32465fe9faf8de6"}
{"flag": "crypto{!3ngth_3xT3nd3r}"}

```

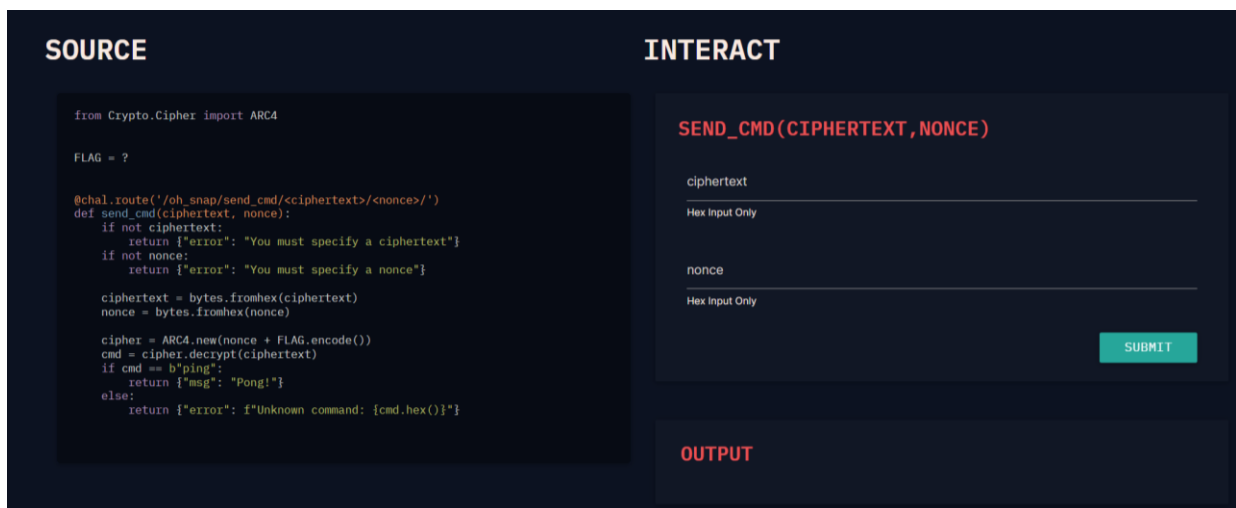
## Challenge's flag

## XIX. Oh SNAP (CRYPTOHACK)

- **Challenge:** Thách thức của bài toán "Oh SNAP" nằm ở việc hệ thống sử dụng giao thức xác thực dựa trên thuật toán mã hóa luồng RC4 với cấu trúc khóa không an toàn là sự kết hợp nối chuỗi giữa một nonce công khai và FLAG bí mật theo dạng nonce + FLAG. Do đặc điểm của thuật toán khởi tạo khóa (KSA) trong RC4, việc đặt các byte nonce đứng trước tạo ra các Weak IVs khiến các byte đầu tiên của luồng khóa (keystream) bị rò rỉ thông tin về các byte tiếp theo của FLAG ngay từ những vòng lặp đầu tiên. Bên cạnh đó, server còn mắc lỗi bảo mật khi thực hiện giải mã và trả về giá trị bản rõ trực tiếp trong thông báo lỗi "Unknown command", cho phép kẻ tấn công gửi bản mã toàn số không để triệt tiêu bản rõ thông qua phép toán XOR và lấy được chính xác byte đầu tiên của keystream. Chính sự kết hợp giữa một thuật toán có lỗ hổng thiết kế và việc rò rỉ dữ liệu qua thông báo lỗi đã tạo điều kiện cho một cuộc tấn công Known-Plaintext Attack để khai thác thông tin từ keystream.



<https://cryptohack.org/challenges/aes/>



## Script Challenge

- **Solution:** Để giải quyết bài toán, phương pháp tấn công thống kê FMS đã được triển khai nhằm khôi phục từng byte của FLAG một cách tuần tự thay vì tìm kiếm toàn bộ chuỗi cùng lúc. Quy trình bắt đầu bằng việc sử dụng một script Python để gửi hàng loạt yêu cầu tới server với các nonce yếu khác nhau kèm bản mã 00 nhằm thu thập các byte đầu tiên của keystream từ phản hồi của hệ thống. Dựa trên các byte keystream thu được và mô phỏng trạng thái của mảng S trong RC4 sau khi xử lý phần nonce, một hệ thống "bỏ phiếu" được thiết lập để dự đoán giá trị có xác suất xuất hiện cao nhất cho byte tiếp theo của khóa. Khi thực hiện lặp lại quy trình này cho từng vị trí, các ký tự của FLAG dần lộ diện mặc dù nhiều thống kê ở các byte cuối có thể tạo ra các ký tự không chính xác. Cuối cùng, bằng cách kết hợp giữa kết quả tính toán tự động và khả năng suy luận của con người để nhận diện cụm từ có nghĩa như "privacy" từ chuỗi ký tự nhiễu, mã FLAG hoàn chỉnh crypto{w1R3d\_equ1v4l3nt\_pr1v4cy} đã được khôi phục thành công.

```

1 import requests
2 from tqdm import tqdm
3
4 BASE_URL = "http://aes.cryptohack.org/oh_snap/send_cmd/"
5
6 def get_keystream_byte(ciphertext_hex, nonce_hex):
7 r = requests.get(f"{BASE_URL}{ciphertext_hex}/{nonce_hex}/")
8 data = r.json()
9 if "error" in data:
10 res_hex = data["error"].split(": ")[1]
11 return int(res_hex[:2], 16)
12 return None
13
14 def solve():
15 known_key = []
16 for byte_idx in range(40):
17 prob = [0] * 256
18 A = byte_idx + 3
19 for X in tqdm(range(256), desc=f"Finding byte {byte_idx}"):
20 nonce = [A, 255, X]
21 nonce_hex = "".join(f"{b:02x}" for b in nonce)
22
23 z = get_keystream_byte("00", nonce_hex)
24 if z is None: continue
25
26 S = list(range(256))
27 j = 0
28 key = nonce + known_key
29 for i in range(A):
30 j = (j + S[i] + key[i]) % 256
31 S[i], S[j] = S[j], S[i]
32
33 predicted_key_byte = (z - j - S[A]) % 256
34 prob[predicted_key_byte] += 1
35
36 best_guess = prob.index(max(prob))
37 known_key.append(best_guess)
38
39 current_flag = "".join(chr(b) for b in known_key)
40 print(f"\n[+] Progress: {current_flag}")
41
42 if current_flag.endswith("}"):
43 break
44
45 if __name__ == "__main__":
46 solve()

```

Script Python

```

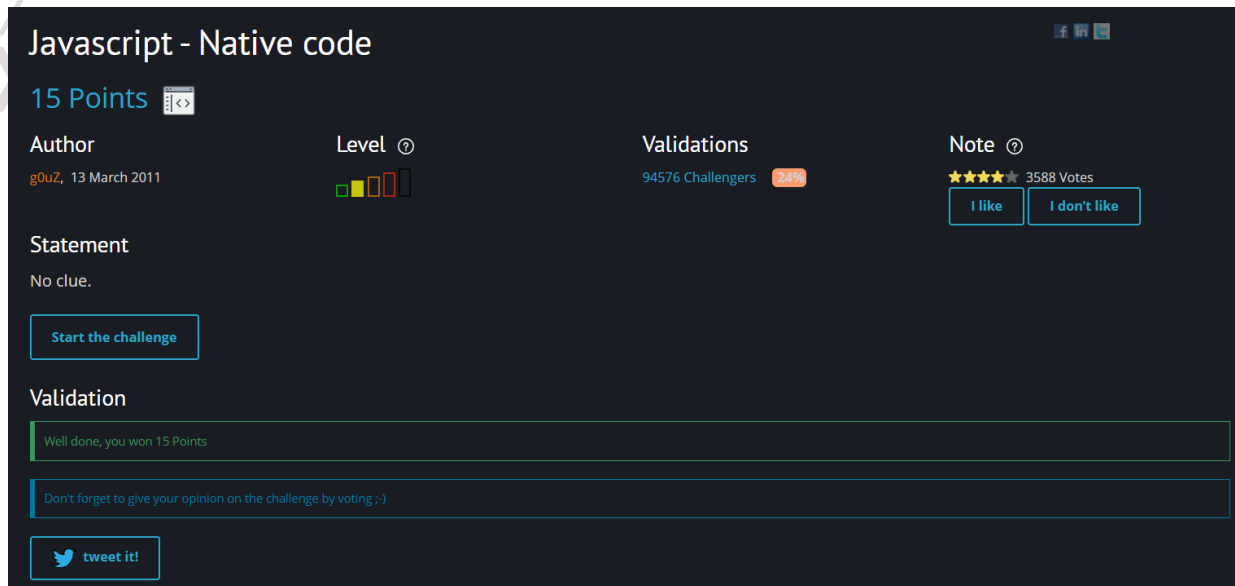
[*] Progress: crypto{w1R3d_equ1v4l3nt_@f1v4cy?!}
PS G:\My Drive\NT219\Demo>

```

Challenge's flag

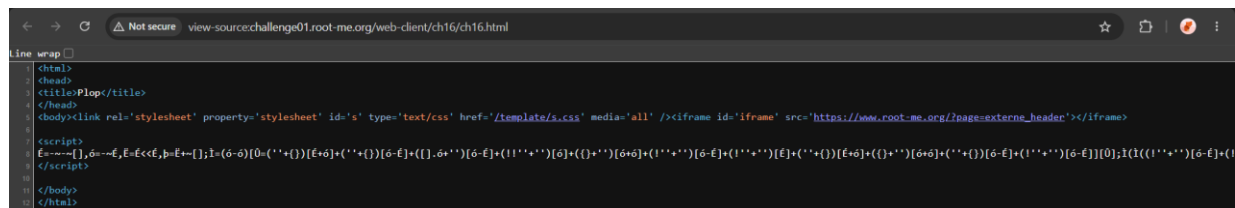
XX. Javascript – Native code (**Root Me**)

- **Challenge:** Javascript - Native code, mục tiêu của thử thách là khai thác cơ chế xử lý mã nguồn nguyên bản của trình duyệt để dịch ngược một đoạn mã JavaScript đã bị làm rối. Khi truy cập vào mã nguồn của trang web, người chơi không thấy các thẻ HTML thông thường mà chỉ có một thẻ <script> chứa một chuỗi ký tự dị dạng được cấu tạo từ các toán tử và dấu ngoặc kỹ thuật JSFuck/JJencode. Ứng dụng sẽ tự động giải mã và thực thi chuỗi này ngay lập tức để ẩn giấu logic xác thực mật khẩu người dùng. Thử thách đặt ra là phải tìm cách ngăn chặn quá trình thực thi tự động này và ép trình duyệt phải nhả ra đoạn mã nguồn gốc nằm bên dưới.



<https://www.root-me.org/en/Challenges/Web-Client/Javascript-Native-code>

- **Solution:** Kỹ thuật khai thác tận dụng chính cơ chế đánh giá biểu thức của JavaScript Engine trên trình duyệt. Đoạn mã làm rối thực chất là một hàm vô danh được gọi tự động thông qua cặp dấu ngoặc () nằm ở tận cùng chuỗi mã. Thay vì tốn thời gian viết script dịch ngược, ta tiến hành sao chép toàn bộ mã này vào Developer Console và xóa bỏ cặp ngoặc () ở vị trí cuối cùng. Việc tước bỏ cặp ngoặc này giúp ngăn chặn hàm được kích hoạt. Lúc này, theo cơ chế hoạt động mặc định, trình duyệt sẽ đánh giá đối tượng Function và tự động in ra chuỗi đại diện của hàm đó. Kết quả, một hàm JavaScript nguyên bản lộ diện trên Console, bên trong chứa rõ luồng kiểm tra: `if(a=='toto123Lol')`. Ta trích xuất được mật khẩu hardcoded là `toto123Lol` và nộp lên hệ thống Root-Me để hoàn thành bài lab.



Ctrl + U

## Console



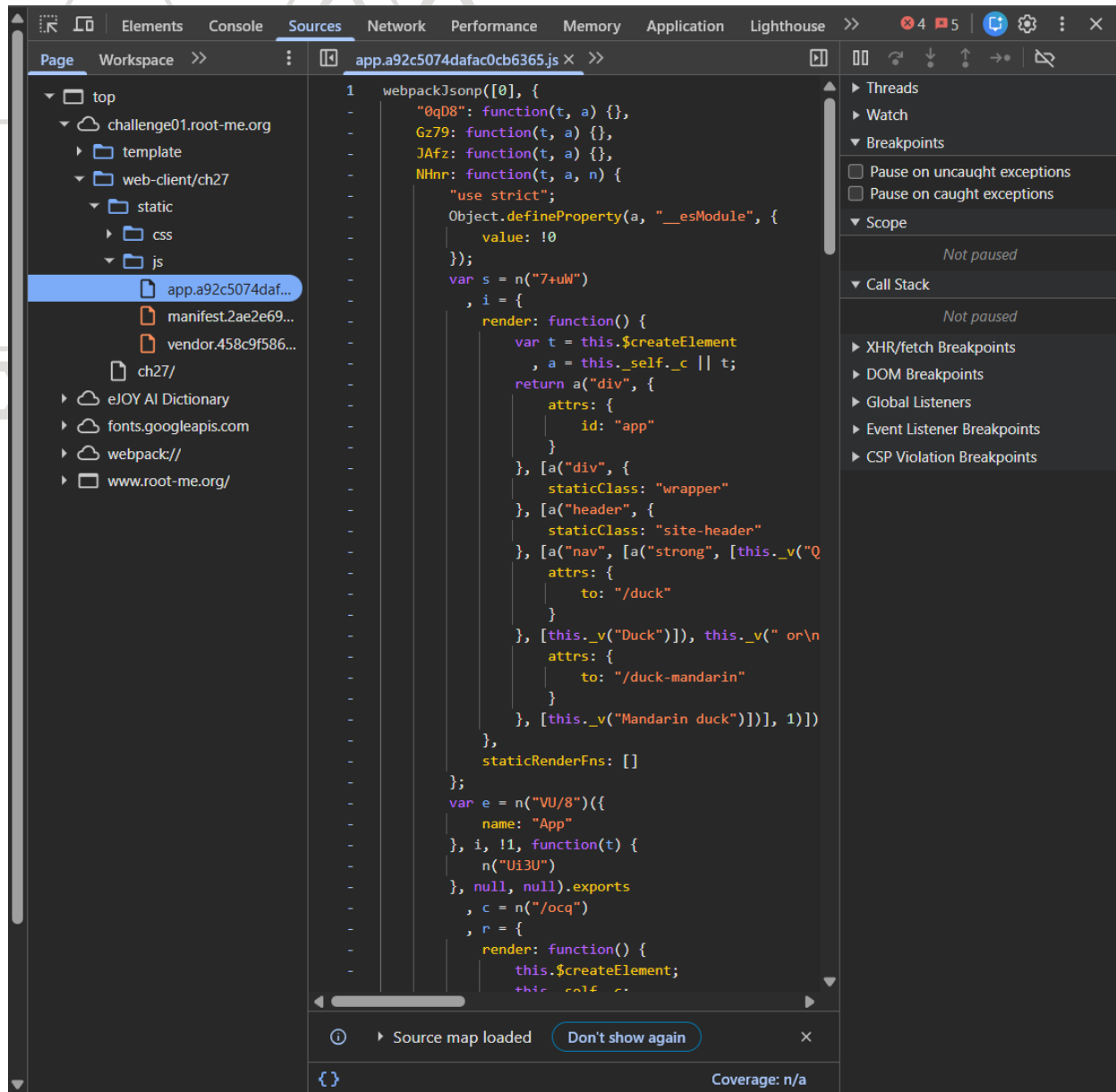
XXI. Javascript – Webpack (**Root Me**)

- **Challenge:** Javascript – Webpack, mục tiêu của thử thách là khai thác lỗ hổng lộ lọt cấu hình mã nguồn của các ứng dụng Web hiện đại được đóng gói bởi Webpack. Trong kiến trúc Production tiêu chuẩn, toàn bộ mã nguồn Frontend như các file components .vue, .js sẽ được tối ưu hóa bằng cách nén và làm rối thành một tệp bundle duy nhất để che giấu logic nghiệp vụ. Tuy nhiên, nếu nhà phát triển cấu hình sai sót, quên vô hiệu hóa tính năng sinh Source Map, toàn bộ cấu trúc thư mục, các đoạn mã gốc ban đầu cùng với các ghi chú nhạy cảm của lập trình viên sẽ bị phơi bày hoàn toàn trước kẻ tấn công.

The screenshot shows the challenge page for 'Javascript - Webpack' on the Root Me CTF platform. The page has a dark theme. At the top, it says 'Javascript - Webpack' with a title bar. Below that, it shows '15 Points' with a code icon, 'Do you know webpack?', 'Author: CanardMandarin, 11 August 2020', 'Level: 1' (indicated by three colored squares), 'Validations: 31788 Challengers, 9%', and 'Note: 5 stars, 1507 Votes' with 'I like' and 'I don't like' buttons. The 'Statement' section says 'Find the password.' with a 'Start the challenge' button. Below that, it lists '1 related resource(s): Webpackjs - Devtool (Exploitation - Web)'. The 'Validation' section shows a green message box saying 'Well done, you won 15 Points' and a blue message box saying 'Don't forget to give your opinion on the challenge by voting :)' with a 'tweet it!' button.

<https://www.root-me.org/en/Challenges/Web-Client/Javascript-Webpack>

- **Solution:** Quá trình khai thác ban đầu được thực hiện bằng cách kiểm tra tệp tin JavaScript cấu trúc giao diện của ứng dụng thông qua Web Developer Tools. Do trình duyệt ghi nhận sự tồn tại của cấu hình Source Map nhưng gặp lỗi hiển thị cục bộ trên giao diện DevTools, ta tiến hành chuyển sang phương pháp tiếp cận trực tiếp. Bằng cách tải xuống tệp tin bản đồ mã nguồn thô từ máy chủ theo đường dẫn tĩnh: /static/js/app.a92c5074dafac0cb6365.js.map, ta thu hồi được một tệp tin định dạng JSON khổng lồ chứa toàn bộ mã nguồn nguyên bản của dự án. Tiến hành phân tích và lục soát tệp tin này bằng Notepad với từ khóa liên quan đến component bị ẩn YouWillNotFindThisRoute, ta định vị được phân đoạn mã nguồn gốc chưa qua xử lý. Tại đây, cấu trúc mã nguồn tiết lộ một thông điệp ghi chú mang tính cảnh báo của nhà phát triển về việc quên tắt tính năng Source Map khi triển khai, đồng thời hardcoded công khai chuỗi Flag cần tìm: BecauseSourceMapsAreGreatForDebuggingButNotForProduction. Sử dụng chuỗi này nộp lên hệ thống để hoàn thành bài lab.



/static/js/app.a92c5074dafac0cb6365.js.map



## XXII. Admin has the power (Cybertalents)

Admin has the power  50 Points  16,400 solved

Administrators only has the power to see the flag , can you be ...



Level: Easy

## 1. Thông tin chung

- **Tên challenge:** Admin has the power
- **Mức độ:** Easy

## 2. Phân tích ban đầu:

- **Mô tả cấu trúc trang web, chức năng các trang chính:** Trang web cung cấp một giao diện đăng nhập cơ bản bao gồm các trường Username và Password. Mục tiêu của thử thách là tìm cách vượt qua cơ chế xác thực hoặc leo thang đặc quyền để có được quyền Administrator và lấy Flag.
- **Kiểm tra source HTML, JS, headers, cookies, ... có gì bất thường:**
  - Kiểm tra mã nguồn HTML tại màn hình đăng nhập phát hiện một dòng comment của lập trình viên vô tình để lộ thông tin tài khoản bảo trì.

```

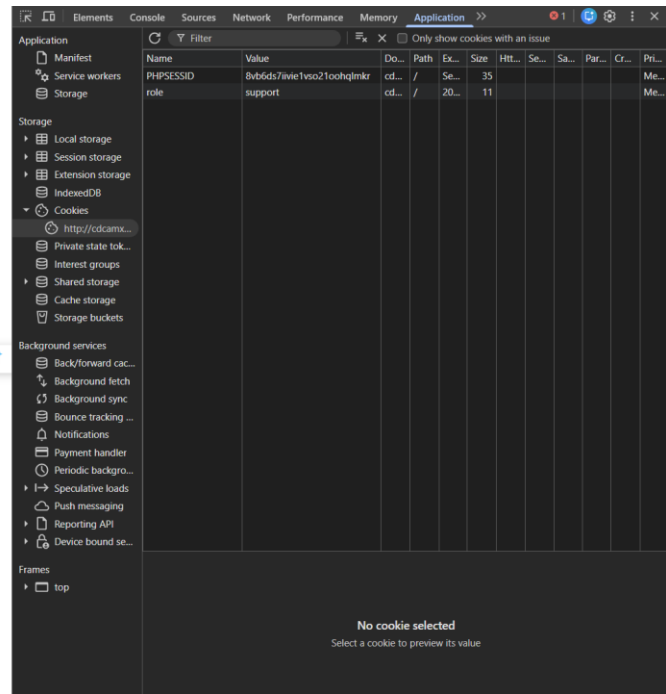
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="utf-8">
5 <meta http-equiv="X-UA-Compatible" content="IE=edge">
6 <meta name="viewport" content="width=device-width, initial-scale=1">
7 <!-- The above 3 meta tags *must* come first in the head; any other head content must come *after* these tags -->
8 <title>Admin Panel</title>
9
10 <!-- Bootstrap -->
11 <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css" integrity="sha384-bWxpmIZAE9fcR45Lpi1bMK+M1xNK18W1mKN1JENRae16RN248xE" crossorigin="anonymous">
12
13 <!-- HTML5 shim and Respond.js for IE8 support of HTML5 elements and media queries -->
14 <!--[if lt IE 9]>
15 <script src="https://oss.maxcdn.com/html5shiv/3.7.3/html5shiv.min.js"></script>
16 <script src="https://oss.maxcdn.com/respond/1.4.2/respond.min.js"></script>
17 <![endif]-->
18 <!-- TODO: remove this line , for maintenance purpose use this info (user:support password:x34245323)-->
19 </head>
20 <body>
21 <div class="container" style="padding-top: 150px;">
22 <div class="row">
23 <div class="col-sm-6 col-sm-offset-3">
24 <h1>Hi admin</h1>
25 <h3>Admin Secret flag : hiadminyouhavethepower </h3>
26 </div>
27 </div>
28 </div>
29 <!-- jQuery (necessary for Bootstrap's JavaScript plugins) -->
30 <script src="https://ajax.googleapis.com/ajax/libs/jquery/1.12.4/jquery.min.js"></script>
31 <!-- Include all compiled plugins (below), or include individual files as needed -->
32 <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js" integrity="sha384-bWxpmIZAE9fcR45Lpi1bMK+M1xNK18W1mKN1JENRae16RN248xE" crossorigin="anonymous"></script>
33 </body>
34 </html>

```

- Sau khi đăng nhập bằng tài khoản này, kiểm tra Cookies của trình duyệt phát hiện ứng dụng sử dụng một cookie có tên là role với giá trị hiện tại là support để quản lý quyền người dùng.

Hi support

Your privilege is support , may be you need better privileges !!



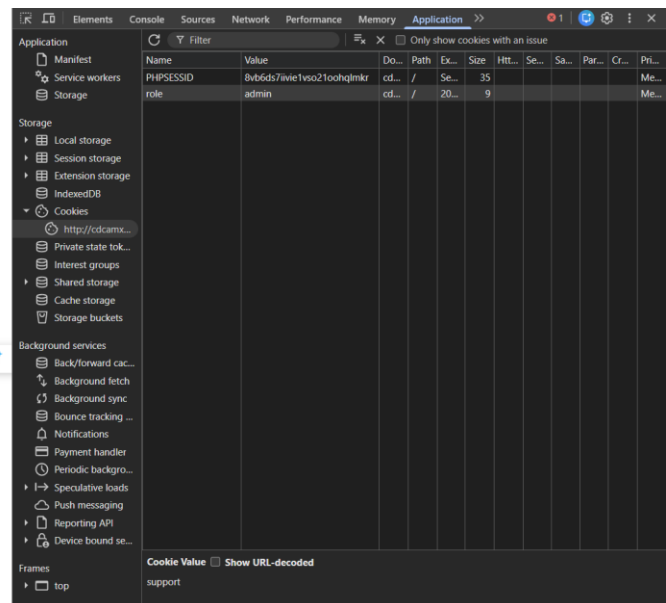
- **Công cụ sử dụng:** Trình duyệt web, Chrome DevTools.
- **Giả thuyết về loại lỗ hổng bảo mật:** Broken Access Control và Information Exposure. Ứng dụng quản lý phân quyền người dùng thông qua dữ liệu Cookie lưu tại Client-side mà không có cơ chế xác thực hoặc mã hóa an toàn từ phía Server.

### 3. Quá trình khai thác:

- **Bước 1:** Tại màn hình đăng nhập, mở DevTools, chuyển sang tab **Sources**. Kiểm tra mã nguồn HTML và thu thập được tài khoản ẩn: Username là support, Password là x34245323.
- **Bước 2:** Sử dụng tài khoản support vừa tìm được để tiến hành đăng nhập vào hệ thống. Việc này giúp khởi tạo một phiên đăng nhập hợp lệ với server để vượt qua lớp xác thực đầu tiên.
- **Bước 3:** Sau khi đăng nhập thành công, mở lại DevTools, điều hướng đến tab **Application** > mục **Storage** > **Cookies**. Nhấn đúp vào giá trị của cookie role và thay đổi từ support thành admin. Nhấn Enter để lưu lại và tải lại trang. Máy chủ đọc giá trị cookie mới, cấp quyền Quản trị viên và hiển thị Flag.

Hi admin

Admin Secret flag : hiadminyouthavethepower



- **Tài liệu tham khảo:**

- OWASP Top 10: Broken Access Control.
- CWE-315: Cleartext Storage of Sensitive Information in a Cookie.
- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor.

#### 4. Mức độ ảnh hưởng:

Lỗ hổng này cho phép kẻ tấn công dễ dàng thực hiện hành vi leo thang đặc quyền. Kẻ tấn công có thể từ một người dùng có quyền hạn thấp hoặc thậm chí chưa xác thực nếu không có bước login phụ tự nâng quyền của mình lên mức cao nhất là Administrator.

Hậu quả là tin tặc có thể chiếm quyền kiểm soát toàn bộ hệ thống, thực hiện các hành vi trái phép như đánh cắp, chỉnh sửa, phá hủy dữ liệu nhạy cảm của người dùng khác và thay đổi cấu hình hệ thống. Việc lộ tài khoản trong mã nguồn cũng tiếp tay trực tiếp cho kẻ tấn công xâm nhập vào bên trong ứng dụng.

#### 5. Cách khắc phục:

- **Quản lý phân quyền tại Server-side:** Tuyệt đối không lưu trữ thông tin phân quyền trực tiếp dưới dạng plaintext ở phía Client như Cookies hay Local Storage. Quyền hạn của người dùng phải được liên kết với một Session ID an toàn, ngẫu nhiên và quản lý hoàn toàn trên cơ sở dữ liệu của Server.
- **Xác thực liên tục:** Mỗi khi người dùng gửi một request truy cập tài nguyên hoặc thực hiện hành động, Server phải kiểm tra lại quyền hạn của Session ID tương ứng thay vì tin tưởng vào dữ liệu gửi từ Client.
- **Rà soát mã nguồn:** Dọn dẹp mã nguồn, loại bỏ toàn bộ các comment chứa thông tin nhạy cảm như hardcoded credentials, API key, ... trước khi triển khai ứng dụng lên môi trường Production.

### XXIII. This is Sparta (Cybertalents)

This is Sparta  50 Points  12,577 solved

Morning has broken today they're fighting in the shade when a...



Level: Easy

#### 1. Thông tin chung

- **Tên challenge:** This is Sparta
- **Mức độ:** Easy

#### 2. Phân tích ban đầu:

- **Mô tả cấu trúc trang web, chức năng các trang chính:** Trang web hiển thị một form đăng nhập tĩnh với hai trường Username và Password, cùng một nút bấm "Hint". Form này dường như không gửi dữ liệu lên server.
- **Kiểm tra source HTML, JS, headers, cookies, ... có gì bất thường:** Khi kiểm tra mã nguồn HTML tab Sources, phát hiện một đoạn mã JavaScript được nhúng trực tiếp ở cuối trang. Đoạn mã này khai báo một mảng biến `_0xae5b` chứa các chuỗi đã bị mã hóa làm rối dưới dạng Hexadecimal nhằm che giấu logic kiểm tra đăng nhập.

```

1 <script>alert("Congratz \n FLAG: {34V1_Scr1Pt_15_Aw3s0me}");
2 </script>
3 </script>
4
5 <script>
6 </script>
7
8 <html>
9 <title>This is Sparta </title>
10 <link href="http://fonts.googleapis.com/css?family=PatuaOne" rel="stylesheet" type="text/css">
11 </link>
12 <body>
13 <div>
14 <div>
15 <div>
16 <div>
17 <div>
18 <div>
19 <div>
20 <div>
21 </div>
22 </div>
23 </div>
24 </div>
25 </div>
26 </div>
27 </div>
28 </div>
29 </div>
30 </div>
31 </div>
32 </div>
33 </div>
34 </div>
35 </div>
36 </div>
37 </div>

```

- **Công cụ sử dụng:** Trình duyệt web, Chrome DevTools.
- **Giả thuyết về loại lỗ hổng bảo mật:** Client-Side Authentication và Hardcoded Credentials. Lập trình viên sử dụng JavaScript để kiểm tra mật khẩu ngay trên trình duyệt và dùng kỹ thuật làm rối để che giấu, nhưng kỹ thuật này rất dễ bị dịch ngược.

### 3. Quá trình khai thác:

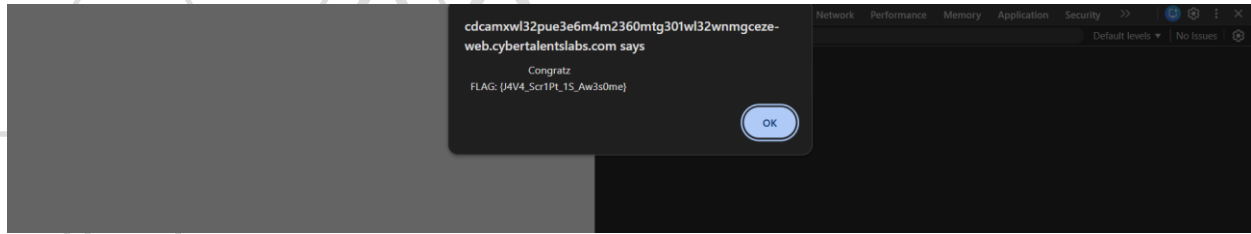
- **Bước 1:** Tại giao diện trang web, nhấn F12 để mở DevTools, chuyển sang tab **Sources** để đọc mã nguồn HTML. Ở cuối file HTML, tìm thấy đoạn mã JavaScript chứa mảng `_0xae5b` bị làm rối.
- **Bước 2:** Copy toàn bộ đoạn khai báo biến mảng `var _0xae5b = [...]` dán vào tab **Console** và nhấn Enter để trình duyệt lưu biến này vào bộ nhớ.
- **Bước 3:** Ở dòng lệnh tiếp theo trong Console, gõ tên biến `_0xae5b` và nhấn Enter. Trình duyệt tự động giải mã Hexadecimal và in ra mảng chuỗi rõ ràng gồm: `['value', 'user', 'getElementById', 'pass', 'Cyber-Talent', 'Congratz \n\n', 'wrong Password']`.

```

> var _0xae5b = [...]
Uncaught runtime.lastError: Could not establish connection. Receiving end does not exist.
> _0xae5b
[
 "value",
 "user",
 "getElementById",
 "pass",
 "Cyber-Talent",
 "Congratz \n\n",
 "wrong Password"
]

```

- **Bước 4:** Phân tích mảng dữ liệu đã giải mã, nhận thấy ứng dụng sẽ lấy giá trị từ trường có id là `user` và `pass` để so sánh với chuỗi **Cyber-Talent**. Quay lại form, nhập Cyber-Talent vào cả ô Username và Password, nhấn Submit để nhận Flag.



#### • Tài liệu tham khảo:

- CWE-602: Client-Side Enforcement of Server-Side Security.
- CWE-798: Use of Hard-coded Credentials.
- Kỹ thuật dịch ngược Obfuscated JavaScript cơ bản.

#### 4. Mức độ ảnh hưởng:

Việc thực hiện cơ chế xác thực ngay tại client-side bằng JavaScript cho phép bất kỳ ai cũng có thể đọc được toàn bộ logic kiểm tra và thông tin đăng nhập.

Kẻ tấn công không cần phải bruteforce hay can thiệp vào máy chủ, chỉ cần đọc mã nguồn HTML và sử dụng Console của trình duyệt để dịch ngược các đoạn code bị làm rối. Hậu quả là cơ chế bảo mật bị vô hiệu hóa hoàn toàn, kẻ tấn công dễ dàng vượt qua bước xác thực để chiếm đoạt tài khoản hoặc truy cập vào nội dung ẩn.

#### 5. Cách khắc phục:

- **Chuyển logic xác thực về Server-Side:** Tuyệt đối không bao giờ thực hiện kiểm tra mật khẩu, quyền hạn hay chứa thông tin nhạy cảm bằng JavaScript ở phía Client. Mọi luồng đăng nhập phải gửi dữ liệu lên máy chủ backend để xử lý và xác thực với cơ sở dữ liệu.
- **Không dùng Obfuscation làm lớp bảo mật chính:** Kỹ thuật làm rối mã nguồn chỉ gây khó khăn một chút cho người đọc chứ không phải là phương pháp bảo mật. Trình duyệt luôn có khả năng hiểu và thực thi các đoạn mã này, do đó kẻ tấn công cũng dễ dàng dịch ngược chúng.

#### XXIV. Share the ideas (Cybertalents)

share the ideas  100 Points  5,536 solved



can you reveal the admin password ?

Level: Medium

#### 1. Thông tin chung

- **Tên challenge:** share the ideas
- **Mức độ:** Medium

#### 2. Phân tích ban đầu:

- **Mô tả cấu trúc trang web, chức năng các trang chính:** Trang web cung cấp tính năng "Share Ideas Online", cho phép người dùng nhập văn bản vào một ô text area và nhấn "Share". Hệ thống sẽ lưu trữ bài đăng này vào cơ sở dữ liệu và hiển thị lại ở danh sách "Latest" ngay bên dưới.
- **Kiểm tra source HTML, JS, headers, cookies, ... có gì bất thường:** Frontend không có thông tin ẩn hay script nào đáng ngờ. Tuy nhiên, việc dữ liệu người dùng nhập vào được lưu trữ và hiển thị lại mở ra khả năng hệ thống tương tác trực tiếp với Database.

## Express your self and share your ideas

Login OR Register



| Latest                                                                                      |                                                                                                                                                             |
|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  Joshua    | Sed vitae orci mattis, vestibulum nisi in, feugiat ante. Donec congue cursus tempor. Curabitur eu neque eu sapien porta dictum.                             |
|  Christina | Praesent nisi nisi, feugiat et diam vitae, porttitor mattis neque. Ut vitae augue id sem vestibulum efficitur.                                              |
|  Maria     | Curabitur massa lacus, dapibus vel quam facilisis, tristique convallis metus. Integer euismod pulvinar aliquam.                                             |
|  Andrew    | Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec laculis, dui at accumsan fringilla, ante odio laculis odio, non facilisis lectus ex nec est. |

- **Công cụ sử dụng:** Trình duyệt web.
- **Giả thuyết về loại lỗ hổng bảo mật:** Stored SQL Injection. Ứng dụng có thể đang sử dụng hệ quản trị cơ sở dữ liệu SQLite và thực thi trực tiếp câu lệnh INSERT với dữ liệu đầu vào từ người dùng mà không qua bước làm sạch hay sử dụng tham số hóa.

### 3. Quá trình khai thác:

- **Bước 1:** Kiểm tra lỗi SQL Injection và trích xuất cấu trúc Database.
  - Do dự đoán backend dùng SQLite, tiến hành khai thác bằng hệ thống sqlite\_master để lấy thông tin các bảng. Lợi dụng toán tử nối chuỗi ||, chèn payload sau vào ô nhập liệu: test' || (SELECT sql FROM sqlite\_master);--
  - Kết quả trả về trên màn hình hiển thị cấu trúc tạo bảng: CREATE TABLE "xde43\_users" ( "id" int(10) NOT NULL, "name" varchar(255) NOT NULL, "email" varchar(255) NOT NULL, "password" varchar(255) NOT NULL, "role" varchar(100) DEFAULT NULL ).
  - Từ đây xác định được bảng mục tiêu là xde43\_users chứa các cột password và role.

## Express your self and share your ideas

Welcome HugndUIT (hugnd.uit@gmail.com)



| Latest                                                                                        |                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  HugndUIT  | testCREATE TABLE "xde43_users" ( "id" int(10) NOT NULL, "name" varchar(255) NOT NULL, "email" varchar(255) NOT NULL, "password" varchar(255) NOT NULL, "role" varchar(100) DEFAULT NULL ) |
|  Joshua    | Sed vitae orci mattis, vestibulum nisi in, feugiat ante. Donec congue cursus tempor. Curabitur eu neque eu sapien porta dictum.                                                           |
|  Christina | Praesent nisi nisi, feugiat et diam vitae, porttitor mattis neque. Ut vitae augue id sem vestibulum efficitur.                                                                            |
|  Maria     | Curabitur massa lacus, dapibus vel quam facilisis, tristique convallis metus. Integer euismod pulvinar aliquam.                                                                           |
|  Andrew    | Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec laculis, dui at accumsan fringilla, ante odio laculis odio, non facilisis lectus ex nec est.                               |

- **Bước 2:** Trích xuất mật khẩu của Admin.
  - Sử dụng thông tin vừa thu thập được, tiếp tục tạo một payload Subquery khác để lấy mật khẩu của tài khoản có role là admin: test' || (SELECT password FROM xde43\_users WHERE role="admin");--

- Nhấn Share. Kết quả trả về trên bài đăng mới nhất chuỗi: testflag245698. Flag cần tìm chính là **flag245698**.

### Express your self and share your ideas

Welcome HugndUIT (hungnd.uit@gmail.com)



| Latest |                                                                                                                                                                                                       |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|        | HugndUIT<br>testflag245698                                                                                                                                                                            |
|        | HugndUIT<br>testCREATE TABLE "xde43_users" ( "id" int(10) NOT NULL, "name" varchar(255) NOT NULL, "email" varchar(255) NOT NULL, "password" varchar(255) NOT NULL, "role" varchar(100) DEFAULT NULL ) |
|        | Joshua<br>Sed vitae orci mattis, vestibulum nisi in, feugiat ante. Donec congue cursus tempor. Curabitur eu neque eu sapien porta dictum.                                                             |
|        | Christina<br>Praesent nisi nisi, feugiat et diam vitae, porttitor mattis neque. Ut vitae augue id sem vestibulum efficitur.                                                                           |
|        | Maria<br>Curabitur massa lacus, dapibus vel quam facilisis, tristique convallis metus. Integer euismod pulvinar aliquam.                                                                              |
|        | Andrew<br>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Donec laculis, dui at accumsan fringilla, ante odio laculis odio, non facilisis lectus ex nec est.                                 |

### Tài liệu tham khảo:

- OWASP Top 10: Injection.
- Kỹ thuật khai thác SQLite Injection.
- CWE-89: Improper Neutralization of Special Elements used in an SQL Command.

### 4. Mức độ ảnh hưởng:

Lỗi hỏng Stored SQL Injection này có mức độ ảnh hưởng cực kỳ nghiêm trọng. Kẻ tấn công có thể chen các truy vấn SQL tùy ý để đọc toàn bộ dữ liệu trong cơ sở dữ liệu, bao gồm thông tin cá nhân, tài khoản, và mật khẩu của quản trị viên cũng như người dùng khác.

Nếu hệ quản trị cơ sở dữ liệu có cấu hình lỏng lẻo, kẻ tấn công thậm chí có thể thực hiện các thao tác ghi, xóa dữ liệu, hoặc leo thang đặc quyền chiếm quyền kiểm soát máy chủ.

### 5. Cách khắc phục

- **Sử dụng Parameterized Queries:** Đây là biện pháp phòng chống SQL Injection hiệu quả nhất. Không bao giờ nối chuỗi trực tiếp dữ liệu người dùng nhập vào câu lệnh SQL.
- **Kiểm tra và làm sạch dữ liệu đầu vào:** Lọc bỏ các ký tự đặc biệt có khả năng làm thay đổi cấu trúc câu lệnh SQL như dấu nháy đơn ', ngoặc đơn (), dấu chấm phẩy ;, dấu gạch ngang --.
- **Áp dụng nguyên tắc quyền hạn tối thiểu:** Cấp quyền truy cập database cho ứng dụng web ở mức thấp nhất chỉ đủ để thực thi các tác vụ cần thiết, hạn chế quyền truy xuất vào các bảng hệ thống (như sqlite\_master).

### XXV. Encrypted Database (Cybertalents)

Encrypted Database 50 Points 4,913 solved

The company hired an inexperienced developer, but he told th...



Level: Easy

### 1. Thông tin chung

- **Tên challenge:** Encrypted Database
- **Mức độ:** Easy

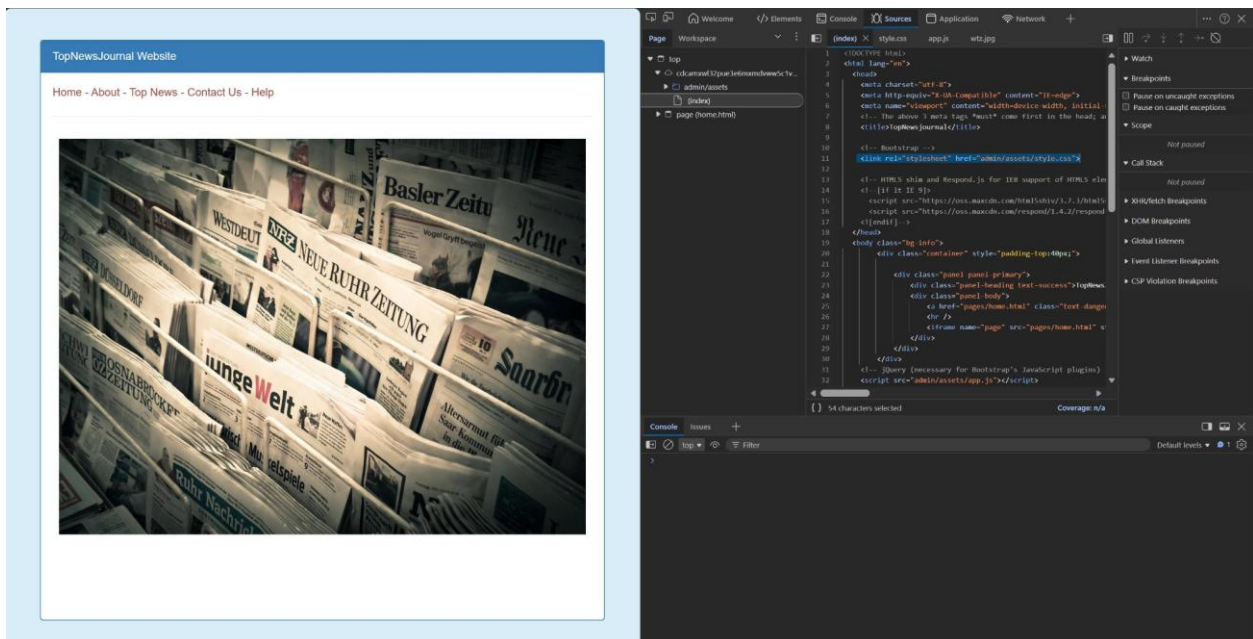
## 2. Phân tích ban đầu:

- **Mô tả cấu trúc trang web, chức năng các trang chính:** Ứng dụng là một trang web tin tức đơn giản (TopNewsJournal). Theo mô tả thử thách, một lập trình viên thiếu kinh nghiệm đã cố tình "giấu" và "mã hóa" cơ sở dữ liệu để bảo mật website. Nhiệm vụ là chứng minh nhận định của anh ta là sai lầm.
- **Kiểm tra source HTML, JS, headers, cookies, ... có gì bất thường:**
  - Kiểm tra mã nguồn trang chủ (Inspect / F12), phát hiện các tài nguyên tĩnh như CSS, JS được nạp từ thư mục admin (ví dụ: admin/assets/style.css). Điều này gợi ý sự tồn tại của thư mục quản trị /admin/ chưa được phân quyền ẩn giấu.

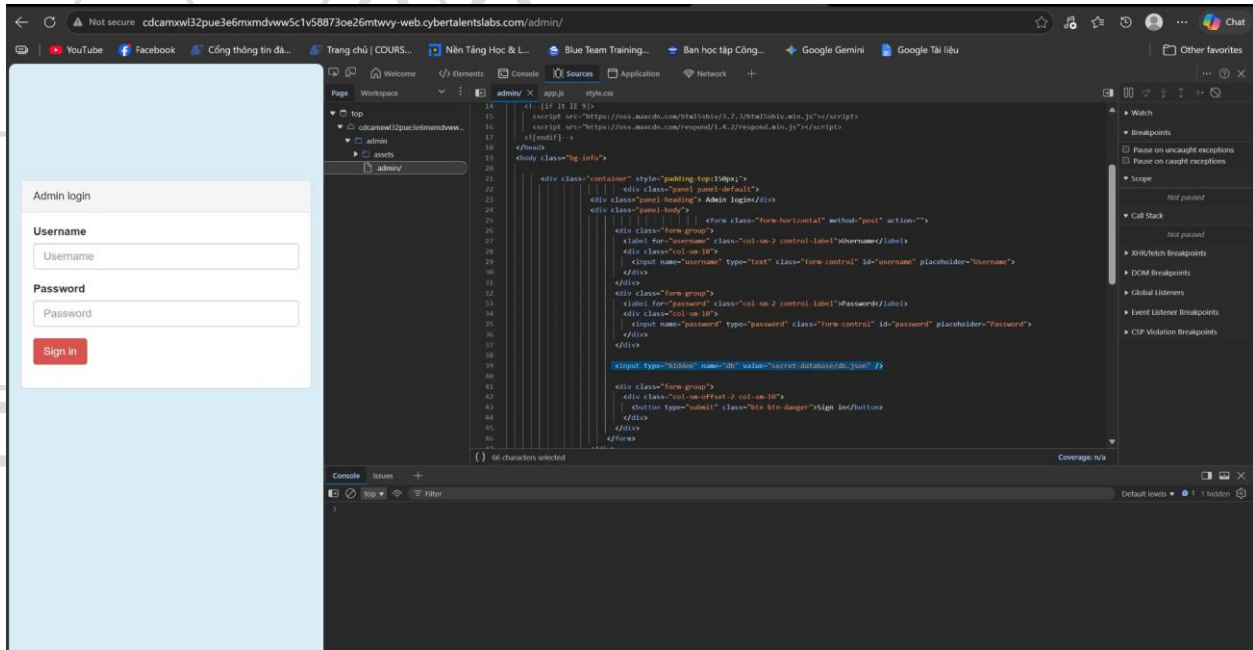
- **Công cụ sử dụng:** Trình duyệt web (Developer Tools) và Công cụ Crack Hash (CrackStation).
- **Giả thuyết về loại lỗ hổng bảo mật:** Lỗi rò rỉ thông tin thông qua mã nguồn frontend, kết hợp với việc kiểm soát truy cập kém (Broken Access Control). Lập trình viên có thể đã để lộ đường dẫn file database trong mã nguồn và sử dụng thuật toán băm lỗi thời thay vì mã hóa thực sự.

## 3. Quá trình khai thác:

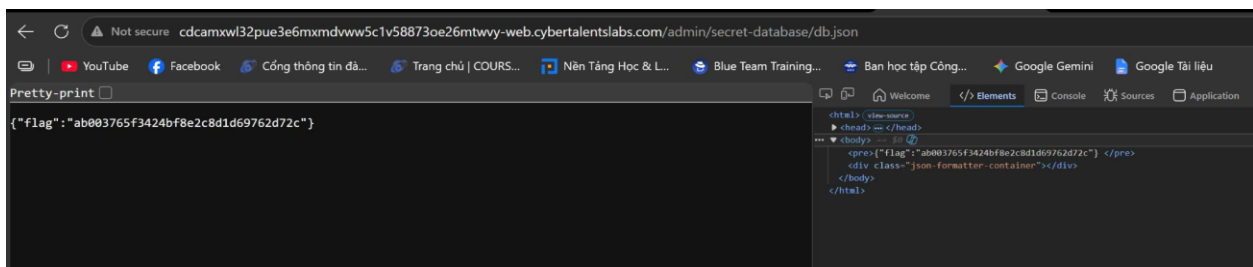
- **Bước 1:** Từ dấu vết ở trang chủ, tiến hành truy cập trực tiếp vào đường dẫn thư mục <http://cdcamxwl32pue3e6mxmdvww5c1v58873oe26mtwvy-web.cybertalentlabs.com//admin/>. Ứng dụng không chặn mà trả về một trang đăng nhập dành cho Admin login).



- **Bước 2:** Tiến hành đọc mã nguồn HTML của form đăng nhập. Phát hiện một thẻ input bị ẩn đi với nội dung nhạy cảm: `<input type="hidden" name="db" value="secret-database/db.json" />`. Lập trình viên đã mắc sai lầm nghiêm trọng khi hardcoded đường dẫn cơ sở dữ liệu trực tiếp vào phía client.



- Bước 3:** Nối đường dẫn vừa tìm được vào URL: `/admin/secret-database/db.json`. Máy chủ cho phép truy cập trực tiếp (Direct Object Reference) và trả về nội dung file JSON chứa cờ: `{"flag": "ab003765f3424bf8e2c8d1d69762d72c"}`

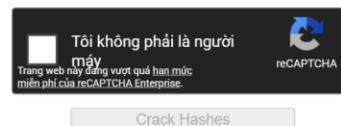


- Bước 4:** Nhận dạng chuỗi `ab003765f3424bf8e2c8d1d69762d72c` là kết quả của hàm băm MD5 (dài 32 ký tự hệ thập lục phân) chứ không phải là dữ liệu được mã hóa hai chiều. Đưa chuỗi hash này vào công cụ CrackStation để thực hiện tấn công từ điển (Dictionary Attack). Kết quả bẻ khóa thành công, thu được bản rõ là `badboy`

### Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

ab003765f3424bf8e2c8d1d69762d72c



Crack Hashes

**Supports:** LM, NTLM, md2, md4, md5, md5(md5\_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1\_bin), QubesV3.1BackupDefaults

| Hash                             | Type | Result |
|----------------------------------|------|--------|
| ab003765f3424bf8e2c8d1d69762d72c | md5  | badboy |

**Color Codes:** Green Exact match, Yellow Partial match, Red Not found.

- Tài liệu tham khảo:**

- OWASP Top 10: Broken Access Control.
- CWE-315: Cleartext Storage of Sensitive Information in a Cookie.

- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor.

#### 4. Mức độ ảnh hưởng:

Lỗi hỏng có mức độ nghiêm trọng cao. Kẻ tấn công không cần bất kỳ đặc quyền nào (không cần đăng nhập) vẫn có thể đọc được toàn bộ cơ sở dữ liệu của hệ thống chỉ bằng cách đọc mã nguồn HTML và truy cập đường dẫn tĩnh.

Việc nhầm lẫn giữa Băm (Hashing - không thể dịch ngược bằng thuật toán) và Mã hóa (Encryption - có thể giải mã bằng khóa) kèm theo việc sử dụng MD5 không có salt khiến dữ liệu dễ dàng bị lộ thông qua các kho dữ liệu băm có sẵn.

#### 5. Cách khắc phục:

- **Loại bỏ thông tin nhạy cảm khỏi Frontend:** Tuyệt đối không lưu trữ các đường dẫn nhạy cảm, cấu hình hệ thống, hoặc thông tin kết nối database trong thẻ `<input type="hidden">`, mã nguồn HTML, hoặc file JavaScript ở phía Client.
- **Kiểm soát truy cập (Access Control):** Các thư mục chứa dữ liệu nhạy cảm như `/secret-database/` không được phép public. Phải cấu hình server (ví dụ qua `.htaccess` hoặc cấu hình Nginx) để chặn truy cập trực tiếp, hoặc đặt file dữ liệu ở ngoài thư mục gốc của Web (Web root).
- **Sử dụng thuật toán mật mã chuẩn:** Nếu cần mã hóa dữ liệu để đọc lại, phải sử dụng các thuật toán mã hóa mạnh (như AES-256). Nếu lưu trữ mật khẩu, cần sử dụng các hàm băm chuyên dụng, sinh ra chuỗi ngẫu nhiên vật cho việc crack (như bcrypt, Argon2) kết hợp với Salt độc nhất cho từng bản ghi. Không sử dụng MD5 cho bất kỳ mục đích bảo mật nào.

## XXVI. Dark Project (Cybertalents)

Dark Project  100 Points  3,442 solved



What kind of Project are you seeking for ?

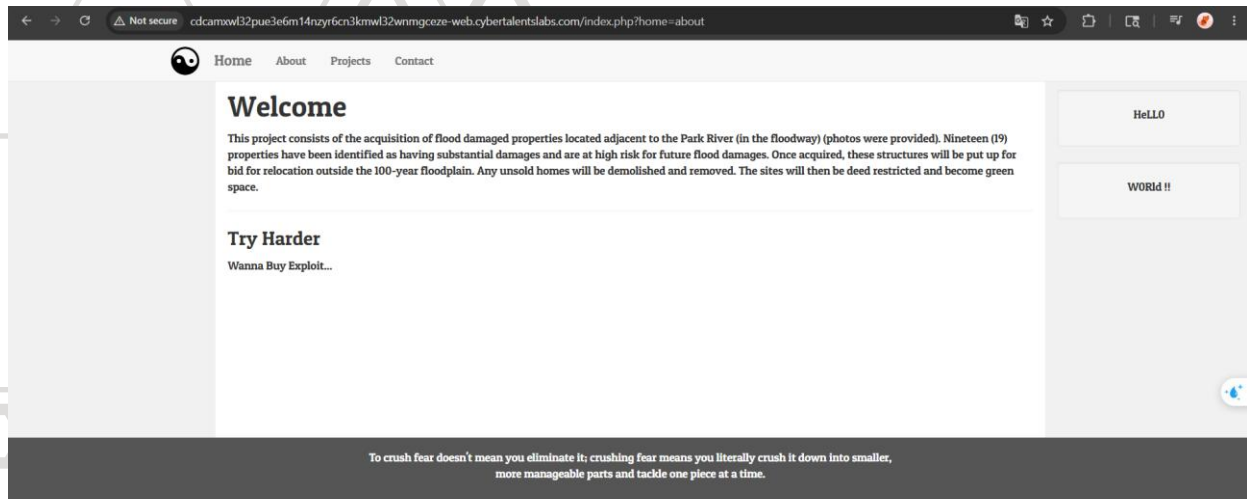
Level: Medium

### 1. Thông tin chung

- **Tên challenge:** Dark Project
- **Mức độ:** Medium

### 2. Phân tích ban đầu:

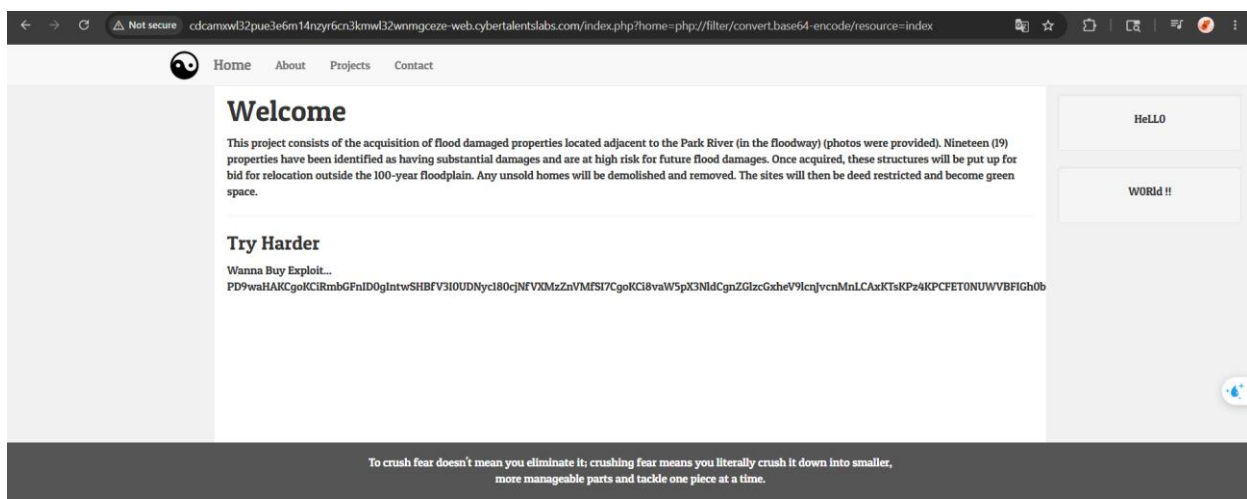
- **Mô tả cấu trúc trang web, chức năng các trang chính:** Trang web hiển thị nội dung tĩnh cơ bản. Tuy nhiên, khi quan sát thanh địa chỉ, phát hiện trang web sử dụng tham số `?home=` để nạp và hiển thị nội dung.



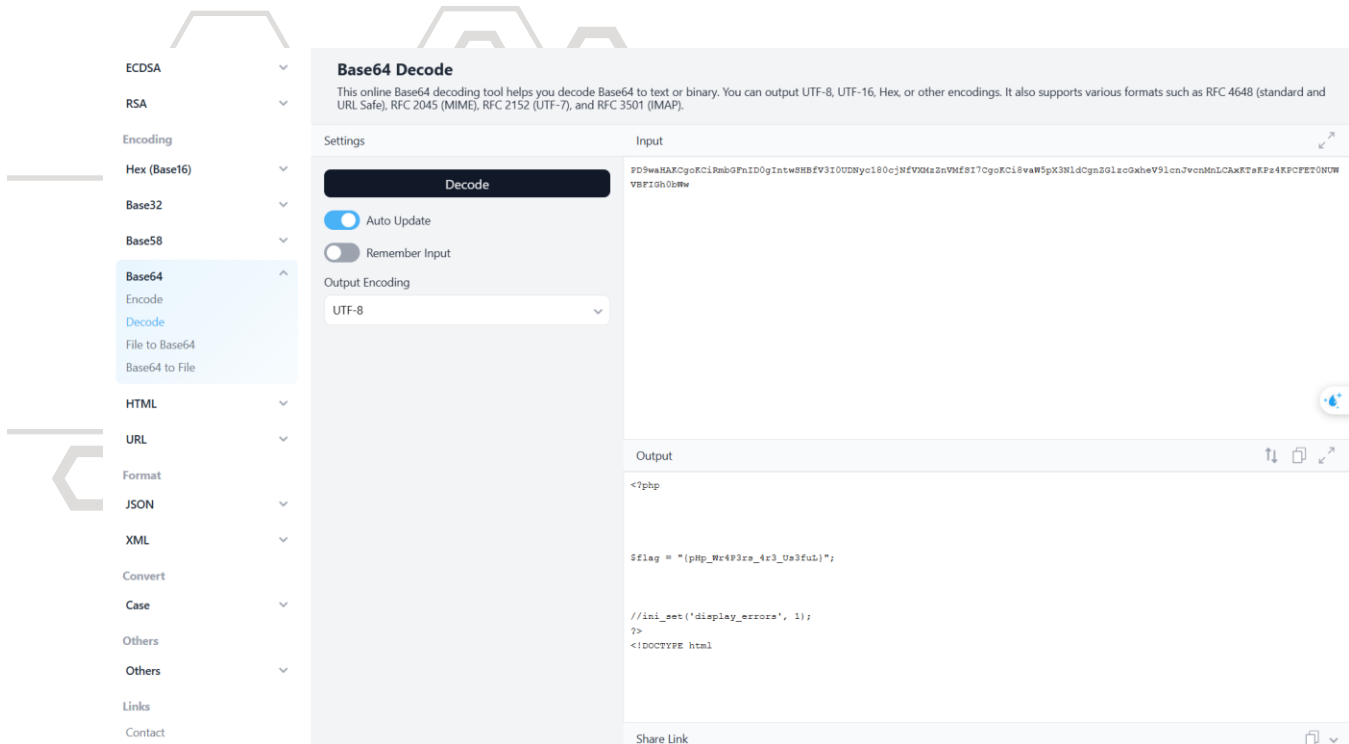
- **Kiểm tra source HTML, JS, headers, cookies, ... có gì bất thường:** Giao diện web không có form nhập liệu hay file ẩn đáng ngờ. Điểm yếu duy nhất nằm ở cơ chế routing thông qua tham số GET trên URL.
- **Công cụ sử dụng:** Trình duyệt web, công cụ giải mã Base64.
- **Giả thuyết về loại lỗ hổng bảo mật:** Local File Inclusion. Máy chủ backend sử dụng PHP có thể đang sử dụng các hàm như include(), require() hoặc file\_get\_contents() để đọc file dựa trên input từ tham số home mà không có cơ chế kiểm tra hoặc lọc an toàn.

### 3. Quá trình khai thác:

- **Bước 1:** Lợi dụng tính năng của PHP Wrappers để can thiệp vào quá trình đọc file của máy chủ. Thay vì truyền tên file thông thường, tiến hành chèn payload sau vào tham số URL để ép máy chủ đọc file index và mã hóa nội dung của nó thành Base64 trước khi thực thi: ?home=php://filter/convert.base64-encode/resource=index



- **Bước 2:** Sao chép toàn bộ chuỗi Base64 này và đưa vào công cụ Base64 Decoder. Kết quả giải mã trả về toàn bộ mã nguồn nguyên thủy của file index.php.



- **Bước 3:** Phân tích mã nguồn PHP vừa giải mã, tìm thấy biến \$flag chứa giá trị cờ tĩnh. Flag của thử thách là: {pHp\_Wr4P3rs\_4r3\_Us3fuL}
- **Tài liệu tham khảo:**
  - OWASP: Local File Inclusion.
  - Kỹ thuật sử dụng bộ lọc PHP Wrappers trong tấn công LFI.
  - CWE-22: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal').

#### 4. Mức độ ảnh hưởng:

Lỗ hổng LFI kết hợp PHP Wrappers cho phép kẻ tấn công đọc được toàn bộ mã nguồn của ứng dụng web.

Từ mã nguồn này, tin tặc có thể lấy được các thông tin cực kỳ nhạy cảm như thông tin kết nối Cơ sở dữ liệu, các khóa API, hoặc tìm ra các lỗ hổng bảo mật khác ẩn sâu bên trong code. Nghiêm trọng hơn, nếu máy chủ cấu hình lỏng lẻo, kẻ tấn công có thể nâng cấp từ LFI thành RCE để chiếm quyền điều khiển hoàn toàn máy chủ.

#### 5. Cách khắc phục:

- **Tránh truyền trực tiếp User Input vào các hàm đọc file:** Tuyệt đối không sử dụng các tham số từ phía người dùng (\$\_GET, \$\_POST, \$\_COOKIE) để đưa thẳng vào các hàm như include(), require().
- **Sử dụng Whitelist:** Nếu bắt buộc phải nạp file động, hãy tạo một danh sách các file được phép nạp ví dụ: if (\$page == 'about') include('about.php');. Nếu input không nằm trong danh sách, máy chủ sẽ từ chối thực thi.
- **Vô hiệu hóa PHP Wrappers không cần thiết:** Tùy chỉnh cấu hình php.ini, xem xét tắt tính năng allow\_url\_include nếu không sử dụng đến để giảm thiểu rủi ro bị lạm dụng các wrapper từ xa.



Webmaster developed a simple script to do cool effects on yo...

Level: Easy

## 1. Thông tin chung

- **Tên challenge:** Cool Name Effect
- **Mức độ:** Easy

## 2. Phân tích ban đầu:

- **Mô tả cấu trúc trang web, chức năng các trang chính:** Trang web hiển thị một form nhập liệu đơn giản.

Name

your name here

- **Kiểm tra source HTML, JS, headers, cookies, ... có gì bất thường:** Khi kiểm tra mã nguồn HTML, phát hiện một đoạn mã JavaScript bị nén và làm rối bằng kỹ thuật Packer, bắt đầu bằng hàm `eval(function(p,a,c,k,e,d){...})`.

```

<!DOCTYPE html>
<html lang="en">
 <head>
 <meta charset="utf-8">
 <meta http-equiv="X-UA-Compatible" content="IE=edge">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <!-- The above 3 tags must come first in the head; any other head content must come "after" these tags -->
 <title>JavaScript</title>

 <!-- Bootstrap -->
 <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css" integrity="sha384-BVYiiSIfeK1dGmJRAKycuWHarG320mUcw7on3RYdg4Va+PmStsz/K68vbdEjhu" crossorigin="anonym

 <!-- HTML5 shim and Respond.js for IE8 support of HTML5 elements and media queries -->
 <!--[if lt IE 9]>
 <script src="https://oss.maxcdn.com/html5shiv/3.7.3/html5shiv.min.js"></script>
 <script src="https://oss.maxcdn.com/respond/1.4.2/respond.min.js"></script>
 <![endif]>

 <!-- jQuery (necessary for Bootstrap's JavaScript plugins) -->
 <script src="https://ajax.googleapis.com/ajax/libs/jquery/1.12.4/jquery.min.js"></script>
 <!-- Latest compiled and minified JavaScript -->
 <script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js" integrity="sha384-Tc51Qib27qvyjSMfHf0MukFvVzZuPnC7A712cWNIpG9wGCD8wGNIcPD71xa" crossorigin="anonymous"></script>

 eval(function(p,a,c,k,e,d){(e=function(c){return(c<<?e(parseInt(c/a)))+(c<<?a):35?String.fromCharCode(c+29):c.toString(36)});if(!''.replace(/\s/,String)){while(c--){d[e(c)]=k[c]||e(c)}k=[function(e){ret

 </script>

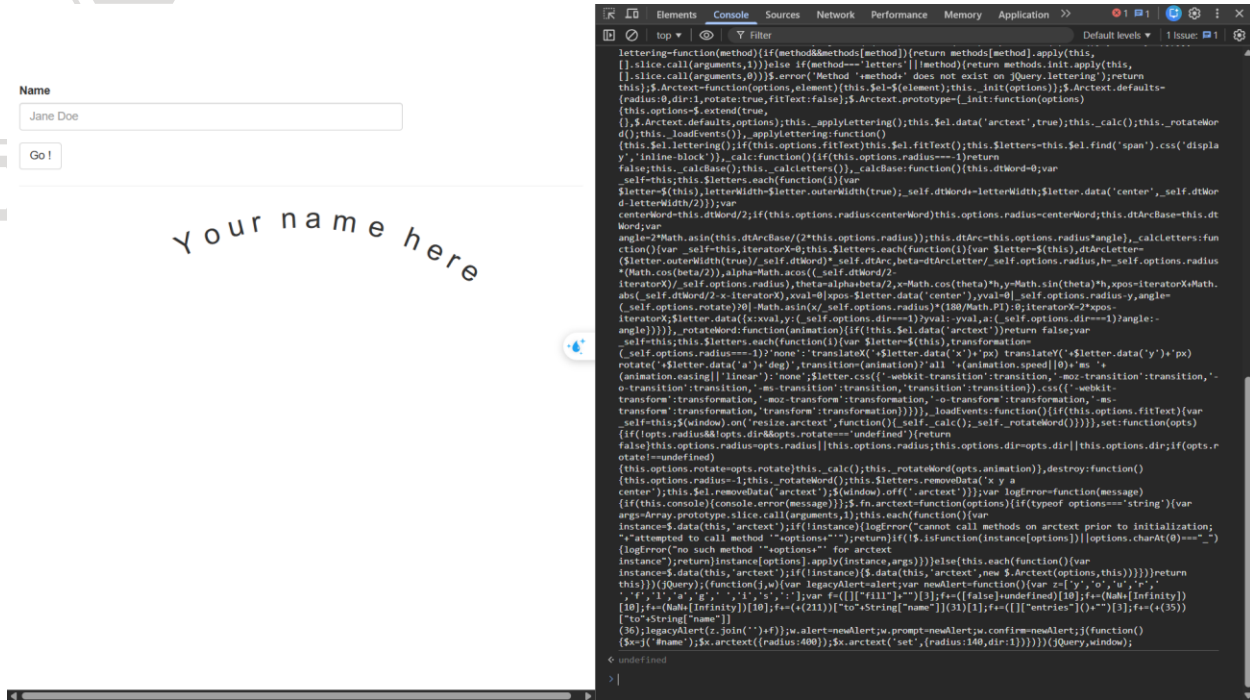
 </head>
 <body>
 <div class="container" style="padding-top: 100px;">
 <div class="row">
 <div class="col-sm-8 col-sm-offset-2">
 <form class="form-inline" method="get">
 <div class="form-group" style="width:80%;">
 <label for="exampleInputName2">Name</label>
 <input style="width: 85%;" type="text" name="name" class="form-control" id="exampleInputName2" placeholder="Jane Doe">
 </div>
 <button type="submit" class="btn btn-default">Go !</button>
 </form>
 <hr />
 <div id="name" class="text-center">
 Your name here
 </div>
 </div>
 </div>
 </body>
 </html>

```

- **Công cụ sử dụng:** Trình duyệt web, Chrome DevTools.
- **Giả thuyết về loại lỗ hổng bảo mật:** Client-Side Logic Exposure / Security through Obscurity. Tác giả cố tình giấu thông tin nhạy cảm và logic ứng dụng ở phía Client bằng cách làm rối mã nguồn, cho rằng người dùng không thể đọc được. Tuy nhiên, mã JavaScript luôn có thể bị dịch ngược ngay trên trình duyệt.

## 3. Quá trình khai thác:

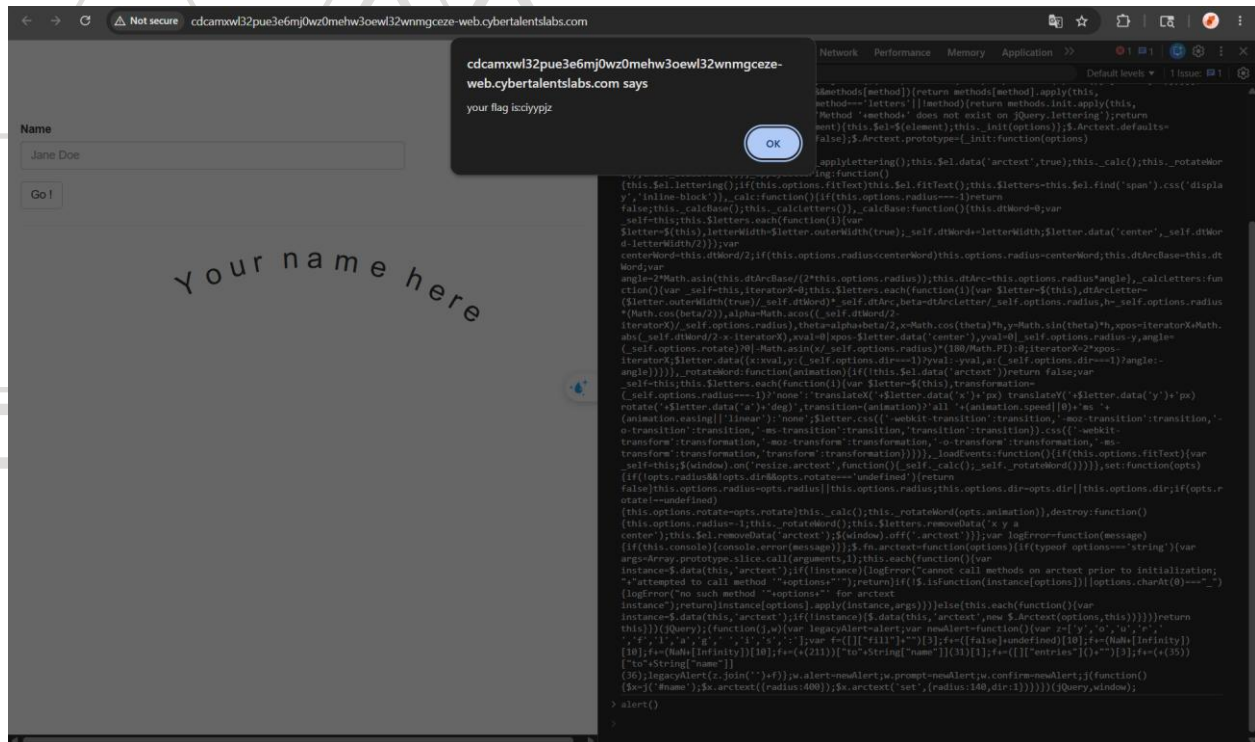
- **Bước 1:** Trích xuất đoạn mã JavaScript bị làm rối từ mã nguồn HTML. Thay vì để trình duyệt tự động thực thi bằng hàm eval(), tiến hành can thiệp bằng cách thay chữ eval thành console.log() trong tab Console để in ra mã nguồn nguyên thủy.
- **Bước 2:** Phân tích mã nguồn đã được giải mã. Nhận thấy lập trình viên đã viết một hàm tự định nghĩa newAlert và dùng nó để ghi đè lên các hàm mặc định của trình duyệt như alert, prompt, confirm.



- **Bước 3:** Phân tích logic của hàm newAlert. Hàm này sử dụng kỹ thuật JavaScript Type Juggling lợi dụng việc ép kiểu dữ liệu ngầm định trong JS như cộng chuỗi với mảng, ép kiểu NaN, Infinity để từng bước lắp ráp ra một biến f chứa nội dung Flag. Sau đó, nó sẽ in biến f này ra màn hình.

```
this}})(jQuery);(function(j,w){var legacyAlert=alert;var newAlert=function(){var z=['y','o','u','r',' ','f','l','a','g',' ',' ','i','s',' ':''];var f=([["fill"]+""])[3];f+=([false]+undefined)[10];f+=(NaN+[Infinity])[10];f+=(NaN+[Infinity])[10];f+=(+211)[["to"+String["name"]]](31)[1];f+=([["entries"]()+""])[3];f+=(+35)[["to"+String["name"]]](36);legacyAlert(z.join('')+f);w.alert=newAlert;w.prompt=newAlert;w.confirm=newAlert;j(function(){var x=j('#name');x.arcxtext({radius:400});x.arcxtext('set',{radius:140,dir:1})});})(jQuery>window);
```

- **Bước 4:** Lợi dụng việc các hàm mặc định đã bị ghi đè, trực tiếp gõ lệnh alert() vào tab Console và nhấn Enter để kích hoạt logic bị giấu. Hệ thống bật lên hộp thoại hiển thị cờ. Flag thu được là: **ciyyypjz**



### • Tài liệu tham khảo:

- CWE-602: Client-Side Enforcement of Server-Side Security.
- Kỹ thuật dịch ngược JS và JS Type Juggling.

## 4. Mức độ ảnh hưởng:

Việc đặt niềm tin vào các đoạn mã JavaScript bị làm rối để bảo vệ dữ liệu nhạy cảm là một sai lầm nghiêm trọng về thiết kế bảo mật.

Bất kỳ logic nào chạy trên máy khách đều có thể bị người dùng xem, chỉnh sửa và dịch ngược hoàn toàn. Kẻ tấn công có thể dễ dàng lấy được các thông tin quan trọng như khóa mã hóa ẩn, thuật toán sinh token, hoặc như trong trường hợp này là lấy trực tiếp Flag mà không cần phải tương tác với Server.

## 5. Cách khắc phục:

- **Chuyển logic quan trọng về Server-Side:** Không bao giờ lưu trữ mật khẩu, hoặc các logic kiểm tra bảo mật, phân quyền ở phía trình duyệt người dùng. Mọi quá trình kiểm tra phải được thực hiện trên máy chủ.
- **Không coi Obfuscation là bảo mật:** Làm rối mã nguồn chỉ giúp chống việc sao chép bản quyền phần mềm ở một mức độ nhất định, hoàn toàn không phải là một phương pháp mã hóa hay bảo mật dữ liệu an toàn.

## XXVIII. Who am i? (Cybertalents)

who am i? 50 Points 4,672 solved

Do not Start a fight you can not stop it



Level: Easy

## 1. Thông tin chung

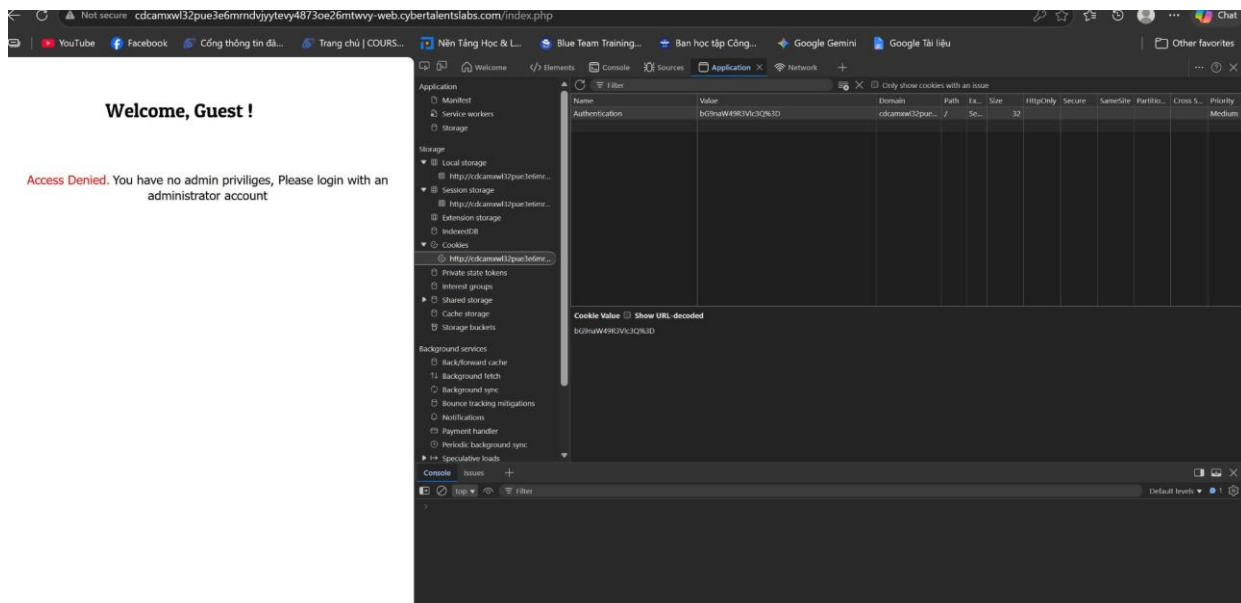
- **Tên challenge:** Who Am I?
- **Mức độ:** Easy

## 2. Phân tích ban đầu

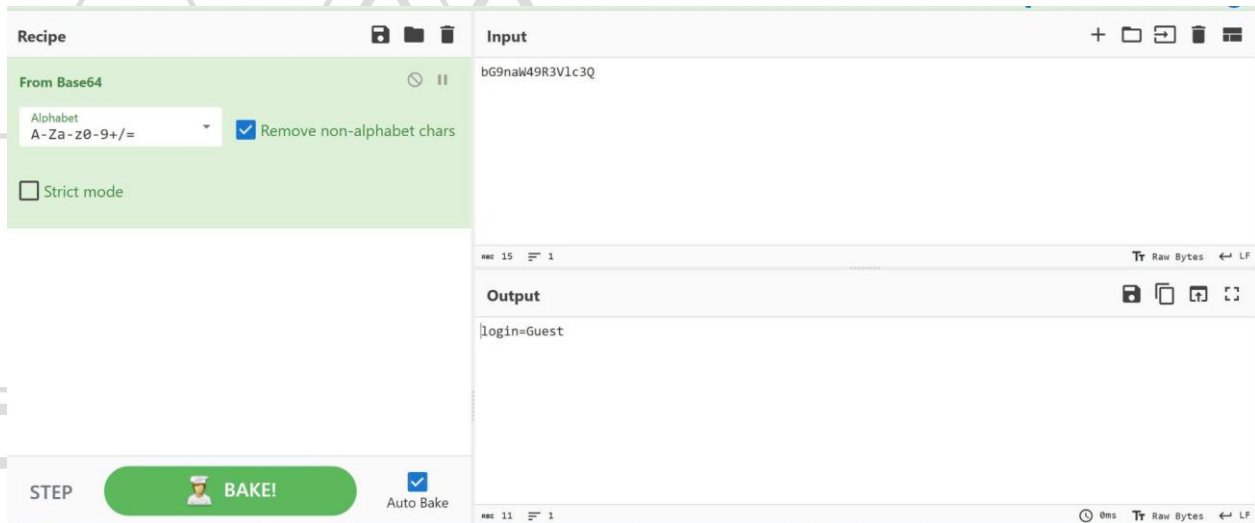
- **Mô tả cấu trúc trang web, chức năng các trang chính:** Ứng dụng cung cấp một giao diện đăng nhập panel quản trị (index.php). Mặc định khi chưa đăng nhập hoặc đăng nhập với quyền thấp, hệ thống hiển thị thông báo *"Access Denied. You have no admin privileges, Please login with an administrator account"*.
- **Kiểm tra source HTML, JS, headers, cookies, ... có gì bất thường:**
  - Khi kiểm tra mã nguồn trang index.php (tab Sources), phát hiện thông tin tài khoản mặc định của Guest được để lại dưới dạng comment HTML.
  - Sau khi dùng tài khoản này để đăng nhập, hệ thống tạo ra một cookie có tên là Authentication với giá trị băm mã hóa dạng chuỗi.
- **Công cụ sử dụng:** Trình duyệt web (Developer Tools - tab Application) và công cụ biến đổi mã dòng lệnh CyberChef.
- **Giả thuyết về loại lỗ hổng bảo mật:** Lỗi Broken Authentication / Cookie Tampering. Hệ thống quản lý phiên mã nguồn mở hoặc tự phát triển này đang tin tưởng hoàn toàn vào dữ liệu lưu dưới client cookie để phân quyền mà không có cơ chế ký số (Digital Signature) hay kiểm tra tính toàn vẹn ở phía Server.

## 3. Quá trình khai thác

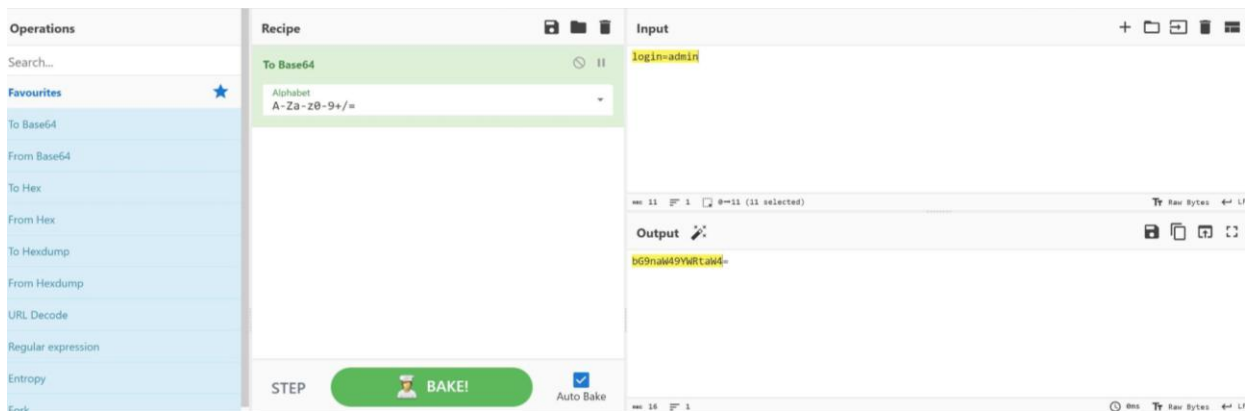
- **Bước 1:** Đăng nhập vào hệ thống bằng tài khoản Guest lấy từ comment HTML. Mở giao diện F12, chuyển sang tab Application > Cookies, phát hiện cookie Authentication có giá trị đã bị URL-encode là bG9naW49R3Vlc3Q%3D.



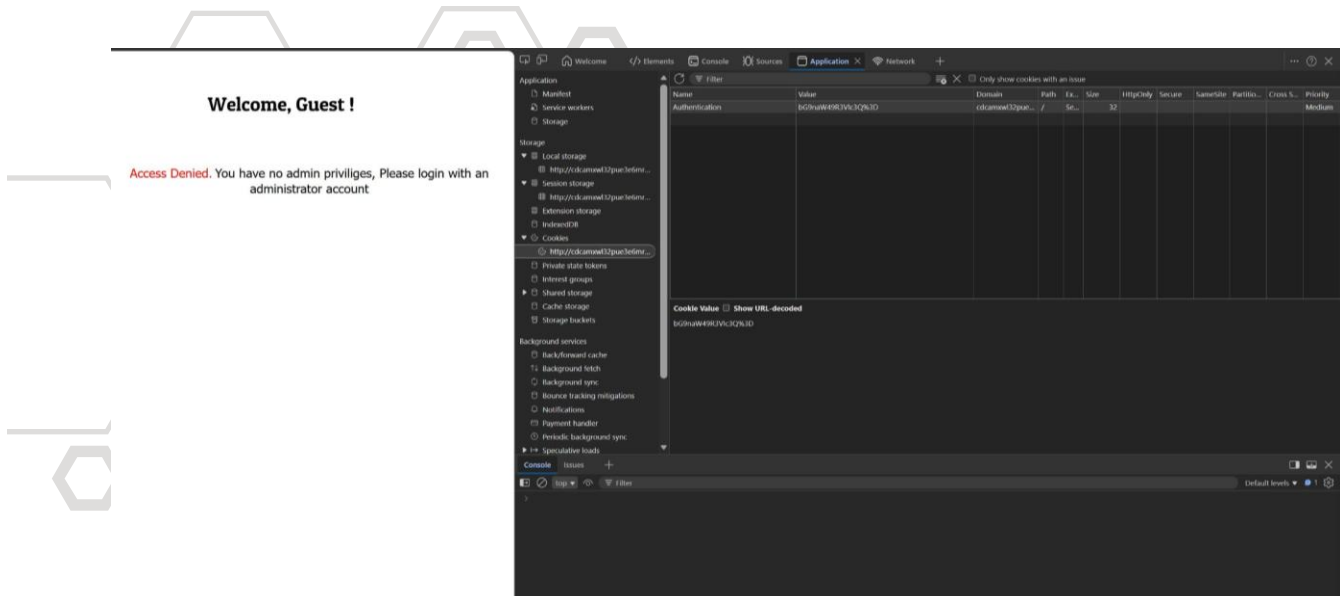
- **Bước 2:** Bỏ ký tự %3D ở đuôi chuỗi để đưa về dạng Base64 nguyên bản: bG9naW49R3Vlc3Q. Đưa chuỗi này vào công cụ CyberChef xài hàm "From Base64", kết quả giải mã trả về chuỗi plain-text có cấu trúc: login=Guest



- **Bước 3:** Tiến hành giả mạo quyền bằng cách thay đổi giá trị của tham số từ Guest sang tài khoản đặc quyền cao nhất. Thay đổi chuỗi thành: login=admin. Đưa chuỗi login=admin này qua bộ mã hóa **To Base64** trên CyberChef, thu được kết quả: bG9naW49YWRtaW4=.



- **Bước 4:** Thực hiện URL-encode dấu = ở cuối chuỗi mã hóa mới thành %3D để khớp định dạng của trình duyệt, ta được chuỗi hoàn chỉnh: bG9naW49YWRtaW4%3D. Quay lại tab Cookies trên trình duyệt, click đúp vào giá trị của cookie Authentication và thay bằng chuỗi admin vừa tạo.
- **Bước 5:** Nhấn F5 để tải lại trang web. Lúc này hệ thống nhận diện cookie mới và hiểu nhầm phiên làm việc thuộc về Quản trị viên. Giao diện thay đổi thành "Welcome, Administrator!" và hiển thị Flag thành công: FLa{B@D\_4uTh1Nt1C4Ti0n}.



#### 4. Mức độ ảnh hưởng

- Lỗ hổng thuộc nhóm kiểm soát truy cập và xác thực yếu, mức độ ảnh hưởng rất nghiêm trọng. Kẻ tấn công có thể dễ dàng leo thang đặc quyền từ một tài khoản thông thường lên thẳng quyền Quyền quản trị tối cao (Admin) chỉ bằng vài thao tác thay đổi chuỗi Base64 đơn giản.
- Việc phơi bày thông tin tài khoản Guest trong comment của mã nguồn tạo điều kiện cho kẻ tấn công thực hiện bước trinh sát ban đầu mà không gặp bất kỳ trở ngại nào.

#### 5. Cách khắc phục

- **Sử dụng cơ chế quản lý Session an toàn:** Server-side session nên sử dụng các chuỗi ID ngẫu nhiên, có độ dài lớn, không mang ý nghĩa logic cụ thể (như login=Guest) và được lưu trữ an toàn trên server (ví dụ: Redis, Database). Client chỉ giữ duy nhất một chuỗi ID ngẫu nhiên đó.
- **Áp dụng Ký số (Cryptographic Signing):** Nếu bắt buộc phải lưu thông tin phân quyền dưới Client (như cơ chế Token), phải sử dụng các chuẩn bảo mật có chữ ký bảo vệ tính toàn vẹn (ví dụ: **JSON Web Token - JWT**) kèm theo một khóa bí mật (Secret Key) đủ mạnh ở phía Server. Khi đó, bất kỳ sự thay đổi nào từ Guest sang admin dưới client đều sẽ làm hỏng chữ ký và bị Server từ chối ngay lập tức.
- **Rà soát mã nguồn:** Gỡ bỏ toàn bộ comment, ghi chú của lập trình viên chứa thông tin tài khoản thử nghiệm hoặc cơ chế logic nội bộ trước khi deploy sản phẩm lên môi trường thực tế.

### XXIX. Iam Legend (Cybertalents)

Iam Legend  50 Points  6,257 solved

If I am a legend, then why am I so lonely?

Flag Format : FLAG{}



Level: Easy

#### 1. Thông tin chung

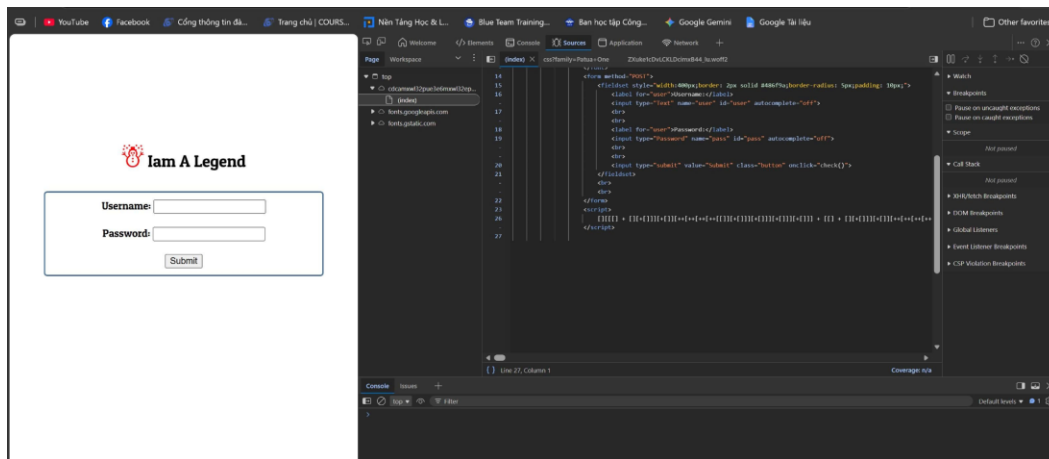
- Tên challenge: Iam Legend
- Mức độ: easy

**2. Phân tích ban đầu Mô tả cấu trúc trang web, chức năng các trang chính:** Ứng dụng cung cấp một biểu mẫu đăng nhập (Login Form) rất đơn giản gồm hai trường nhập liệu chính: Username và Password kèm theo một nút Submit.

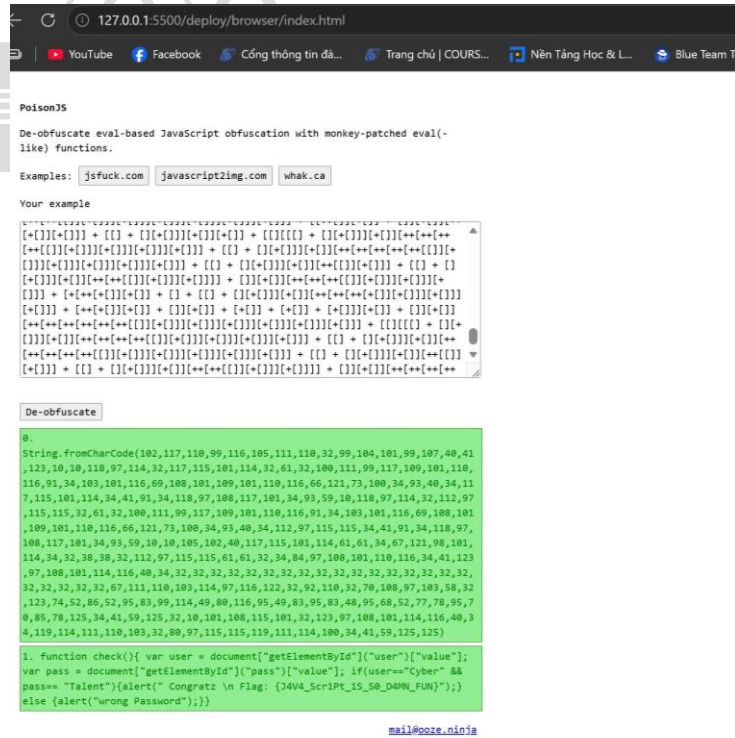
- **Kiểm tra source HTML, JS, headers, cookies, ... có gì bất thường**
  - Khi kiểm tra mã nguồn HTML (tab Sources hoặc Elements) hoặc tab Console, phát hiện một khối mã JavaScript bị làm rối (obfuscated) hoàn toàn bằng kỹ thuật **JSFuck**, chỉ sử dụng đúng 6 ký tự đặc trưng là [, ], (, ), +, !.
  - Đoạn mã này được nhúng trực tiếp trong thẻ <script> ở client-side và sẽ được thực thi ngay khi biểu mẫu được tương tác hoặc tải trang.
  - Công cụ sử dụng: **DevTools** (Trình duyệt Web), Công cụ giải mã **PoisonJS** chạy trên môi trường mã nguồn mở Local.
- **Giả thuyết về loại lỗ hổng bảo mật:** Lỗ hổng **Insecure Obfuscation / Client-Side Authentication**. Hệ thống thực hiện việc kiểm tra tài khoản và mật khẩu trực tiếp ở trình duyệt của người dùng thay vì gửi về server xử lý. Lập trình viên cố tình che giấu thông tin đăng nhập (hardcoded credentials) bằng cách dùng JSFuck với suy nghĩ người dùng không thể đọc được, vi phạm nguyên tắc bảo mật cơ bản.

### 3. Quá trình khai thác

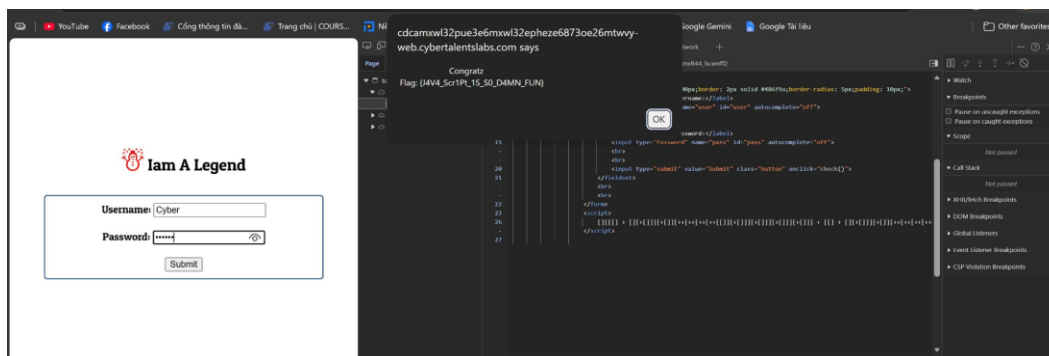
- Các bước thực thực hiện và minh chứng (ảnh chụp màn hình,...)
  - **Bước 1:** Truy cập vào liên kết thử thách, kiểm tra tab Console và Sources của trình duyệt để xác nhận sự tồn tại của khối mã JSFuck loằng ngoằng nằm trong thẻ <script> ngay dưới form đăng nhập.



- **Bước 2:** Sao chép toàn bộ khối mã ký tự đặc biệt này (bỏ qua phần chuỗi lỗi hệ thống không liên quan) và dán vào công cụ de-obfuscate **PoisonJS** đang được dựng sẵn dưới máy local (127.0.0.1:5500). Nhấn nút De-obfuscate để đảo ngược quá trình làm rối chuỗi. Kết quả trả về một hàm kiểm tra bản rõ.



- **Bước 3:** Thu thập được thông tin tài khoản quản trị trực tiếp từ mã nguồn sau khi giải mã: Username: Cyber và Password: Talent. Quay lại giao diện trang web thử thách, điền cặp thông tin này vào biểu mẫu và nhấn nút Submit. Trình duyệt lập tức kích hoạt hàm alert thành công và hiển thị Flag cần tìm: **J4V4\_Scr1Pt\_1S\_S0\_D4MN\_FUN**.



**4. Mức độ ảnh hưởng:** Lỗ hổng này có tác động bảo mật cực kỳ nghiêm trọng đối với cơ chế kiểm soát truy cập. Do toàn bộ logic xác thực và thông tin tài khoản được lưu trữ cố định (hardcoded) ở phía giao diện người dùng (Client-side), bất kỳ ai cũng có thể đọc vị, giải mã ngược và lấy được thông tin đăng nhập tối cao mà không cần vượt qua bất kỳ lớp tường lửa hay cơ chế kiểm soát an toàn nào từ phía máy chủ. Điều này khiến form đăng nhập hoàn toàn vô dụng.

## 5. Cách khắc phục

- **Chuyển toàn bộ logic xác thực về Server-side:** Mọi quá trình kiểm tra tài khoản, mật khẩu và phê duyệt quyền hạn bắt buộc phải được xử lý bảo mật trên máy chủ (Backend). Tuyệt đối không thực hiện các câu lệnh so sánh điều kiện nhảy cảm dạng `if (user === '...' && pass === '...')` ở trình duyệt Client.

- **Không hardcode thông tin nhạy cảm:** Thông tin mật khẩu phải được băm an toàn và đối chiếu trong cơ sở dữ liệu được bảo vệ nghiêm ngặt ở phía máy chủ.
- **Không lạm dụng Obfuscation thay thế cho mã hóa:** Kỹ thuật làm rối mã (như JSFuck) chỉ có tác dụng kéo dài thời gian đọc hiểu chuỗi chứ hoàn toàn không có tính năng bảo mật hay mã hóa dữ liệu. Không áp dụng nó để bảo vệ các tài sản hoặc dữ liệu bí mật của ứng dụng.

### XXX. Big Number (Cybertalents)

Big Number   100 Points  1,507 solved



a big number is needed to save the world

Level: Medium

#### 1. Thông tin chung

- Tên challenge: Big Number
- Mức độ: easy

#### 2. Phân tích ban đầu

- **Mô tả cấu trúc trang web, chức năng các trang chính:** Trang web có giao diện cực kỳ tối giản, cung cấp một biểu mẫu (form) nhập mật khẩu với yêu cầu: cung cấp một con số có giá trị lớn hơn 999. Trên giao diện có kèm một đường dẫn wanna source? để xem mã nguồn xử lý.
- **Kiểm tra source HTML, JS, headers, cookies, ... có gì bất thường**
  - Khi bấm vào xem mã nguồn, hệ thống trả về đoạn code PHP xử lý logic ở Backend:

```
if (isset($_GET['password'])) {

 if (is_numeric($_GET['password'])) {

 if (strlen($_GET['password']) < 4) {

 if ($_GET['password'] > 999)

 die($flag);
```

- **Phân tích logic:** Backend yêu cầu đầu vào password phải thỏa mãn đồng thời 3 điều kiện khắt khe:
  1. Phải là định dạng số (is\_numeric() == true).
  2. Độ dài chuỗi nhập vào phải nhỏ hơn 4 ký tự (strlen() < 4). Tức là tối đa chỉ được nhập 3 ký tự.
  3. Giá trị thực của số đó phải lớn hơn 999 (> 999).
- **Công cụ sử dụng:** DevTools (Trình duyệt Web), RapidTables (để quy đổi hệ cơ số).
- **Giả thuyết về loại lỗ hổng bảo mật:** Lỗ hổng **Logic Flaw / Type Juggling trong PHP**. code sử dụng hàm is\_numeric() để kiểm tra số, nhưng trong PHP, hàm này không chỉ chấp nhận số thập phân bình thường (như 100, 999) mà còn chấp nhận cả các định dạng biểu diễn số khác như số hệ thập lục phân.

#### 3. Quá trình khai thác

Các bước thực thực hiện và minh chứng (ảnh chụp màn hình,...)

- **Bước 1:** Đọc source code PHP, xác định bài toán là cần tìm một chuỗi tối đa 3 ký tự nhưng khi chuyển thành số thì lớn hơn 999.



Help The World With a Big Number

You can help the world by providing 3 numbers can exceed 999 in value:

Password:

Enter password

..... wanna source?

Submit

```
<?php if (isset($_GET['password'])) {
 if (is_numeric($_GET['password'])) {
 if (strlen($_GET['password']) < 4) {
 if ($_GET['password'] > 999)
 die($flag);
 else
 print 'Too little';
 } else
 print 'Too long';
 } else
 print 'Password is not numeric';
 }
}
```

- **Bước 2:** Lên công cụ RapidTables chuyển số 1000 (hệ thập phân) sang hệ Hex (hệ thập lục phân) và thu được kết quả là 3E8 - vừa đúng 3 ký tự.

### Decimal to Hexadecimal converter

From To

Decimal Hexadecimal

Enter decimal number

1000 10

= Convert × Reset ↕ Swap

Hex result

(1000)<sub>10</sub> = (3E8)<sub>16</sub>

Hex number (3 digits)

3E8 16

Hex signed 2's complement (4 digits)

03E8 16

Binary number (10 digits)

1111101000 2

☐ Digit grouping

- **Bước 3:** Nhập chuỗi 3e8 vào ô Password và nhấn Submit. Trình duyệt tải lại và trả về Flag thành công: **FLAG{Yes\_You\_C4n\_Use\_Exp0nentialL}**.

## Help The World With a Big Number

You can help the world by providing 3 numbers can exceed 999 in value:

Password:

..... wanna [source?](#)

FLAG{Yes\_Y0u\_C4n\_Use\_Exp0nential}

**4. Mức độ ảnh hưởng:** Lỗ hổng logic này cho phép kẻ tấn công dễ dàng vượt qua các cơ chế kiểm tra tính hợp lệ của dữ liệu đầu vào (Input Validation). Nếu ứng dụng sử dụng các con số này cho các nghiệp vụ nhạy cảm (như tính toán tiền tệ, giới hạn số lượng mua hàng, chuyển tiền), kẻ tấn công có thể chèn số mũ khoa học (ví dụ 1e9) để tạo ra các giá trị khổng lồ, gây thiệt hại nghiêm trọng về tài chính hoặc phá vỡ hoàn toàn luồng xử lý dữ liệu của cơ sở dữ liệu.

### 5. Cách khắc phục

**Kiểm tra kiểu dữ liệu nghiêm ngặt:** Thay vì dùng `is_numeric()` (quá lỏng lẻo vì nhận cả số mũ), nên sử dụng các biểu thức chính quy (Regex) hoặc hàm ép kiểu để đảm bảo người dùng chỉ được nhập các chữ số thập phân cơ bản [0-9].